

Assignment_05_CNN

September 25, 2026

1 Assignment 05 - CNN Depth and Framework Comparison

1.1 0. Project overview

The purpose of A5 is not simply to obtain a high test score. The purpose is to understand what happens when a convolutional neural network is made deeper, and to understand how the same learning idea is expressed through two different deep-learning frameworks: TensorFlow/Keras and PyTorch.

The report therefore connects five levels of reasoning:

Data representation

↓

Data quality and preprocessing

↓

CNN architecture and tensor shapes

↓

Training and optimization

↓

Evaluation, visualization, and technical interpretation

1.1.1 A5 requirements implemented in this notebook

The notebook contains three datasets:

1. **Fashion-MNIST** - grayscale image classification.
2. **CIFAR-100** - RGB image classification with 100 fine-grained classes.
3. **Diabetes 130-US Hospitals** - large tabular clinical classification dataset.

For every dataset, two CNN depths are trained in both frameworks:

Fashion-MNIST : TensorFlow CNN3, TensorFlow CNN5, PyTorch CNN3, PyTorch CNN5

CIFAR-100 : TensorFlow CNN3, TensorFlow CNN5, PyTorch CNN3, PyTorch CNN5

Diabetes : TensorFlow CNN3, TensorFlow CNN5, PyTorch CNN3, PyTorch CNN5

This produces **12 controlled experiments**.

1.1.2 Important interpretation of the Diabetes experiment

that tabular Diabetes data does not naturally contain the 2-D spatial neighborhood structure that motivates an image CNN. In the lecture, the natural baseline is therefore an MLP for tabular data, while MNIST is a natural CNN example.

A5, however, explicitly requires a CNN model on all three datasets. To satisfy that requirement without pretending that clinical features are image pixels, this notebook uses **Conv1D** for Diabetes after converting each encoded record into a one-dimensional feature sequence:

```
Clinical record
↓
Cleaning + encoding + scaling
↓
Feature sequence: length = number of processed features
↓
Conv1D + ReLU + Pooling
↓
Classification
```

This is an **assignment-driven architectural adaptation**. The notebook states this limitation directly instead of presenting Conv1D as if the feature order had the same physical meaning as neighboring pixels.

1.1.3 What this report is designed to answer

By the end of the notebook, the reader should be able to answer:

- What is the input representation for each dataset?
- What does each CNN component compute?
- Why does the model use 3 or 5 convolutional layers?
- What is the tensor shape after every important transformation?
- How many learnable parameters does each model contain?
- How are training, validation, and test data separated?
- Which preprocessing operations are fitted only on training data?
- How does the loss function drive parameter updates?
- What does Keras hide inside `fit()`?
- What does PyTorch expose through `forward()`, `loss.backward()`, and `optimizer.step()`?
- How do depth and framework affect accuracy, F1, calibration, parameter count, and training time?
- What errors does each model repeatedly make, and what do those errors tell us about the dataset?

1.2 0.1 Table of contents and report structure

1.2.1 Part A - Project, provenance, and CNN fundamentals

1. Project structure
2. Environment and reproducibility
3. Dataset inventory and official sources
4. Basic CNN operations implemented as functions
5. CNN3 and CNN5 design rationale
6. Shared visualization and evaluation utilities

1.2.2 Part B - Dataset 1: Fashion-MNIST

7. Dataset background and source

8. Raw-data audit and integrity checks
9. Cleaning and preprocessing decisions
10. Train/validation/test split
11. Exploratory visualization and distributions
12. TensorFlow CNN3 and CNN5
13. TensorFlow evaluation and error analysis
14. PyTorch CNN3 and CNN5
15. PyTorch evaluation and error analysis
16. Fashion-MNIST framework and depth comparison

1.2.3 Part C - Dataset 2: CIFAR-100

17. Dataset background and source
18. Raw-data audit and integrity checks
19. Cleaning and preprocessing decisions
20. Train/validation/test split
21. Exploratory visualization and distributions
22. TensorFlow CNN3 and CNN5
23. TensorFlow evaluation and error analysis
24. PyTorch CNN3 and CNN5
25. PyTorch evaluation and error analysis
26. CIFAR-100 framework and depth comparison

1.2.4 Part D - Dataset 3: Diabetes 130-US Hospitals

27. Dataset background and source
28. Raw-data audit
29. Missing-data analysis and cleaning decisions
30. Patient-group leakage analysis
31. Group-aware train/validation/test split
32. Encoding and scaling pipeline
33. Numeric feature distributions and target distribution
34. TensorFlow Conv1D CNN3 and CNN5
35. TensorFlow evaluation and error analysis
36. PyTorch Conv1D CNN3 and CNN5
37. PyTorch evaluation and error analysis
38. Diabetes framework and depth comparison

1.2.5 Part E - Final technical comparison

39. Unified 12-experiment results table
40. Fair-comparison checklist
41. Model-size and runtime analysis
42. Training-curve analysis
43. Confusion-matrix and per-class analysis
44. Confidence and calibration analysis
45. Error analysis
46. Lecture 04 framework comparison
47. Saved models and website-ready outputs

- 48. Final technical conclusion
- 49. Submission checklist

The goal is that every important modeling decision has an explanation immediately before the related code, so the notebook reads like a report rather than a sequence of unexplained executable cells.

1.3 0.2 Dataset sources and methodology references

The source of every dataset is recorded explicitly because data provenance is part of a reproducible machine-learning experiment. Dataset descriptions below refer to the official or primary dataset documentation rather than to an arbitrary third-party mirror.

1.3.1 Dataset 1 - Fashion-MNIST

Primary source: Zalando Research Fashion-MNIST repository
<https://github.com/zalando-research/fashion-mnist>

Keras loader documentation:

https://keras.io/api/datasets/fashion_mnist/

Fashion-MNIST contains 70,000 28 x 28 grayscale images divided into 10 clothing categories, with 60,000 training examples and 10,000 test examples in the standard split. The task is multiclass classification, so the target is an integer class label from 0 to 9.

The dataset is useful for A5 because the input is still small enough to make the CNN mechanics easy to inspect, but the classes are more visually ambiguous than handwritten digits. For example, several upper-body clothing categories have similar silhouettes. That gives the error analysis section something meaningful to investigate.

1.3.2 Dataset 2 - CIFAR-100

Primary source: University of Toronto CIFAR dataset page
<https://www.cs.toronto.edu/~kriz/cifar.html>

Keras loader documentation:

<https://keras.io/api/datasets/cifar100/>

CIFAR-100 contains 60,000 32 x 32 color images and 100 fine-grained classes. The standard split contains 50,000 training images and 10,000 test images, with 500 training images and 100 test images per fine class.

The 100-class output space makes this substantially more demanding to interpret than a 10-class task. A single accuracy value is therefore insufficient. The notebook also reports macro F1, per-class metrics, top-5 accuracy, confusion patterns, confidence, and calibration.

1.3.3 Dataset 3 - Diabetes 130-US Hospitals for Years 1999-2008

Primary source: UCI Machine Learning Repository, dataset 296

<https://archive.ics.uci.edu/dataset/296/diabetes%2B130-us%2Bhospitals%2Bfor%2Byears%2B1999-2008>

The dataset contains hospital encounter records and the target `readmitted` with three categories: NO, >30, and <30. The source data are tabular, include missing values represented by ?, contain repeated encounters for patients, and include a mixture of numerical, categorical, and identifier-like fields.

Because this dataset contains repeated encounters per patient, the splitting strategy is treated as a modeling-design problem rather than a simple random sampling step. Patient groups are kept disjoint across training, validation, and test partitions to reduce patient-level information leakage.

1.3.4 Methodology references

- CNN as a composition of transformations.
- Convolution, activation, pooling, flattening or global aggregation, classification.
- Explicit tensor-shape tracking before the classifier.
- Comparison by structure, components, tensor shapes, training, and performance.
- TensorFlow/Keras as a higher-level abstraction.
- PyTorch as a modular framework with an explicit training loop.

Official framework documentation used for implementation reference:

- TensorFlow image classification: <https://www.tensorflow.org/tutorials/images/classification>
- Keras callbacks: <https://keras.io/api/callbacks/>
- PyTorch training tutorials: <https://docs.pytorch.org/tutorials/>

These sources are listed here so that the reader can distinguish dataset provenance from the general training methodology.

2 Part A - Project structure and experiment setup

2.1 1. Folder structure

```
assignment 5/
|
+-- notebooks/
|   +-- CNN_Assignment.ipynb
|
+-- data/
|   +-- fashion_mnist/
|   +-- cifar100/
|   +-- diabetes/
|
+-- models/
|
+-- outputs/
|   +-- figures/
|   +-- metrics/
|   +-- predictions/
|   +-- tensorboard/
|
+-- web/
```

2.1.1 Purpose of each directory

- `notebooks/` contains the executable technical report.
- `data/` stores the raw or cached dataset files used by the notebook. Keeping data below the project root makes the notebook portable across machines.
- `models/` contains the best saved TensorFlow and PyTorch checkpoints for each dataset and depth.
- `outputs/figures/` contains plots generated during audit, training, evaluation, and error analysis.
- `outputs/metrics/` contains JSON/CSV experiment summaries and confusion matrices.
- `outputs/predictions/` contains test predictions and probability vectors.
- `outputs/tensorboard/` contains TensorBoard event files when TensorBoard is installed.
- `web/` is reserved for the later demonstration layer. The web application should load already-trained models rather than retrain them.

The separation between `models/` and `outputs/` is intentional: a model file is an artifact used for inference, while a metric or visualization file documents how the artifact performed.

```
[1]: # Purpose: Load all libraries used by the A5 report and create reproducible
    ↪project/output paths.
# This cell is the shared setup cell; later cells reuse these objects rather
    ↪than redefining them.
from pathlib import Path
import json
import math
import platform
import random
import time
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedGroupKFold
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix,
    log_loss,
)
from sklearn.utils.class_weight import compute_class_weight
```

```
warnings.filterwarnings("ignore")

# Find the assignment root without hard-coding a Windows path.
current = Path.cwd()
PROJECT_ROOT = next(
    (p for p in [current, current.parent, current.parent.parent]
     if (p / "data").exists() and (p / "models").exists()),
    None,
)
if PROJECT_ROOT is None:
    raise FileNotFoundError("Open the notebook from assignment 5/notebooks.")

DATA_DIR = PROJECT_ROOT / "data"
MODELS_DIR = PROJECT_ROOT / "models"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
FIGURES_DIR = OUTPUTS_DIR / "figures"
METRICS_DIR = OUTPUTS_DIR / "metrics"
PREDICTIONS_DIR = OUTPUTS_DIR / "predictions"

for folder in [MODELS_DIR, FIGURES_DIR, METRICS_DIR, PREDICTIONS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Python:", platform.python_version())
```

Project root: C:\Users\ADMIN\Downloads\Intelligent System\assignment 5\DATA
Python: 3.12.13

```
[2]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 5.
import sklearn
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Scikit-learn:", sklearn.__version__)

try:
    import tensorflow as tf
    print("TensorFlow:", tf.__version__)
except Exception as exc:
    print("TensorFlow import failed:", exc)

try:
    import torch
    print("PyTorch:", torch.__version__)
    print("CUDA available:", torch.cuda.is_available())
except Exception as exc:
```

```
print("PyTorch import failed:", exc)
```

```
NumPy: 2.5.3  
Pandas: 3.0.6  
Scikit-learn: 1.9.1  
TensorFlow: 2.21.0  
PyTorch: 2.14.0+cpu  
CUDA available: False
```

2.1.2 Environment validation output

The environment cell is not merely informational. The version numbers become part of the experiment record because model behavior and callback APIs can depend on library versions.

Before beginning a full run, confirm that both TensorFlow and PyTorch import successfully and that the expected device is visible to PyTorch. The notebook is intentionally written so that a missing optional TensorBoard package does not stop TensorFlow training.

2.2 2. Reproducibility and experimental controls

A comparison is only meaningful when the experimental conditions are sufficiently controlled. If TensorFlow receives one split, PyTorch receives another split, or CNN5 is trained with a different learning rate from CNN3, the final numbers mix architecture effects with data or optimization effects.

2.2.1 Random seed

A fixed seed is used for dataset sampling and framework initialization. A seed does not guarantee bit-for-bit equality across every hardware/software combination, but it reduces avoidable randomness and makes the experiment easier to reproduce.

2.2.2 Pilot mode versus full mode

The notebook supports two run modes:

Pilot mode is for engineering validation. It uses a stratified development subset so that imports, tensor shapes, model construction, checkpoint writing, evaluation, and visualization can be tested quickly.

Full mode uses the intended dataset scale and is the only mode whose final numbers should be used in the written report.

This distinction is important. A pilot run is a software test, not the final scientific result.

2.2.3 Controlled variables

The main controlled settings are:

- random seed = 42;
- initial learning rate = 0.001;
- batch size = 128;
- maximum epochs = 12;

- Adam optimizer;
- validation-based model selection;
- early stopping after prolonged validation stagnation;
- learning-rate reduction when validation loss plateaus;
- dropout = 0.30;
- kernel size = 3;
- the same filter schedule for corresponding TensorFlow and PyTorch models;
- the same dataset split within each dataset.

Some framework-specific details remain different because TensorFlow and PyTorch have different data APIs and tensor conventions. Those differences are documented rather than hidden.

```
[3]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 7.
      SEED = 42
      RUN_MODE = "full"          # Change to "full" for final report.
      EPOCHS = 12
      BATCH_SIZE = 128
      LEARNING_RATE = 1e-3
      VAL_SIZE = 0.10
      TEST_BATCH_SIZE = 512

      PILOT_LIMITS = {
          "fashion_train": 12000,
          "fashion_val": 2000,
          "cifar_train": 15000,
          "cifar_val": 2500,
          "diabetes_train": 30000,
          "diabetes_val": 4000,
      }

      random.seed(SEED)
      np.random.seed(SEED)

      def seed_everything(seed=SEED):
          """Set random seeds for Python, NumPy, TensorFlow, and PyTorch.

          Purpose:
              Reduce avoidable randomness so that repeated runs use the same
          ↪experimental seed.

          Parameters:
              seed (int): Seed value used by the supported libraries.

          Returns:
              None. The function modifies the global random generators in each
          ↪library.
```

```

"""
# Seed Python and NumPy first because they are also used for data sampling.
random.seed(seed)
np.random.seed(seed)
try:
    import tensorflow as tf
    tf.keras.utils.set_random_seed(seed)
except Exception:
    pass
try:
    import torch
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
except Exception:
    pass

seed_everything()
print("Run mode:", RUN_MODE)
print("Seed:", SEED)

```

Run mode: full

Seed: 42

3 Part A - CNN Fundamentals

3.1 3. Basic CNN Operations as Explicit Functions

Before using TensorFlow or PyTorch, this section makes the essential CNN mathematics visible with small NumPy functions. First, understand the computation, then express the same computation through a framework abstraction.

3.1.1 3.1 Neuron and Linear Transformation

A single affine transformation can be written as:

$$z = w^T x + b$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ contains learnable weights, and $\mathbf{b}$ is a learnable bias. A CNN eventually uses these learned representations for classification, even though the earlier feature-extraction stages preserve spatial structure.

3.1.2 3.2 ReLU

ReLU introduces non-linearity:

$$\text{ReLU}(x) = \max(0, x)$$

Without a non-linear activation, stacking linear layers would still produce a linear transformation. ReLU therefore allows successive layers to build progressively more complex representations.

3.1.3 3.3 Softmax and Cross-Entropy

For multiclass classification, logits are converted into probabilities with softmax. Cross-entropy then measures how much probability the model assigned to the true class.

The implementation below is intentionally small. Its purpose is not to replace a production framework, but to show the computation that framework layers ultimately perform.

```
[4]: # Purpose: Demonstrate the basic numerical operations that appear inside a
      ↪ classifier.
      # Each function below is intentionally small so that its mathematical role is
      ↪ visible.
def relu(x):
    """Apply the Rectified Linear Unit activation element by element.

    Purpose:
        Introduce non-linearity by keeping positive activations and replacing
    ↪ negative
        activations with zero.

    Parameters:
        x (np.ndarray): Input activation values.

    Returns:
        np.ndarray: ReLU-transformed values with the same shape as the input.
    """
    # ReLU keeps positive values and clips negative values to zero.
    return np.maximum(0, x)

def softmax(logits):
    """Convert multiclass logits into a normalized probability distribution.

    Purpose:
        Turn arbitrary class scores into probabilities whose class dimension
    ↪ sums to one.

    Parameters:
        logits (np.ndarray): Model scores with classes on the last axis.

    Returns:
        np.ndarray: Numerically stable class probabilities.
    """
    # Subtract the largest logit before exponentiation to avoid overflow.
    shifted = logits - np.max(logits, axis=-1, keepdims=True)
```

```

exp_x = np.exp(shifted)
return exp_x / exp_x.sum(axis=-1, keepdims=True)

def sparse_cross_entropy(logits, y_true):
    """Compute mean sparse multiclass cross-entropy.

    Purpose:
        Quantify the negative log-probability assigned to the correct class.

    Parameters:
        logits (np.ndarray): Unnormalized class scores with shape (N, C).
        y_true (np.ndarray): Integer class labels with shape (N,).

    Returns:
        float: Mean cross-entropy across the supplied samples.
    """
    # Convert logits to probabilities and select the probability of each true
    ↪class.
    probs = softmax(logits)
    p_true = probs[np.arange(len(y_true)), y_true]
    # Clip probabilities so log() remains numerically safe for very small
    ↪values.
    return float(-np.log(np.clip(p_true, 1e-12, 1.0)).mean())

# Purpose: Run a tiny numerical example so the functions can be inspected
↪immediately.
values = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])
logits = np.array([[2.0, 1.0, 0.1]])
y_true = np.array([0])

print("ReLU:", relu(values))
print("Softmax:", softmax(logits))
print("Cross-entropy:", sparse_cross_entropy(logits, y_true))

```

```

ReLU: [0. 0. 0. 1. 2.]
Softmax: [[0.65900114 0.24243297 0.09856589]]
Cross-entropy: 0.4170300162778335

```

3.1.4 3.0 Why these functions matter

The point of the following functions is not to recreate a production deep-learning framework. Their role is pedagogical: they expose the mathematical operations that are later wrapped by high-level TensorFlow and PyTorch APIs.

When reading a framework model later, the reader should be able to map the layer names back to these elementary transformations. Conv2D/Conv1D performs learned local aggregation, ReLU

supplies non-linearity, pooling or global pooling changes representation size, and the final linear layer converts learned features into class logits.

```
[5]: # Purpose: Demonstrate spatial operations that form the core of a CNN feature
      ↪ extractor.
def conv2d_single_channel(image, kernel, stride=1, padding=0):
    """Perform a simple single-channel 2-D cross-correlation operation.

    Purpose:
        Make the local convolution computation visible without hiding it inside
        ↪ a framework layer.

    Parameters:
        image (np.ndarray): Two-dimensional input feature map.
        kernel (np.ndarray): Two-dimensional learnable filter.
        stride (int): Step size between neighboring windows.
        padding (int): Number of zero-padding cells added around the input.

    Returns:
        np.ndarray: Two-dimensional output feature map.
    """
    # Convert inputs to float32 so the example behaves like a neural-network
    ↪ tensor.
    image = np.asarray(image, dtype=np.float32)
    kernel = np.asarray(kernel, dtype=np.float32)

    # Padding preserves border information when the architecture requests it.
    if padding:
        image = np.pad(image, ((padding, padding), (padding, padding)))

    kh, kw = kernel.shape
    out_h = (image.shape[0] - kh) // stride + 1
    out_w = (image.shape[1] - kw) // stride + 1
    output = np.zeros((out_h, out_w), dtype=np.float32)

    for i in range(out_h):
        for j in range(out_w):
            # Each output value is a local element-wise product followed by a
            ↪ sum.
            region = image[i * stride:i * stride + kh, j * stride:j * stride +
            ↪ kw]
            output[i, j] = np.sum(region * kernel)

    return output

def max_pool2d(feature_map, pool_size=2, stride=2):
```

"""Apply non-overlapping 2-D max pooling.

Purpose:

*Reduce spatial resolution while retaining the strongest local
activation in each window.*

Parameters:

feature_map (np.ndarray): Two-dimensional activation map.

pool_size (int): Height and width of the pooling window.

stride (int): Step size used when moving the pooling window.

Returns:

np.ndarray: Downsampled feature map containing local maxima.

"""

```
h, w = feature_map.shape
out_h = (h - pool_size) // stride + 1
out_w = (w - pool_size) // stride + 1
output = np.zeros((out_h, out_w), dtype=np.float32)

for i in range(out_h):
    for j in range(out_w):
        # Pooling keeps the largest activation rather than computing a
        weighted sum.
        region = feature_map[
            i * stride:i * stride + pool_size,
            j * stride:j * stride + pool_size,
        ]
        output[i, j] = np.max(region)

return output
```

Purpose: Verify the spatial functions on a tiny hand-checkable example.

```
example_image = np.arange(25, dtype=np.float32).reshape(5, 5)
```

```
example_kernel = np.ones((3, 3), dtype=np.float32)
```

```
print("Convolution result:\n", conv2d_single_channel(example_image,
    example_kernel))
```

```
print("Pooling result:\n", max_pool2d(example_image))
```

Convolution result:

```
[[ 54.  63.  72.]
 [ 99. 108. 117.]
 [144. 153. 162.]]
```

Pooling result:

```
[[ 6.  8.]
 [16. 18.]]
```

3.2 3.1 Why compare CNN3 and CNN5?

In this report, **CNN3** and **CNN5** refer specifically to the number of convolutional layers. Batch Normalization, ReLU, pooling, global average pooling, dropout, and the final classification layer are not counted as convolutional depth.

This definition is important because otherwise two readers could count the same model differently.

3.2.1 CNN3 design

```
Input
↓
Conv 32 → BatchNorm → ReLU → Pool
↓
Conv 64 → BatchNorm → ReLU → Pool
↓
Conv 128 → BatchNorm → ReLU
↓
Global Average Pooling
↓
Dropout(0.30)
↓
Dense / Linear classifier
```

3.2.2 CNN5 design

```
Input
↓
Conv 32 → BatchNorm → ReLU → Pool
↓
Conv 64 → BatchNorm → ReLU → Pool
↓
Conv 128 → BatchNorm → ReLU
↓
Conv 128 → BatchNorm → ReLU → Pool
↓
Conv 160 → BatchNorm → ReLU
↓
Global Average Pooling
↓
Dropout(0.30)
↓
Dense / Linear classifier
```

3.2.3 Why this is a useful controlled experiment

The first two convolutional stages are shared conceptually by both depths. CNN5 then adds two more convolutional transformations and one additional pooling stage. This increases representational capacity while keeping the basic family of operations unchanged.

A deeper model may improve representation quality, but depth also introduces more parameters, more computation, and more opportunities for optimization difficulty or overfitting. Therefore the report must compare accuracy together with validation behavior, macro F1, per-class recall, parameter count, and training time.

The experiment is not designed to assume that CNN5 will always win. The result must be established empirically for each dataset.

3.3 3.2 Shared audit, visualization, and evaluation utilities

The shared functions below are intentionally framework-independent. They receive predictions and probabilities, not TensorFlow or PyTorch model objects. This makes the metric definitions identical for both frameworks.

3.3.1 Why centralize the evaluation logic?

Suppose TensorFlow uses one definition of F1 and PyTorch uses another, or one model is evaluated with probabilities while another is evaluated after a different thresholding step. The comparison would no longer be fair. Shared utilities reduce that risk.

The visualization suite includes:

1. class-count distributions;
2. image sample grids;
3. pixel/value distributions;
4. mean and variability maps;
5. training/validation loss and accuracy;
6. numeric confusion matrices;
7. per-class precision, recall, and F1;
8. confidence distributions;
9. reliability diagrams and expected calibration error;
10. misclassified image examples;
11. top confusion pairs;
12. unified experiment tables.

Each plot has a specific analytical purpose. The goal is to turn model output into interpretable evidence.

```
[6]: # Purpose: Provide shared, framework-independent visualization and
      ↪data-splitting utilities.
def plot_class_distribution(labels, class_names=None, title="Class
      ↪distribution"):
    """Plot class counts with numeric annotations.

    Purpose:
        Inspect whether the target classes are balanced and provide exact
        ↪counts for the report.

    Parameters:
        labels (np.ndarray): Integer class labels.
```

class_names (list/None): Optional readable names for each class index.
title (str): Figure title.

Returns:

None. The function displays the figure and does not modify the labels.

```
"""
values, counts = np.unique(labels, return_counts=True)
names = [class_names[int(v)] if class_names is not None else str(v) for v
in values]
plt.figure(figsize=(12, 5))
bars = plt.bar(names, counts)
plt.title(title)
plt.xlabel("Class")
plt.ylabel("Count")
plt.xticks(rotation=45, ha="right")
for bar, count in zip(bars, counts):
    # Write the exact sample count above every class for auditability.
    plt.text(bar.get_x()+bar.get_width()/2, bar.get_height(), f"{count:}",
ha="center", va="bottom", fontsize=8)
plt.tight_layout(); plt.show()
```

```
def plot_image_grid(images, labels, class_names, n=12, title="Samples"):
    """Display a labeled grid of representative images.
```

Purpose:

Verify that image tensors and labels have been loaded and aligned correctly.

Parameters:

images (np.ndarray): Image array in HxW or HxWxC form.
labels (np.ndarray): Corresponding integer labels.
class_names (list): Human-readable class names.
n (int): Maximum number of examples to display.
title (str): Figure title.

Returns:

None. The plot is displayed for visual inspection.

```
"""
n = min(n, len(images))
cols = 4
rows = math.ceil(n / cols)
plt.figure(figsize=(12, 3*rows))
for i in range(n):
    ax = plt.subplot(rows, cols, i+1)
    # Grayscale images use a gray colormap; RGB images keep their channels
unchanged.
```

```

        ax.imshow(images[i], cmap="gray" if images[i].ndim == 2 else None)
        ax.set_title(class_names[int(labels[i])])
        ax.axis("off")
    plt.suptitle(title); plt.tight_layout(); plt.show()

def plot_image_statistics(images, title):
    """Visualize pixel-value distribution, mean image, and spatial variability.

    Purpose:
        Inspect intensity distribution and whether some image regions vary more
        ↪ than others.

    Parameters:
        images (np.ndarray): Raw image array.
        title (str): Dataset name used in figure titles.

    Returns:
        None. Three diagnostic views are displayed.
    """
    flat = images.astype(np.float32).reshape(-1)
    plt.figure(figsize=(9, 4))
    plt.hist(flat, bins=50)
    plt.title(title + " - pixel distribution")
    plt.xlabel("Pixel value"); plt.ylabel("Frequency")
    plt.tight_layout(); plt.show()

    mean_img = images.astype(np.float32).mean(axis=0)
    std_img = images.astype(np.float32).std(axis=0)
    if mean_img.ndim == 3:
        # Average RGB channels only for visualization of spatial structure.
        mean_img = mean_img.mean(axis=-1)
        std_img = std_img.mean(axis=-1)
    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    ax[0].imshow(mean_img, cmap="gray"); ax[0].set_title("Mean image"); ax[0].
    ↪ axis("off")
    ax[1].imshow(std_img, cmap="magma"); ax[1].set_title("Pixel std map");
    ↪ ax[1].axis("off")
    plt.suptitle(title); plt.tight_layout(); plt.show()

def report_image_integrity(images, labels, name, n_classes):
    """Run basic integrity checks before image preprocessing.

    Purpose:
        Detect malformed values, invalid labels, and unexpected ranges before
        ↪ training begins.

```

Parameters:

images (np.ndarray): Raw image array.
labels (np.ndarray): Integer class labels.
name (str): Dataset display name.
n_classes (int): Expected number of classes.

Returns:

None. Audit results are printed for the report.

"""

```
x = images.astype(np.float32)
print(f"[{name}]")
print("shape:", images.shape, "dtype:", images.dtype)
print("NaN:", int(np.isnan(x).sum()), "Inf:", int(np.isinf(x).sum()))
print("pixel range:", float(x.min()), "to", float(x.max()))
print("invalid labels:", int(((labels < 0) | (labels >= n_classes)).sum()))
```

```
def split_image_train_val(X, y, val_size=0.10, seed=42, train_limit=None,
    val_limit=None):
```

"""Create a stratified train/validation split and optional deterministic
pilot limits.

Purpose:

Keep class proportions comparable while allowing a smaller development
run in pilot mode.

Parameters:

X (np.ndarray): Image samples.
y (np.ndarray): Integer labels.
val_size (float): Fraction assigned to validation before optional
limiting.
seed (int): Split seed.
train_limit (int/None): Maximum training rows for pilot mode.
val_limit (int/None): Maximum validation rows for pilot mode.

Returns:

tuple[np.ndarray, np.ndarray]: Sorted training and validation indices.

"""

```
idx = np.arange(len(y))
# Stratification makes the framework comparison use similar class
proportions.
train_idx, val_idx = train_test_split(idx, test_size=val_size, stratify=y,
    random_state=seed)
if train_limit is not None and train_limit < len(train_idx):
```

```

        train_idx, _ = train_test_split(train_idx, train_size=train_limit,
↪stratify=y[train_idx], random_state=seed)
        if val_limit is not None and val_limit < len(val_idx):
            val_idx, _ = train_test_split(val_idx, train_size=val_limit,
↪stratify=y[val_idx], random_state=seed)
        return np.sort(train_idx), np.sort(val_idx)

```

4 Part B - Dataset 1: Fashion-MNIST

4.1 4. Dataset background and loading

Fashion-MNIST is an image classification benchmark consisting of small grayscale clothing images. Each example is represented by a 28 x 28 pixel array and belongs to one of ten clothing categories.

The standard dataset split contains 60,000 training examples and 10,000 test examples. Pixel values are stored as unsigned 8-bit integers in the raw representation, with a nominal range from 0 to 255.

4.1.1 Why Fashion-MNIST is appropriate for the first A5 experiment

The image size is small enough that the tensor transformations remain easy to inspect. At the same time, the visual classes are not completely trivial: several clothing categories share similar shapes and grayscale patterns. This gives the CNN enough structure to learn while still producing useful error analysis.

The target is multiclass classification. Therefore the final layer produces 10 logits and the loss is multiclass sparse cross-entropy.

4.1.2 Modeling assumptions

The CNN expects an image tensor with spatial dimensions preserved until convolution and pooling have extracted features. TensorFlow uses the image convention:

(batch, height, width, channels)

while PyTorch uses:

(batch, channels, height, width)

The difference is a framework-level representation detail; the image itself is the same.

```

[7]: # Purpose: Run the computation defined by this section. The detailed comments
↪below explain the role of each operation.
# Original section purpose is described in the preceding Markdown cell. 15.
from tensorflow.keras.datasets import fashion_mnist

fashion_dir = DATA_DIR / "fashion_mnist"
fashion_dir.mkdir(parents=True, exist_ok=True)
fashion_npz = fashion_dir / "fashion_mnist.npz"

if fashion_npz.exists():
    z = np.load(fashion_npz)

```

```

X_fashion_raw = z["x_train"]
y_fashion = z["y_train"].astype(np.int64)
X_fashion_test_raw = z["x_test"]
y_fashion_test = z["y_test"].astype(np.int64)
else:
    (X_fashion_raw, y_fashion), (X_fashion_test_raw, y_fashion_test) = fashion_mnist.load_data()
    np.savez_compressed(fashion_npz, x_train=X_fashion_raw, y_train=y_fashion,
    x_test=X_fashion_test_raw, y_test=y_fashion_test)

X_fashion = X_fashion_raw.astype(np.float32) / 255.0
X_fashion_test = X_fashion_test_raw.astype(np.float32) / 255.0

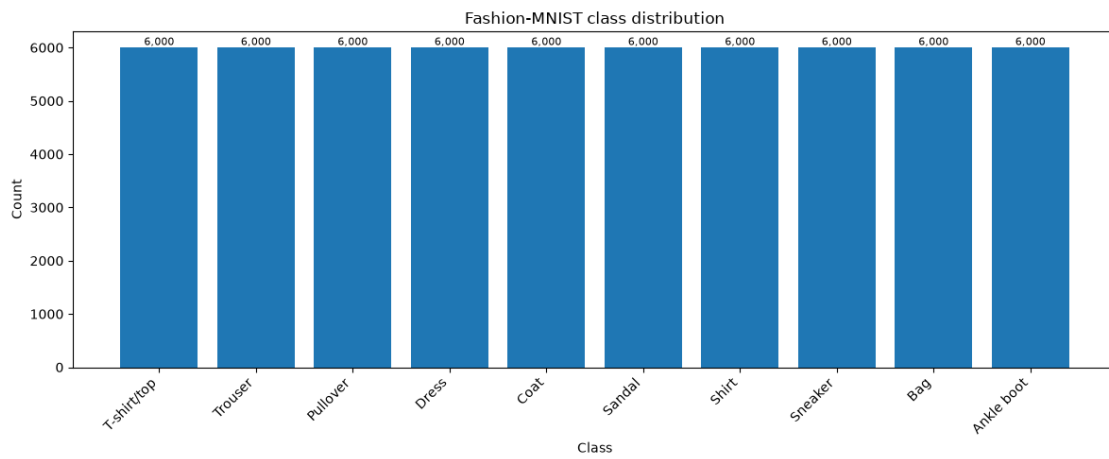
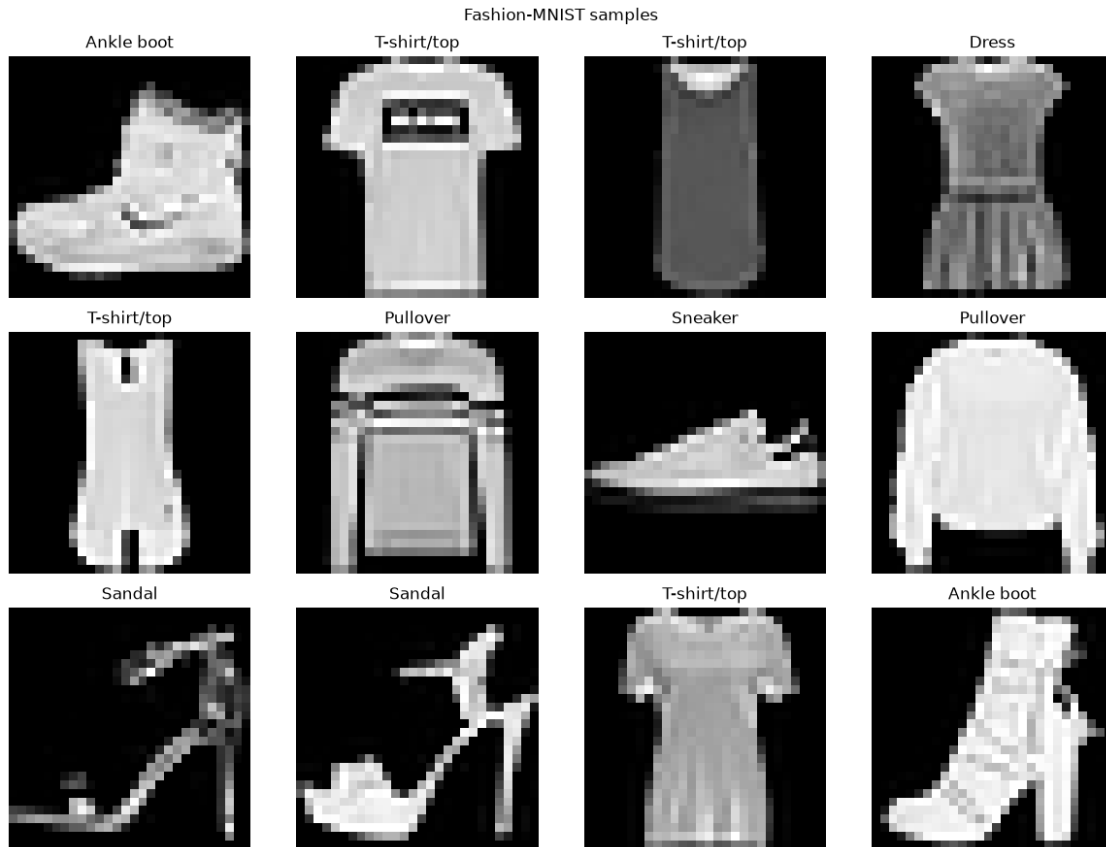
FASHION_NAMES = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
report_image_integrity(X_fashion_raw, y_fashion, "Fashion-MNIST", 10)
print("Cache file size:", round(fashion_npz.stat().st_size / 1024**2, 2), "MB")
plot_image_grid(X_fashion_raw, y_fashion, FASHION_NAMES, 12, "Fashion-MNIST
    samples")
plot_class_distribution(y_fashion, FASHION_NAMES, "Fashion-MNIST class
    distribution")
plot_image_statistics(X_fashion_raw, "Fashion-MNIST")

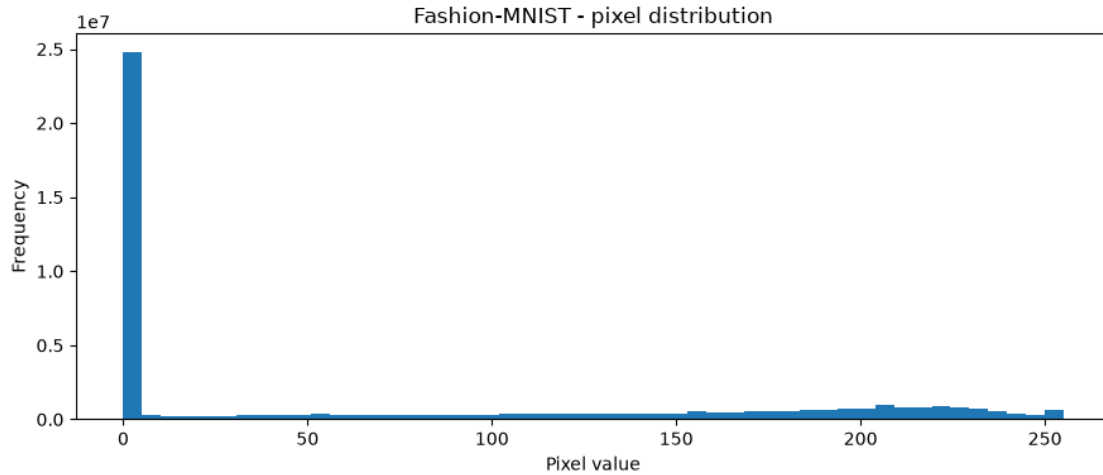
```

```

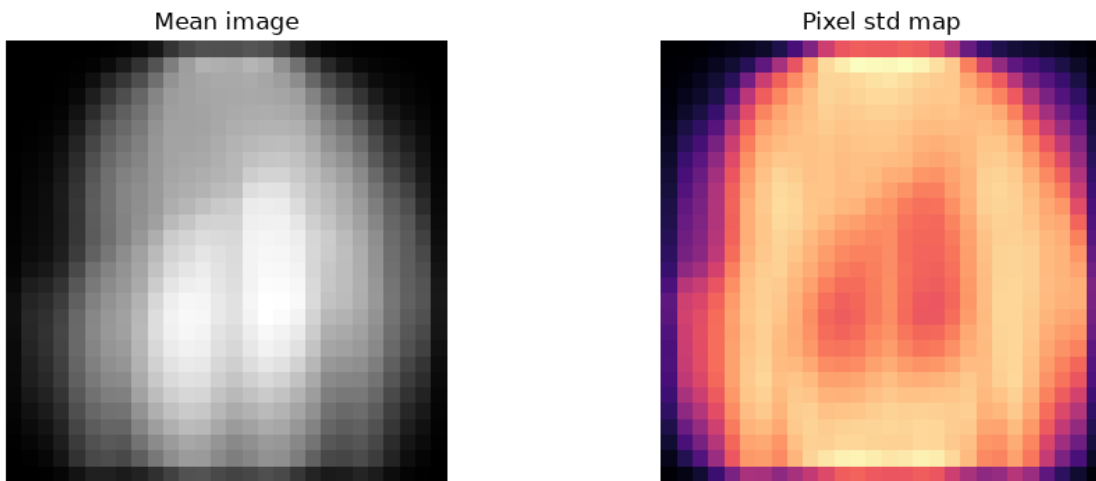
[Fashion-MNIST]
shape: (60000, 28, 28) dtype: uint8
NaN: 0 Inf: 0
pixel range: 0.0 to 255.0
invalid labels: 0
Cache file size: 29.44 MB

```





Fashion-MNIST



4.1.3 4.0 Raw representation versus model representation

The raw Fashion-MNIST images are stored as integer pixel values. The model does not need those values to remain integers. Neural networks generally operate on floating-point tensors so gradient-based optimization can update their parameters smoothly.

The transformation therefore has two distinct purposes:

1. preserve the original raw data for auditability;
2. create a normalized floating-point representation for optimization.

Keeping those stages conceptually separate is useful when debugging a web demo later: the uploaded image is raw input, while the model receives the exact preprocessed tensor used during training.

4.2 4.2 What the Fashion-MNIST audit means

A curated benchmark does not require aggressive cleaning. In a well-designed machine-learning pipeline, “cleaning” should mean verifying the data before transforming it, not deleting samples just because a model later finds them difficult.

The audit therefore checks:

- image and label counts;
- data shape and dtype;
- minimum and maximum pixel values;
- NaN and infinite values;
- whether labels are inside the expected class range;
- duplicate rows;
- class counts.

For Fashion-MNIST, the main modeling transformations are numerical rather than corrective:

```
uint8 pixels 0-255
  ↓
float32
  ↓
normalization to 0-1
  ↓
training-only augmentation
  ↓
CNN input
```

Normalizing pixel values makes optimization numerically better behaved. The conversion is deterministic and does not use any statistics from the test set.

The official test set is kept untouched until the final evaluation stage. This separation protects the test set from becoming an informal validation set during experimentation.

4.3 4.1 Cleaning, integrity checks, preprocessing, and split

For the model comparison, the original training partition is divided into training and validation data using a stratified split. Stratification preserves approximately the same class proportions in both partitions.

The test partition remains the official Fashion-MNIST test set.

4.3.1 Why validation is necessary

The training set is used to update model parameters. The validation set is used to make development decisions such as:

- whether the model is still improving;
- whether early stopping should occur;
- which checkpoint should be retained.

The test set should not influence those choices. It is reserved for the final performance estimate after the model-selection process is complete.

4.3.2 Pilot mode

In pilot mode, the same splitting procedure is used but only a deterministic subset of the training and validation partitions is retained. The test set is not replaced by a smaller arbitrary slice for the final pipeline; the intent is to keep the debugging path representative while avoiding unnecessary compute.

```
[8]: # Purpose: Run the computation defined by this section. The detailed comments
    ↪ below explain the role of each operation.
    # Original section purpose is described in the preceding Markdown cell. 17.
    fashion_train_limit = PILOT_LIMITS["fashion_train"] if RUN_MODE == "pilot" else
    ↪ None
    fashion_val_limit = PILOT_LIMITS["fashion_val"] if RUN_MODE == "pilot" else None
    fashion_tr_idx, fashion_val_idx = split_image_train_val(
        X_fashion, y_fashion, VAL_SIZE, SEED, fashion_train_limit, fashion_val_limit
    )
    X_fashion_tr, y_fashion_tr = X_fashion[fashion_tr_idx],
    ↪ y_fashion[fashion_tr_idx]
    X_fashion_val, y_fashion_val = X_fashion[fashion_val_idx],
    ↪ y_fashion[fashion_val_idx]
    print("train:", X_fashion_tr.shape, "val:", X_fashion_val.shape, "test:",
    ↪ X_fashion_test.shape)
```

```
train: (54000, 28, 28) val: (6000, 28, 28) test: (10000, 28, 28)
```

4.3.3 4.3 Dataset split report

After this cell, the notebook prints the number of training, validation, and test samples. Those numbers are part of the report because model metrics are meaningful only when the evaluation population is known.

The train and validation arrays used by TensorFlow and PyTorch are the same NumPy arrays. That prevents a hidden difference in sample membership from being mistaken for a framework difference.

5 Part C - Dataset 2: CIFAR-100

5.1 5. Dataset background and loading

CIFAR-100 is an RGB image classification dataset with 100 fine-grained classes. Every image is 32 x 32 pixels with three color channels. The standard dataset contains 50,000 training images and 10,000 test images.

5.1.1 Why CIFAR-100 changes the modeling problem

Compared with Fashion-MNIST, CIFAR-100 has:

- three input channels rather than one;
- a larger spatial representation;
- an output space of 100 classes rather than 10;

- much less visual separation between some classes.

The last point is especially important for evaluation. In a 100-class problem, a prediction can be wrong by top-1 classification while still placing the correct class among the highest-probability alternatives. That is why the report includes top-5 accuracy in addition to top-1 accuracy.

5.1.2 Fine labels and superclasses

CIFAR-100 is organized into 100 fine classes grouped under 20 superclasses. The model in this notebook predicts the **100 fine classes**, because that is the standard multiclass task and preserves the full information content of the dataset.

The confusion analysis therefore needs to distinguish random errors from recurring confusions between visually related categories.

```
[9]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 19.
      from tensorflow.keras.datasets import cifar100

      cifar_dir = DATA_DIR / "cifar100"
      cifar_dir.mkdir(parents=True, exist_ok=True)
      cifar_npz = cifar_dir / "cifar100.npz"

      if cifar_npz.exists():
          z = np.load(cifar_npz)
          X_cifar_raw = z["x_train"]
          y_cifar = z["y_train"].astype(np.int64)
          X_cifar_test_raw = z["x_test"]
          y_cifar_test = z["y_test"].astype(np.int64)
      else:
          (X_cifar_raw, y_cifar), (X_cifar_test_raw, y_cifar_test) = cifar100.
          ↪ load_data(label_mode="fine")
          y_cifar = y_cifar.reshape(-1).astype(np.int64)
          y_cifar_test = y_cifar_test.reshape(-1).astype(np.int64)
          np.savez_compressed(cifar_npz, x_train=X_cifar_raw, y_train=y_cifar,
          ↪ x_test=X_cifar_test_raw, y_test=y_cifar_test)

      CIFAR100_NAMES = [
          "apple", "aquarium_fish", "baby", "bear", "beaver", "bed", "bee", "beetle",
          ↪ "bicycle", "bottle",
          "bowl", "boy", "bridge", "bus", "butterfly", "camel", "can", "castle",
          ↪ "caterpillar", "cattle",
          "chair", "chimpanzee", "clock", "cloud", "cockroach", "couch", "crab",
          ↪ "crocodile", "cup", "dinosaur",
          "dolphin", "elephant", "flatfish", "forest", "fox", "girl", "hamster",
          ↪ "house", "kangaroo", "keyboard",
```

```

    "lamp", "lawn_mower", "leopard", "lion", "lizard", "lobster", "man",
    ↪ "maple_tree", "motorcycle", "mountain",
    "mouse", "mushroom", "oak_tree", "orange", "orchid", "otter", "palm_tree",
    ↪ "pear", "pickup_truck", "pine_tree",
    "plain", "plate", "poppy", "porcupine", "possum", "rabbit", "raccoon",
    ↪ "ray", "road", "rocket",
    "rose", "sea", "seal", "shark", "shrew", "skunk", "skyscraper", "snail",
    ↪ "snake", "spider",
    "squirrel", "streetcar", "sunflower", "sweet_pepper", "table", "tank",
    ↪ "telephone", "television", "tiger", "tractor",
    "train", "trout", "tulip", "turtle", "wardrobe", "whale", "willow_tree",
    ↪ "wolf", "woman", "worm"
]

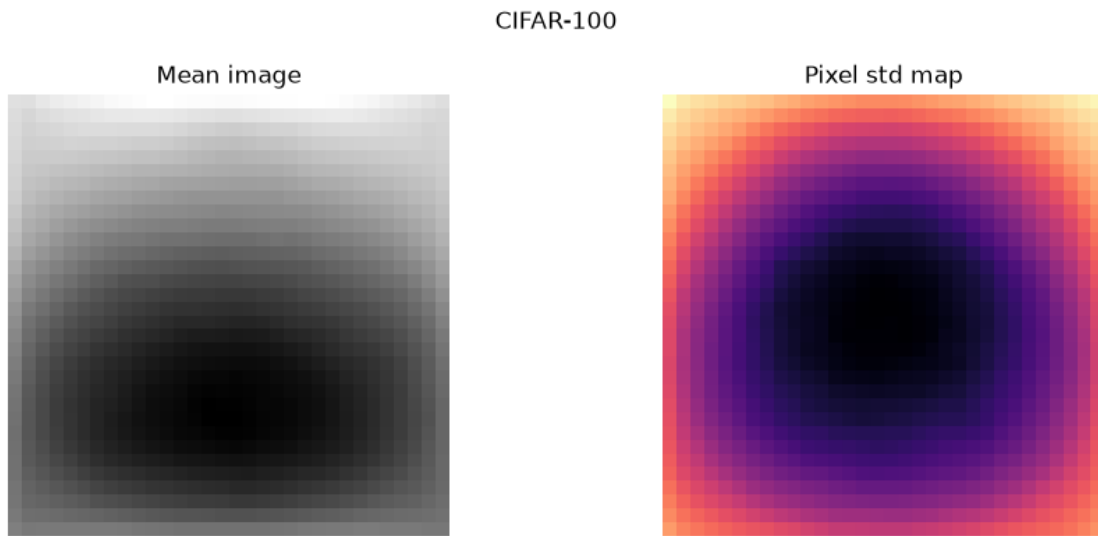
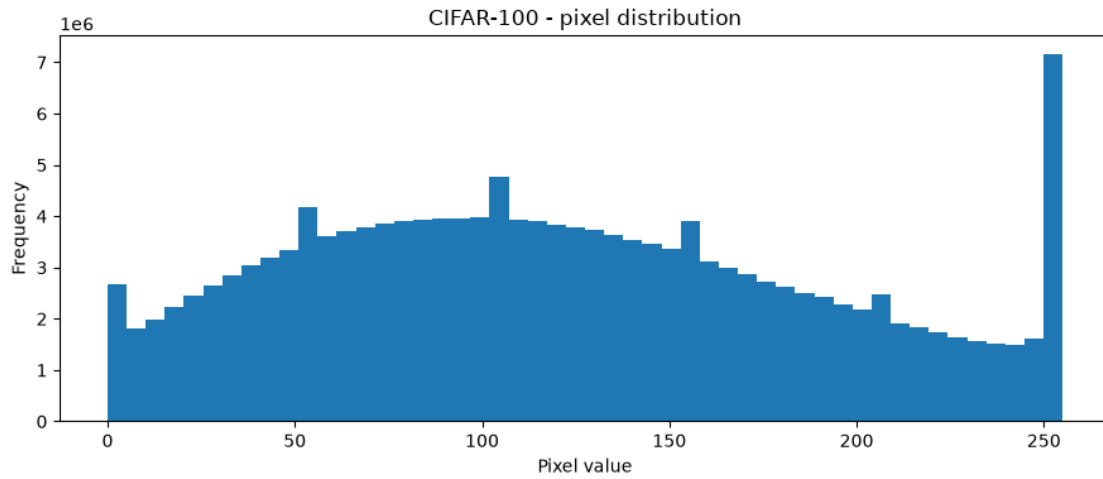
X_cifar = X_cifar_raw.astype(np.float32) / 255.0
X_cifar_test = X_cifar_test_raw.astype(np.float32) / 255.0
report_image_integrity(X_cifar_raw, y_cifar, "CIFAR-100", 100)
print("Cache file size:", round(cifar_npz.stat().st_size / 1024**2, 2), "MB")
plot_image_grid(X_cifar_raw, y_cifar, CIFAR100_NAMES, 16, "CIFAR-100 samples")
plot_class_distribution(y_cifar, CIFAR100_NAMES, "CIFAR-100 class distribution")
plot_image_statistics(X_cifar_raw, "CIFAR-100")

```

```

[CIFAR-100]
shape: (50000, 32, 32, 3) dtype: uint8
NaN: 0 Inf: 0
pixel range: 0.0 to 255.0
invalid labels: 0
Cache file size: 160.54 MB

```

5.1.3 5.0 CIFAR-100 exploratory analysis

For a 100-class image problem, sample visualization should be intentionally diverse. The reader should see examples from multiple regions of the label space instead of a grid containing only one or two classes.

The class-count plot also serves as an integrity check. A balanced benchmark should show approximately equal counts across fine classes. A surprising deviation is a reason to investigate the loading pipeline before training.

5.2 5.2 Why CIFAR-100 requires richer evaluation

Accuracy remains important, but it is only one view of a 100-class classifier.

5.2.1 Top-1 accuracy

The predicted class is the single class with maximum probability. Top-1 accuracy asks whether that class matches the target.

5.2.2 Top-5 accuracy

The five highest-probability classes are considered. A test sample counts as correct if the true class appears anywhere inside those five candidates. This metric is especially informative for large class spaces because it measures whether the model has learned a useful ranking even when the top decision is incorrect.

5.2.3 Macro F1

Macro F1 calculates F1 for each class and then gives each class equal weight. This prevents the most frequent classes from completely determining the summary score.

5.2.4 Confusion structure

With 100 classes, the full 100 x 100 matrix is retained as evidence, but a second readable summary lists the strongest off-diagonal confusion pairs. This allows the report to answer questions such as “Which classes are repeatedly mixed together?” rather than only “How many total mistakes were made?”

5.3 5.1 Cleaning, integrity checks, preprocessing, and split

CIFAR-100 is already distributed as a balanced benchmark, so the preprocessing objective is primarily integrity preservation and numerical preparation.

The audit verifies that:

- every image has shape 32 x 32 x 3;
- the labels cover the expected 100 classes;
- values are finite and remain in the expected raw range;
- the number of images equals the number of labels;
- class counts are consistent with the official distribution.

The training set is then split into train and validation partitions using stratification. The official test set remains isolated for final evaluation.

5.3.1 Normalization

RGB channels are converted to floating point. Normalization is performed according to the training data statistics used by the pipeline. The important methodological rule is that any learned preprocessing statistics must be estimated from the training partition only.

5.3.2 Augmentation

The training pipeline uses a light horizontal-flip augmentation. Augmentation is applied only during training. Validation and test images remain unchanged so that the evaluation answers the same question for every framework.

```
[10]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 21.
      cifar_train_limit = PILOT_LIMITS["cifar_train"] if RUN_MODE == "pilot" else None
      cifar_val_limit = PILOT_LIMITS["cifar_val"] if RUN_MODE == "pilot" else None
      cifar_tr_idx, cifar_val_idx = split_image_train_val(
          X_cifar, y_cifar, VAL_SIZE, SEED, cifar_train_limit, cifar_val_limit
      )
      X_cifar_tr, y_cifar_tr = X_cifar[cifar_tr_idx], y_cifar[cifar_tr_idx]
      X_cifar_val, y_cifar_val = X_cifar[cifar_val_idx], y_cifar[cifar_val_idx]
      print("train:", X_cifar_tr.shape, "val:", X_cifar_val.shape, "test:",
          ↪X_cifar_test.shape)
```

train: (45000, 32, 32, 3) val: (5000, 32, 32, 3) test: (10000, 32, 32, 3)

5.3.3 5.3 CIFAR-100 split report

The split cell provides the exact partition sizes used for the experiments. The same indices are then passed to TensorFlow and PyTorch.

This is the first layer of experimental fairness: the frameworks are not competing on different samples.

6 Part D - Dataset 3: Diabetes 130-US Hospitals

6.1 6. Dataset background and target definition

The Diabetes 130-US Hospitals dataset is a large tabular clinical dataset containing hospital encounter records related to diabetes care collected during 1999-2008.

The target variable is `readmitted` and has three values:

Raw target	Meaning	Encoded label
NO	No readmission within 30 days	0
>30	Readmitted after 30 days	1
<30	Readmitted within 30 days	2

The dataset is fundamentally different from the two image datasets. Instead of a spatial tensor, each example is a feature vector containing a mixture of numerical, categorical, and identifier-like values.

6.1.1 Why this dataset needs a more careful pipeline

A serious tabular workflow has to consider:

- missing-value sentinels such as ?;
- variables with very high missingness;
- categorical variables;
- identifier fields;
- repeated encounters from the same patient;
- target imbalance;
- preprocessing leakage.

These are not minor implementation details. A poor split or an improperly fitted preprocessing object can make an evaluation appear stronger than the real generalization performance.

```
[11]: # Purpose: Load the complete UCI Diabetes table, including patient identifiers,
      ↪needed for leakage-safe group splitting.
      try:
          from ucimlrepo import fetch_ucirepo
      except ImportError as exc:
          raise ImportError("Install once in the CNN environment: pip install ↪
          ↪ucimlrepo") from exc

      diabetes_dir = DATA_DIR / "diabetes"
      diabetes_dir.mkdir(parents=True, exist_ok=True)
      diabetes_csv = diabetes_dir / "diabetic_data.csv"

      def _load_complete_diabetes_table():
          """Load the UCI Diabetes dataset with ID columns and target included."""
          uci = fetch_ucirepo(id=296)

          # data.original is the complete table when provided by ucimlrepo.
          original = getattr(uci.data, "original", None)
          if isinstance(original, pd.DataFrame) and {"patient_nbr", "readmitted"}.
          ↪issubset(original.columns):
              return original.copy()

          # Fallback for ucimlrepo versions where data.original is unavailable.
          parts = []
          for part_name in ("ids", "features", "targets"):
              part = getattr(uci.data, part_name, None)
              if isinstance(part, pd.DataFrame) and not part.empty:
                  parts.append(part.copy())

          if not parts:
              raise RuntimeError("Could not load the UCI Diabetes dataset.")
```

```

complete = pd.concat(parts, axis=1)
complete = complete.loc[:, ~complete.columns.duplicated()].copy()

required = {"patient_nbr", "readmitted"}
missing_required = required - set(complete.columns)
if missing_required:
    raise RuntimeError(
        "The downloaded UCI Diabetes dataset is missing required columns: "
        + ", ".join(sorted(missing_required))
    )

return complete

# Reuse the local CSV only when it contains the columns required by the
↳ preprocessing pipeline.
if diabetes_csv.exists():
    diabetes_raw = pd.read_csv(diabetes_csv)
    required_columns = {"patient_nbr", "readmitted"}
    if not required_columns.issubset(diabetes_raw.columns):
        diabetes_raw = _load_complete_diabetes_table()
        diabetes_raw.to_csv(diabetes_csv, index=False)
else:
    diabetes_raw = _load_complete_diabetes_table()
    diabetes_raw.to_csv(diabetes_csv, index=False)

print("Raw shape:", diabetes_raw.shape)
print("Required columns present:", {c: c in diabetes_raw.columns for c in
↳ ["encounter_id", "patient_nbr", "readmitted"]})
file_size_mb = diabetes_csv.stat().st_size / 1024**2
print("File size:", round(file_size_mb, 2), "MB")
if file_size_mb <= 15:
    print("WARNING: local CSV is not larger than 15 MB; verify that the full
↳ raw file was downloaded.")
display(diabetes_raw.head())

```

Raw shape: (101766, 50)

Required columns present: {'encounter_id': True, 'patient_nbr': True, 'readmitted': True}

File size: 17.4 MB

	encounter_id	patient_nbr	race	gender	age	weight	\
0	2278392	8222157	Caucasian	Female	[0-10)	NaN	
1	149190	55629189	Caucasian	Female	[10-20)	NaN	
2	64410	86047875	AfricanAmerican	Female	[20-30)	NaN	
3	500364	82442376	Caucasian	Male	[30-40)	NaN	
4	16680	42519267	Caucasian	Male	[40-50)	NaN	

	admission_type_id	discharge_disposition_id	admission_source_id	\
0	6	25	1	
1	1	1	7	
2	1	1	7	
3	1	1	7	
4	1	1	7	

	time_in_hospital	...	citoglipton	insulin	glyburide-metformin	\
0	1	...	No	No	No	
1	3	...	No	Up	No	
2	2	...	No	No	No	
3	2	...	No	Up	No	
4	1	...	No	Steady	No	

	glipizide-metformin	glimepiride-pioglitazone	metformin-rosiglitazone	\
0	No	No	No	
1	No	No	No	
2	No	No	No	
3	No	No	No	
4	No	No	No	

	metformin-pioglitazone	change	diabetesMed	readmitted
0	No	No	No	NO
1	No	Ch	Yes	>30
2	No	No	Yes	NO
3	No	Ch	Yes	NO
4	No	Ch	Yes	NO

[5 rows x 50 columns]

6.1.2 6.0 Raw clinical-data audit

The first inspection of the clinical dataset intentionally happens before preprocessing. If missing values are imputed or categories are encoded too early, the original data quality pattern becomes harder to see.

The audit therefore records shape, file size, missingness, duplicates, target distribution, and the presence of the source missing-value marker. Those observations justify the cleaning decisions that follow.

```
[12]: # Purpose: Execute the main computation for this notebook section.
# Audit missing values, duplicates and target distribution.
diabetes_audit = diabetes_raw.replace("?", np.nan)
missing = pd.DataFrame({
    "missing_count": diabetes_audit.isna().sum(),
    "missing_pct": diabetes_audit.isna().mean() * 100,
}).sort_values("missing_pct", ascending=False)
```

```
display(missing.head(20))
print("Duplicate rows (raw):", int(diabetes_audit.duplicated().sum()))
print("Target distribution:")
display(diabetes_audit["readmitted"].value_counts(dropna=False).
        rename_axis("target").reset_index(name="count"))
```

	missing_count	missing_pct
weight	98569	96.858479
max_glu_serum	96420	94.746772
A1Cresult	84748	83.277322
medical_specialty	49949	49.082208
payer_code	40256	39.557416
race	2273	2.233555
diag_3	1423	1.398306
diag_2	358	0.351787
diag_1	21	0.020636
patient_nbr	0	0.000000
time_in_hospital	0	0.000000
admission_source_id	0	0.000000
num_lab_procedures	0	0.000000
encounter_id	0	0.000000
admission_type_id	0	0.000000
discharge_disposition_id	0	0.000000
gender	0	0.000000
age	0	0.000000
number_inpatient	0	0.000000
number_emergency	0	0.000000

Duplicate rows (raw): 0

Target distribution:

target	count
0 NO	54864
1 >30	35545
2 <30	11357

```
[13]: # Feature validity checks
feature_df = diabetes_audit.drop(columns=["readmitted"], errors="ignore")

numeric_features = feature_df.select_dtypes(include=[np.number])

print("=== Feature validity checks ===")

print("Total features:", feature_df.shape[1])
print("Numeric features:", numeric_features.shape[1])
print("Categorical features:", feature_df.shape[1] - numeric_features.shape[1])

print("\nColumns with 100% missing:")
```

```

print(feature_df.columns[feature_df.isna().all()].tolist())

print("\nNumeric columns with infinite values:")
if not numeric_features.empty:
    inf_counts = np.isinf(numeric_features).sum()
    print(inf_counts[inf_counts > 0])
else:
    print("None")

print("\nUnique values per categorical feature:")
for col in feature_df.select_dtypes(exclude=[np.number]).columns:
    print(f"{col}: {feature_df[col].nunique(dropna=True)} unique")

print("\nDuplicate rows:")
print(int(feature_df.duplicated().sum()))

print("\nFeature matrix contains NaN:", bool(feature_df.isna().any().any()))
print(
    "Feature matrix contains inf:",
    bool(np.isinf(numeric_features.to_numpy()).any())
    if not numeric_features.empty else False
)

```

=== Feature validity checks ===

Total features: 49

Numeric features: 13

Categorical features: 36

Columns with 100% missing:

[]

Numeric columns with infinite values:

Series([], dtype: int64)

Unique values per categorical feature:

race: 5 unique

gender: 3 unique

age: 10 unique

weight: 9 unique

payer_code: 17 unique

medical_specialty: 72 unique

diag_1: 716 unique

diag_2: 748 unique

diag_3: 789 unique

max_glu_serum: 3 unique

A1Cresult: 3 unique

metformin: 4 unique

repaglinide: 4 unique

```
nateglinide: 4 unique
chlorpropamide: 4 unique
glimepiride: 4 unique
acetohexamide: 2 unique
glipizide: 4 unique
glyburide: 4 unique
tolbutamide: 2 unique
pioglitazone: 4 unique
rosiglitazone: 4 unique
acarbose: 4 unique
miglitol: 4 unique
troglitazone: 2 unique
tolazamide: 3 unique
examide: 1 unique
citoglipton: 1 unique
insulin: 4 unique
glyburide-metformin: 4 unique
glipizide-metformin: 2 unique
glimepiride-pioglitazone: 2 unique
metformin-rosiglitazone: 2 unique
metformin-pioglitazone: 2 unique
change: 2 unique
diabetesMed: 2 unique
```

```
Duplicate rows:
```

```
0
```

```
Feature matrix contains NaN: True
```

```
Feature matrix contains inf: False
```

6.1.3 6.0.1 Interpreting missingness

A missingness table answers two questions before any imputation is performed: **which variables are incomplete, and how incomplete are they?**

A high missingness percentage does not automatically prove that a variable must be deleted. The decision depends on how the variable is represented, whether its values are usable at inference time, and whether its absence is itself meaningful. In this assignment, the notebook documents explicit drop decisions and makes the remaining imputation rules visible.

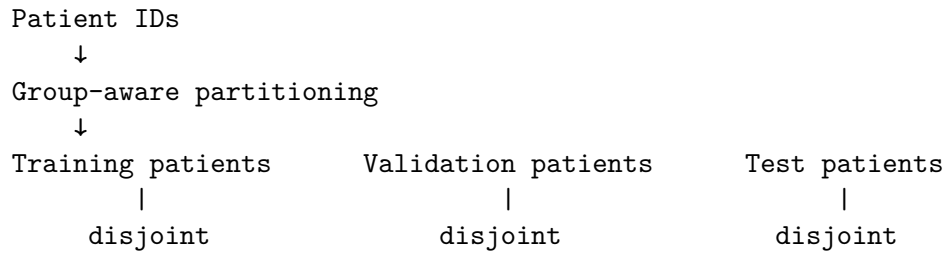
6.2 6.3 Patient leakage: why a random row split is not sufficient

The dataset contains multiple encounters for the same patient. That means the rows are not necessarily independent observations at the patient level.

Consider a patient with several hospital encounters. If some encounters are placed in the training set while another encounter from the same patient is placed in the test set, the model can benefit from patient-specific patterns that were already present during training. The resulting test set is less independent than it appears.

6.2.1 Group-aware splitting principle

The notebook therefore treats `patient_nbr` as a grouping variable during partitioning:



After the split, the notebook explicitly computes the intersection size between the patient ID sets. A correctly separated experiment should report zero overlap for every partition pair.

This is one of the most important methodological controls in the A5 pipeline because it concerns the independence of the evaluation data, not just the appearance of the score.

6.3 6.1 Cleaning strategy for the clinical table

The cleaning process is documented as a sequence of decisions rather than hidden inside one line of code.

6.3.1 Step 1 - Missing-value representation

The original data use `?` as a sentinel for missing information. That sentinel is replaced with `NaN` so that Pandas and scikit-learn can measure and process missingness explicitly.

6.3.2 Step 2 - Target validation

Rows without a valid target are not useful for supervised training and are removed before model construction.

6.3.3 Step 3 - Identifier handling

`encounter_id` identifies the encounter and `patient_nbr` identifies the patient. They are not treated as ordinary predictive features because they can encode identity rather than clinically meaningful signal.

`patient_nbr` is temporarily retained for the group-aware split and removed from the model input afterwards.

6.3.4 Step 4 - Highly missing or unsuitable variables

Variables with extremely high missingness are documented as explicit drop decisions rather than being silently passed into the model. The notebook records which columns are dropped so that the preprocessing is auditable.

6.3.5 Step 5 - Numeric and categorical preprocessing

Numerical columns use median imputation followed by standardization. Categorical columns use most-frequent imputation, ordinal encoding with an explicit unknown value, and standardization.

The ordinal encoding is a compact representation chosen for the Conv1D experiment. It does **not** claim that category code 3 is intrinsically three times or twice as meaningful as category code 1. The code represents category membership so that the network can consume a fixed-length tensor.

6.3.6 Step 6 - Training-only fitting

The preprocessing pipeline is fitted using training rows only. Validation and test rows are transformed using the already-fitted pipeline.

This rule is fundamental:

```
TRAIN: fit preprocessing + transform
VAL  : transform only
TEST : transform only
```

Fitting a scaler or imputer on the complete dataset would allow information from validation/test data to influence the representation used during training.

```
[14]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
def prepare_diabetes(df, seed=42, run_mode="full"):
    """Clean, split, encode, scale, and reshape the Diabetes dataset for Conv1D.
    ↪ """
    df = df.replace("?", np.nan).copy()
    df = df.dropna(subset=["readmitted"]).reset_index(drop=True)

    required_columns = {"patient_nbr", "readmitted"}
    missing_required = required_columns - set(df.columns)
    if missing_required:
        raise KeyError(
            "diabetes_raw is missing required columns: "
            + ", ".join(sorted(missing_required))
            + ". Re-run the Diabetes data-loading cell above."
        )

    # Preserve patient_nbr only for the group-aware split.
    group_ids = df["patient_nbr"].to_numpy()

    # Encode target labels.
    target_map = {"NO": 0, ">30": 1, "<30": 2}
    y = df["readmitted"].map(target_map)
    if y.isna().any():
        bad_targets = sorted(df.loc[y.isna(), "readmitted"].astype(str).
        ↪ unique().tolist())
        raise ValueError(f"Unexpected readmitted values: {bad_targets}")
    y = y.astype(int).to_numpy()

    # Remove identifiers and documented high-missingness fields from model
    ↪ input.
```

```

drop_columns = ["encounter_id", "patient_nbr", "weight", "payer_code"]
X = df.drop(columns=["readmitted"] + drop_columns, errors="ignore")

# Patient-level stratified group split.
sgkf = StratifiedGroupKFold(n_splits=10, shuffle=True, random_state=seed)
folds = list(sgkf.split(X, y, groups=group_ids))

test_idx = folds[0][1]
val_idx = folds[1][1]
train_idx = np.concatenate([folds[k][1] for k in range(2, 10)])

if run_mode == "pilot":
    train_limit = min(PILOT_LIMITS["diabetes_train"], len(train_idx))
    val_limit = min(PILOT_LIMITS["diabetes_val"], len(val_idx))

    train_idx, _ = train_test_split(
        train_idx,
        train_size=train_limit,
        stratify=y[train_idx],
        random_state=seed,
    )
    val_idx, _ = train_test_split(
        val_idx,
        train_size=val_limit,
        stratify=y[val_idx],
        random_state=seed,
    )

X_train_df = X.iloc[train_idx].copy()
X_val_df = X.iloc[val_idx].copy()
X_test_df = X.iloc[test_idx].copy()

y_train = y[train_idx]
y_val = y[val_idx]
y_test = y[test_idx]

# Keep numeric-looking categorical IDs as categorical variables.
numeric_cols = X_train_df.select_dtypes(include=[np.number]).columns.
↪to list()
categorical_cols = [c for c in X_train_df.columns if c not in numeric_cols]

for col in ["admission_type_id", "discharge_disposition_id",
↪"admission_source_id"]:
    if col in numeric_cols:
        numeric_cols.remove(col)
        categorical_cols.append(col)

```

```

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OrdinalEncoder(handle_unknown="use_encoded_value",
unknown_value=-1)),
    ("scaler", StandardScaler()),
])

preprocessor = ColumnTransformer([
    ("num", num_pipe, numeric_cols),
    ("cat", cat_pipe, categorical_cols),
])

# Fit preprocessing on training data only.
X_train = preprocessor.fit_transform(X_train_df).astype(np.float32)
X_val = preprocessor.transform(X_val_df).astype(np.float32)
X_test = preprocessor.transform(X_test_df).astype(np.float32)

# Conv1D input: (samples, sequence_length, channels).
X_train = X_train[..., None]
X_val = X_val[..., None]
X_test = X_test[..., None]

# Class weights from training labels only.
classes = np.unique(y_train)
weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train,
)
class_weight = {int(c): float(w) for c, w in zip(classes, weights)}

# Verify patient IDs do not overlap between partitions.
train_patients = set(group_ids[train_idx])
val_patients = set(group_ids[val_idx])
test_patients = set(group_ids[test_idx])

overlaps = {
    "train_val": len(train_patients & val_patients),
    "train_test": len(train_patients & test_patients),
    "val_test": len(val_patients & test_patients),
}

```

```

return {
    "X_train": X_train,
    "y_train": y_train,
    "X_val": X_val,
    "y_val": y_val,
    "X_test": X_test,
    "y_test": y_test,
    "preprocessor": preprocessor,
    "class_weight": class_weight,
    "dropped_columns": drop_columns,
    "feature_count": X_train.shape[1],
    "numeric_cols": numeric_cols,
    "categorical_cols": categorical_cols,
    "patient_overlap": overlaps,
}

```

```

diabetes_data = prepare_diabetes(diabetes_raw, SEED, RUN_MODE)
X_diab_tr, y_diab_tr = diabetes_data["X_train"], diabetes_data["y_train"]
X_diab_val, y_diab_val = diabetes_data["X_val"], diabetes_data["y_val"]
X_diab_test, y_diab_test = diabetes_data["X_test"], diabetes_data["y_test"]
DIABETES_NAMES = ["NO", ">30", "<30"]

```

```

print("Train:", X_diab_tr.shape, y_diab_tr.shape)
print("Val  :", X_diab_val.shape, y_diab_val.shape)
print("Test :", X_diab_test.shape, y_diab_test.shape)
print("Encoded features:", diabetes_data["feature_count"])
print("Dropped columns:", diabetes_data["dropped_columns"])
print("Patient overlaps:", diabetes_data["patient_overlap"])
print("Class weights:", diabetes_data["class_weight"])

```

```

Train: (81412, 45, 1) (81412,)
Val  : (10177, 45, 1) (10177,)
Test : (10177, 45, 1) (10177,)
Encoded features: 45
Dropped columns: ['encounter_id', 'patient_nbr', 'weight', 'payer_code']
Patient overlaps: {'train_val': 0, 'train_test': 0, 'val_test': 0}
Class weights: {0: 0.6182892468463542, 1: 0.9543301917756835, 2:
2.9870482480278846}

```

6.3.7 6.4 Output of the Diabetes preprocessing pipeline

The processed arrays have the form expected by Conv1D. The final shape is printed together with the feature count, class weights, dropped columns, and patient overlap statistics.

These values are part of the reproducibility record. In particular, a future inference service must preserve the same feature count and feature order. Loading the trained model without the corresponding preprocessing pipeline is not sufficient.

6.4 6.2 Feature Distributions and Target Imbalance

The purpose of the exploratory distributions is not to decorate the notebook. They provide evidence for decisions made later in the pipeline.

6.4.1 Numeric Distributions

Hospital-related variables can have very different scales and skewness. A feature representing a count may concentrate near zero, while another variable may occupy a broader range. Neural-network optimization can be unnecessarily difficult when such scales differ strongly.

The standardization step therefore transforms training features using:

$$z = \frac{x - \mu_{train}}{\sigma_{train}}$$

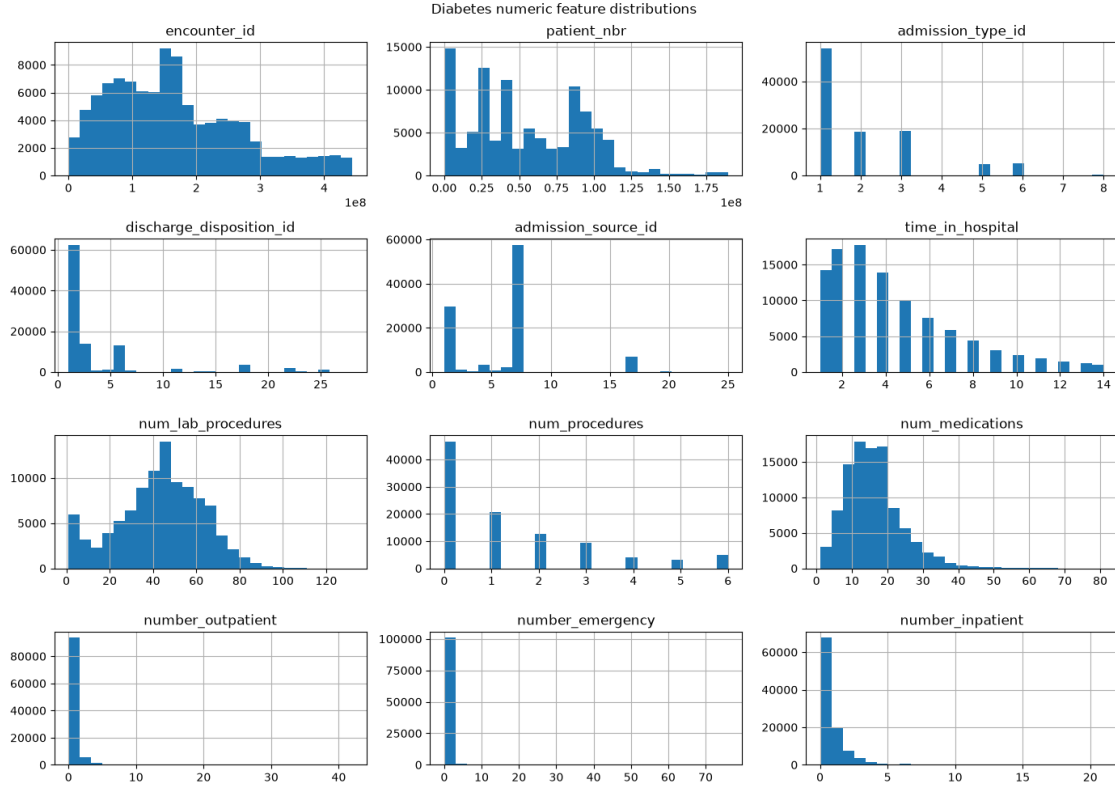
where the mean and standard deviation are estimated from the training partition only.

6.4.2 Target Distribution

The three `readmitted` classes are not equally frequent. This imbalance is why the training loss uses training-derived class weights and why macro-level metrics are reported alongside standard accuracy.

The report should never interpret a good overall accuracy as proof that all three clinical outcomes are being recognized equally well. Per-class recall and the confusion matrix provide that missing detail.

```
[15]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 28.
      numeric_raw = diabetes_audit.select_dtypes(include=[np.number])
      if not numeric_raw.empty:
          numeric_raw.iloc[:, :12].hist(figsize=(14, 10), bins=25)
          plt.suptitle("Diabetes numeric feature distributions")
          plt.tight_layout(); plt.show()
```



6.4.3 6.2.1 Why Class Weights Are Reported

The training loss can be written conceptually as:

$$L = -w_y \log p(y | x)$$

where w_y is the training-derived weight for the true class. The effect is to increase the contribution of under-represented classes to the optimization objective.

The test metrics themselves are still reported on the original test distribution. Class weights change how the model is trained; they do not change the underlying evaluation samples.

7 Part E - Shared evaluation and visualization methodology

7.1 7. Evaluation: what counts as evidence?

A machine-learning experiment should answer more than “how accurate is it?”. A serious evaluation asks whether the model generalizes, whether its performance is distributed across classes, whether its probability estimates are reasonable, and what types of errors remain.

7.1.1 Core metrics

Accuracy is the proportion of correctly classified samples.

Precision asks, for a predicted class, how often the prediction is correct.

Recall asks, for a true class, how many of its examples are detected.

F1-score is the harmonic mean of precision and recall and is useful when both false positives and false negatives matter.

Macro F1 averages class-level F1 equally across classes.

Weighted F1 weights each class by its number of samples.

Log loss evaluates the quality of the entire predicted probability distribution. A model can have the same top-1 accuracy as another model while having worse log loss because it is confidently wrong.

Top-5 accuracy is included for datasets with at least five classes, especially CIFAR-100.

7.1.2 Calibration diagnostics

The notebook also compares confidence against empirical accuracy using a reliability diagram and computes expected calibration error (ECE). These diagnostics answer a different question from accuracy: when the model says it is 90% confident, does it behave like a 90%-reliable predictor on average?

7.1.3 Error analysis

The confusion matrix and misclassified examples provide the qualitative side of the report. Repeated errors can reveal:

- visually similar categories;
- classes with insufficient discriminative information;
- systematic class imbalance effects;
- high-confidence mistakes;
- behavior that differs substantially between CNN3 and CNN5.

```
[16]: # Purpose: Compute consistent metrics and diagnostics for TensorFlow and
      ↪ PyTorch predictions.
def top_k_accuracy(y_true, probs, k=5):
    """Compute top-k classification accuracy.

    Purpose:
        Measure whether the true class appears among the model's k_
    ↪ highest-probability classes.

    Parameters:
        y_true (np.ndarray): Integer target labels.
        probs (np.ndarray): Class probabilities with shape (N, C).
        k (int): Number of highest-probability candidates to consider.

    Returns:
        float: Fraction of samples whose true class is inside the top-k set.
    """
```

```

k = min(k, probs.shape[1])
topk = np.argsort(probs, axis=1)[: , -k:]
return float(np.mean([y in row for y, row in zip(y_true, topk)]))

def ece_score(y_true, probs, n_bins=10):
    """Compute expected calibration error using confidence bins.

    Purpose:
        Quantify the gap between average predicted confidence and empirical
        ↪ accuracy.

    Parameters:
        y_true (np.ndarray): Integer target labels.
        probs (np.ndarray): Class probabilities.
        n_bins (int): Number of confidence intervals.

    Returns:
        float: Expected calibration error in the range approximately 0 to 1.
    """
    conf = probs.max(axis=1)
    pred = probs.argmax(axis=1)
    correct = pred == y_true
    edges = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    for i in range(n_bins):
        lo, hi = edges[i], edges[i+1]
        mask = (conf >= lo) & ((conf < hi) if i < n_bins-1 else (conf <= hi))
        if mask.any():
            # Weight each bin by its frequency so large bins contribute
            ↪ proportionally.
            ece += mask.mean() * abs(correct[mask].mean() - conf[mask].mean())
    return float(ece)

def evaluate_predictions(y_true, probs, class_names):
    """Compute the common multiclass metric set used by the final comparison
    ↪ table.

    Purpose:
        Ensure both frameworks are evaluated with identical metric definitions.

    Parameters:
        y_true (np.ndarray): Ground-truth integer labels.
        probs (np.ndarray): Predicted class probabilities.
        class_names (list): Names used to determine the number of classes for
        ↪ log loss.

```

```

Returns:
    tuple: Dictionary of scalar metrics and the argmax class predictions.
    """
    # Convert probability vectors into the final top-1 class prediction.
    preds = probs.argmax(axis=1)
    p_m, r_m, f1_m, _ = precision_recall_fscore_support(y_true, preds,
↪average="macro", zero_division=0)
    p_w, r_w, f1_w, _ = precision_recall_fscore_support(y_true, preds,
↪average="weighted", zero_division=0)
    out = {
        "accuracy": float(accuracy_score(y_true, preds)),
        "precision_macro": float(p_m),
        "recall_macro": float(r_m),
        "f1_macro": float(f1_m),
        "precision_weighted": float(p_w),
        "recall_weighted": float(r_w),
        "f1_weighted": float(f1_w),
        "log_loss": float(log_loss(y_true, probs, labels=np.
↪arange(len(class_names)))),
        "mean_confidence": float(probs.max(axis=1).mean()),
        "ece": ece_score(y_true, probs),
    }
    if probs.shape[1] >= 5:
        # Top-5 is informative for large class spaces such as CIFAR-100.
        out["top5_accuracy"] = top_k_accuracy(y_true, probs, 5)
    return out, preds

def plot_history(history, title):
    """Plot training and validation loss/accuracy across epochs.

    Purpose:
        Diagnose convergence, plateaus, and potential overfitting from the
↪training trajectory.

    Parameters:
        history (dict): Training history containing loss/accuracy and
↪validation counterparts.
        title (str): Figure title prefix.

    Returns:
        None. Two diagnostic plots are displayed.
        """
    epochs = np.arange(1, len(history["loss"])+1)
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    ax[0].plot(epochs, history["loss"], marker="o", label="train")

```

```

    ax[0].plot(epochs, history["val_loss"], marker="o", label="val")
    ax[0].set_title(title + " - loss"); ax[0].set_xlabel("Epoch"); ax[0].
    ↪set_ylabel("Loss"); ax[0].legend()
    ax[1].plot(epochs, history["accuracy"], marker="o", label="train")
    ax[1].plot(epochs, history["val_accuracy"], marker="o", label="val")
    ax[1].set_title(title + " - accuracy"); ax[1].set_xlabel("Epoch"); ax[1].
    ↪set_ylabel("Accuracy"); ax[1].legend()
    plt.tight_layout(); plt.show()

def plot_confusion(y_true, preds, class_names, title, dataset_key):
    """Plot an annotated confusion matrix and a readable subset for large class_
    ↪spaces.

    Purpose:
        Show exact error counts and, for 100-class tasks, highlight the_
    ↪strongest confusion region.

    Parameters:
        y_true (np.ndarray): Ground-truth labels.
        preds (np.ndarray): Predicted labels.
        class_names (list): Human-readable class names.
        title (str): Model/dataset label used in the figure title.
        dataset_key (str): File-name-safe dataset identifier.

    Returns:
        np.ndarray: Full confusion matrix counts.
    """
    cm = confusion_matrix(y_true, preds)
    n = len(class_names)
    fig, ax = plt.subplots(figsize=(8 if n <= 20 else 12, 8 if n <= 20 else 12))
    im = ax.imshow(cm, cmap="Blues")
    plt.colorbar(im, ax=ax)
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title(title + " -_
    ↪count confusion matrix")
    ax.set_xticks(range(n)); ax.set_yticks(range(n))
    if n <= 20:
        ax.set_xticklabels(class_names, rotation=45, ha="right", fontsize=8)
        ax.set_yticklabels(class_names, fontsize=8)
        threshold = cm.max() * 0.5 if cm.size else 0
        for i in range(n):
            for j in range(n):
                # Annotate every cell for small class spaces so exact counts_
    ↪are visible.
                ax.text(j, i, str(cm[i,j]), ha="center", va="center",_
    ↪fontsize=7,

```

```

        color="white" if cm[i,j] > threshold else "black")
    else:
        ax.set_xticklabels(range(n), fontsize=5); ax.set_yticklabels(range(n),
↪fontsize=5)
        fig.savefig(FIGURES_DIR / f"{dataset_key}_{title.replace(' ', '_').
↪lower()}_cm.png", dpi=160, bbox_inches="tight")
        plt.show()

    if n > 20:
        # Select the classes with the largest off-diagonal confusion mass for
↪readability.
        off = cm.copy(); np.fill_diagonal(off, 0)
        important = np.argsort(off.sum(axis=1))[-20:]
        sub = cm[np.ix_(important, important)]
        names = [class_names[i] for i in important]
        fig, ax = plt.subplots(figsize=(12, 10))
        im = ax.imshow(sub, cmap="Blues"); plt.colorbar(im, ax=ax)
        ax.set_xticks(range(20)); ax.set_yticks(range(20))
        ax.set_xticklabels(names, rotation=60, ha="right", fontsize=7); ax.
↪set_yticklabels(names, fontsize=7)
        for i in range(20):
            for j in range(20):
                # Keep numeric counts visible for the focused confusion subset.
                ax.text(j, i, str(sub[i,j]), ha="center", va="center",
↪fontsize=6,
                color="white" if sub[i,j] > sub.max()*0.5 else "black")
        ax.set_title(title + " - annotated top confusion subset")
        plt.tight_layout(); plt.show()
    return cm

def plot_per_class_metrics(y_true, preds, class_names, title):
    """Plot precision, recall, and F1 for every class.

    Purpose:
        Detect classes that are hidden by an otherwise strong aggregate metric.

    Parameters:
        y_true (np.ndarray): Ground-truth labels.
        preds (np.ndarray): Predicted labels.
        class_names (list): Human-readable labels.
        title (str): Figure title prefix.

    Returns:
        pd.DataFrame: Per-class precision/recall/F1 table.
    """

```

```

    report = classification_report(y_true, preds, target_names=class_names,
    ↪output_dict=True, zero_division=0)
    df = pd.DataFrame(report).T.iloc[:len(class_names)]
    ax = df[["precision", "recall", "f1-score"]].plot(kind="bar", figsize=(12,
    ↪max(5, len(class_names)*0.18)))
    ax.set_ylim(0, 1.05); ax.set_title(title + " - per-class metrics"); ax.
    ↪set_ylabel("Score")
    if len(class_names) <= 20:
        ax.tick_params(axis="x", rotation=35)
    else:
        ax.set_xticks([])
    plt.tight_layout(); plt.show()
    return df

```

def plot_confidence(y_true, probs, title):

"""Plot confidence distributions for correct and incorrect predictions.

Purpose:

Identify whether the model tends to be uncertain or highly confident
 ↪*when making mistakes.*

Parameters:

y_true (np.ndarray): Ground-truth labels.

probs (np.ndarray): Predicted probabilities.

title (str): Figure title prefix.

Returns:

None. Histogram is displayed.

"""

```

    pred = probs.argmax(axis=1)
    conf = probs.max(axis=1)
    correct = pred == y_true
    plt.figure(figsize=(9,4))
    plt.hist(conf[correct], bins=20, alpha=0.7, label="Correct")
    plt.hist(conf[~correct], bins=20, alpha=0.7, label="Incorrect")
    plt.xlabel("Confidence"); plt.ylabel("Count"); plt.title(title + " -
    ↪confidence distribution"); plt.legend(); plt.tight_layout(); plt.show()

```

def plot_reliability(y_true, probs, title, n_bins=10):

"""Plot mean confidence against empirical accuracy across confidence bins.

Purpose:

Visualize probability calibration and compare model confidence with
 ↪*actual reliability.*

Parameters:

y_true (np.ndarray): Ground-truth labels.
probs (np.ndarray): Predicted probabilities.
title (str): Figure title prefix.
n_bins (int): Number of confidence bins.

Returns:

None. Reliability diagram is displayed.

```
"""
conf = probs.max(axis=1); pred = probs.argmax(axis=1); correct = pred == y_true
edges = np.linspace(0, 1, n_bins+1); bin_c=[]; bin_a=[]
for i in range(n_bins):
    mask = (conf >= edges[i]) & ((conf < edges[i+1]) if i < n_bins-1 else (conf <= edges[i+1]))
    if mask.any():
        bin_c.append(conf[mask].mean()); bin_a.append(correct[mask].mean())
plt.figure(figsize=(6,6)); plt.plot([0,1],[0,1], "--", label="Perfect calibration");
plt.plot(bin_c, bin_a, "o-", label="Model")
plt.xlabel("Mean confidence"); plt.ylabel("Accuracy"); plt.title(title + "reliability diagram");
plt.legend(); plt.tight_layout(); plt.show()
```

```
def plot_misclassified(images, y_true, preds, probs, class_names, title, n=12):
    """Display a small set of wrong predictions with true/predicted labels and confidence.
```

Purpose:

Provide concrete examples for qualitative error analysis.

Parameters:

images (np.ndarray): Test images aligned with y_true.
y_true (np.ndarray): True labels.
preds (np.ndarray): Predicted labels.
probs (np.ndarray): Predicted probabilities.
class_names (list): Readable class names.
title (str): Figure title prefix.
n (int): Maximum number of mistakes to show.

Returns:

None. Error examples are displayed when available.

```
"""
wrong = np.where(y_true != preds)[0][:n]
if len(wrong) == 0:
    print("No misclassified examples."); return
cols = 4; rows = math.ceil(len(wrong)/cols)
```

```

plt.figure(figsize=(12,3*rows))
for k, idx in enumerate(wrong):
    ax = plt.subplot(rows, cols, k+1)
    ax.imshow(images[idx], cmap="gray" if images[idx].ndim == 2 else None)
    ax.set_title(f"True: {class_names[y_true[idx]]}\nPred:␣
↪{class_names[preds[idx]]} | {probs[idx].max():.3f}")
    ax.axis("off")
plt.suptitle(title + " - error examples"); plt.tight_layout(); plt.show()

def top_confusion_pairs(y_true, preds, class_names, top_n=15):
    """Return the strongest off-diagonal confusion pairs.

    Purpose:
        Turn a large confusion matrix into a readable list of recurring␣
    ↪class-to-class errors.

    Parameters:
        y_true (np.ndarray): Ground-truth labels.
        preds (np.ndarray): Predicted labels.
        class_names (list): Readable class names.
        top_n (int): Number of pairs to return.

    Returns:
        pd.DataFrame: Columns are count, true_class, and predicted_class.
    """
    cm = confusion_matrix(y_true, preds)
    pairs=[]
    for i in range(len(class_names)):
        for j in range(len(class_names)):
            if i != j and cm[i,j] > 0:
                pairs.append((int(cm[i,j]), class_names[i], class_names[j]))
    pairs.sort(reverse=True)
    return pd.DataFrame(pairs[:top_n],␣
    ↪columns=["count", "true_class", "predicted_class"])

def save_experiment(dataset_key, framework, depth, y_true, probs, metrics):
    """Persist predictions, probabilities, metrics, and confusion counts for␣
    ↪one run.

    Purpose:
        Make every experiment auditable and ready for later web-demo␣
    ↪integration.

    Parameters:
        dataset_key (str): File-safe dataset identifier.

```

```

        framework (str): `tensorflow` or `pytorch`.
        depth (int): CNN convolutional depth (3 or 5).
        y_true (np.ndarray): Ground-truth test labels.
        probs (np.ndarray): Test class probabilities.
        metrics (dict): Scalar metric dictionary.

    Returns:
        None. Files are written under the project output directories.
    """
    prefix = f"{dataset_key}_{framework}_cnn{depth}"
    # Save the full probability vector, not only the final class, so confidence
    ↪ can be analyzed later.
    np.savez_compressed(PREDICTIONS_DIR / f"{prefix}_predictions.npz",
    ↪ y_true=y_true, y_pred=probs.argmax(1), probabilities=probs)
    with open(METRICS_DIR / f"{prefix}.json", "w", encoding="utf-8") as f:
        json.dump(metrics, f, indent=2)
    pd.DataFrame(confusion_matrix(y_true, probs.argmax(1))).to_csv(METRICS_DIR /
    ↪ f"{prefix}_confusion_matrix.csv", index=False)

def evaluate_and_visualize(dataset_key, framework, depth, y_true, probs,
    ↪ class_names, images=None, history=None):
    """Run the complete standardized evaluation and visualization workflow.

    Purpose:
        Keep the evaluation pipeline identical across TensorFlow and PyTorch
    ↪ experiments.

    Parameters:
        dataset_key (str): Dataset identifier.
        framework (str): Framework name.
        depth (int): CNN convolutional depth.
        y_true (np.ndarray): Ground-truth labels.
        probs (np.ndarray): Test probabilities.
        class_names (list): Readable class names.
        images (np.ndarray/None): Test images for visual error examples.
        history (dict/None): Training history for convergence plots.

    Returns:
        tuple: Scalar metrics dictionary and predicted class labels.
    """
    metrics, preds = evaluate_predictions(y_true, probs, class_names)
    metrics.update({"dataset": dataset_key, "framework": framework,
    ↪ "conv_layers": depth})
    print(f"\n{dataset_key} | {framework} | CNN{depth}")
    display(pd.DataFrame([metrics]).T.rename(columns={0: "value"}))

```

```

if history is not None:
    # Training curves diagnose optimization behavior before test metrics
    ↪are interpreted.
    plot_history(history, f"{dataset_key} {framework} CNN{depth}")
    plot_confusion(y_true, preds, class_names, f"{dataset_key} {framework}
    ↪CNN{depth}", dataset_key)
    print("Per-class metrics:")
    display(plot_per_class_metrics(y_true, preds, class_names, f"{dataset_key}
    ↪{framework} CNN{depth}"))
    plot_confidence(y_true, probs, f"{dataset_key} {framework} CNN{depth}")
    plot_reliability(y_true, probs, f"{dataset_key} {framework} CNN{depth}")
    if images is not None:
        plot_misclassified(images, y_true, preds, probs, class_names,
        ↪f"{dataset_key} {framework} CNN{depth}")
    print("Top confusion pairs:")
    display(top_confusion_pairs(y_true, preds, class_names))
    save_experiment(dataset_key, framework, depth, y_true, probs, metrics)
    return metrics, preds

```

7.1.4 7.1 Calibration and confidence are different from accuracy

Accuracy answers whether the top-1 class is correct. Confidence answers how strongly the model prefers its top class. A model can therefore have high accuracy but poor calibration, or modest accuracy but relatively honest confidence.

The reliability diagram compares these two quantities across bins. ECE compresses the same idea into one summary number. Neither should replace the main classification metrics; they complement them.

7.2 7.0 Experimental hyperparameters and model-design rationale

Hyperparameters determine how the optimization problem is solved. They should therefore be explained rather than treated as arbitrary numbers.

Hyperparameter	Value	Purpose and interpretation
Convolution kernel	3 x 3 for images / 3 for Conv1D	Captures local neighborhoods while keeping parameter count manageable
Initial learning rate	0.001	Controls the size of optimizer updates at the beginning of training
Optimizer	Adam	Adaptive gradient-based optimization used consistently across frameworks

Hyperparameter	Value	Purpose and interpretation
Batch size	128	Balances gradient stability, memory usage, and update frequency
Maximum epochs	12	Upper bound; early stopping can terminate earlier
Dropout	0.30	Regularization that randomly removes classifier inputs during training
Batch Normalization	Enabled	Stabilizes intermediate activations and often makes optimization easier
LR reduction factor	0.5	Halves the learning rate when validation progress stalls
Minimum learning rate	1e-6	Prevents the scheduler from reducing the learning rate toward zero
Early-stopping patience	3	Allows temporary validation fluctuations before stopping
Checkpoint criterion	Lowest validation loss	Selects the model according to generalization on held-out validation data

7.2.1 Why the same settings are reused

A5 compares depth and frameworks. If every experiment receives a separate manually tuned learning rate, the study becomes difficult to interpret because the observed difference may come from hyperparameter tuning rather than from depth or framework structure.

The notebook therefore uses the same initial optimization policy and the same conceptual architecture schedule for the corresponding TensorFlow and PyTorch models. The final report should acknowledge that framework implementations can still exhibit small runtime or numerical differences.

```
[17]: # Purpose: Calculate output sizes before training so the classifier dimensions
      ↪are auditable.
def convolution_output_size(size, kernel=3, stride=1, padding=1):
    """Compute the output length of one convolution dimension.

    Purpose:
        Apply the standard convolution output-size formula before constructing
        ↪the model.

    Parameters:
```

```

    size (int): Input spatial or sequence size.
    kernel (int): Kernel size.
    stride (int): Convolution stride.
    padding (int): Padding size on each side.

Returns:
    int: Output size after convolution.
"""
return (size + 2 * padding - kernel) // stride + 1

def print_shape_plan(input_size, pool_after_layers, depth):
    """Print the spatial/sequence size after every convolutional block.

    Purpose:
        Make tensor-shape reasoning explicit before the model is trained.

    Parameters:
        input_size (int): Initial height, width, or sequence length.
        pool_after_layers (set): Zero-based convolution indices followed by
        pooling.
        depth (int): Number of convolutional layers.

    Returns:
        None. Shape transitions are printed to the notebook.
    """
    size = input_size
    print(f"Input size: {size}")
    for layer_id in range(depth):
        # Convolution with padding=1 and kernel=3 preserves the current size.
        size = convolution_output_size(size)
        if layer_id in pool_after_layers:
            # Pool size 2 with stride 2 halves the spatial/sequence dimension
            size //= 2
        print(f"After Conv{layer_id + 1}" + (" + Pool" if layer_id in
        pool_after_layers else "") + f": {size}")

print("Fashion-MNIST shape plan for CNN3")
print_shape_plan(28, {0, 1}, 3)
print("\nFashion-MNIST shape plan for CNN5")
print_shape_plan(28, {0, 1, 3}, 5)
print("\nCIFAR-100 shape plan for CNN5")
print_shape_plan(32, {0, 1, 3}, 5)

```

Fashion-MNIST shape plan for CNN3
Input size: 28

After Conv1 + Pool: 14
After Conv2 + Pool: 7
After Conv3: 7

Fashion-MNIST shape plan for CNN5
Input size: 28
After Conv1 + Pool: 14
After Conv2 + Pool: 7
After Conv3: 7
After Conv4 + Pool: 3
After Conv5: 3

CIFAR-100 shape plan for CNN5
Input size: 32
After Conv1 + Pool: 16
After Conv2 + Pool: 8
After Conv3: 8
After Conv4 + Pool: 4
After Conv5: 4

8 Part F - TensorFlow / Keras implementation

8.1 8. TensorFlow CNN architecture and abstraction level

TensorFlow/Keras represents the CNN as a sequence of reusable layers

Conceptually:

Convolution → ReLU → Pooling → ... → Global aggregation → Classifier

becomes a Keras model built from Conv2D, BatchNormalization, ReLU, MaxPooling2D, GlobalAveragePooling2D, Dropout, and Dense layers. For the tabular dataset, the same idea is expressed with the 1-D counterparts.

8.1.1 Why Batch Normalization is used

Batch Normalization standardizes intermediate activations using batch statistics during training and running estimates during inference. In this experiment it is used as an optimization aid after convolution and before ReLU.

8.1.2 Why Global Average Pooling is used

Instead of flattening a large feature map into a very large dense input, global average pooling reduces each feature map to one average value. This dramatically reduces the number of classifier parameters and makes the comparison of CNN3 and CNN5 focus more on convolutional representation capacity.

8.1.3 Why dropout is placed before the classifier

Dropout is used as a regularizer. During training, a random subset of classifier inputs is suppressed, reducing reliance on a small set of activation patterns. It is disabled automatically during evaluation.

```
[18]: # Purpose: Define, train, and use TensorFlow/Keras CNN3 and CNN5 models for all
      ↪ A5 datasets.
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

def build_tf_cnn(input_shape, num_classes, depth=3, task_type="image"):
    """Build the A5 CNN family in TensorFlow/Keras.

    Purpose:
        Construct a controlled CNN3 or CNN5 architecture for either image data
    ↪ (Conv2D)
        or the assignment-driven tabular adaptation (Conv1D).

    Parameters:
        input_shape (tuple): Shape excluding the batch dimension.
        num_classes (int): Number of output classes.
        depth (int): Number of convolutional layers; only 3 or 5 is allowed.
        task_type (str): `image` for Conv2D or `tabular` for Conv1D.

    Returns:
        keras.Model: Compiled-later Keras model architecture.
    """
    if depth not in (3,5):
        raise ValueError("depth must be 3 or 5")

    # Keras Input makes the expected tensor shape explicit before any layer is
    ↪ added.
    inputs = keras.Input(shape=input_shape)
    x = inputs
    filters = [32,64,128,128,160] if depth == 5 else [32,64,128]
    pool_after = {0,1} if depth == 3 else {0,1,3}

    if task_type == "image":
        # Use the same light augmentation policy for every image model: random
    ↪ horizontal flip.
        x = layers.RandomFlip("horizontal")(x)
        for i in range(depth):
            # BatchNorm stabilizes intermediate activations; ReLU supplies
    ↪ non-linearity.
```

```

        x = layers.Conv2D(filters[i], 3, padding="same", use_bias=False)(x)
        x = layers.BatchNormalization()(x)
        x = layers.ReLU()(x)
        if i in pool_after:
            # Pooling reduces spatial resolution and the amount of later
            ↪computation.
            x = layers.MaxPooling2D(2)(x)
            # Global pooling avoids a large dense layer tied to the spatial
            ↪dimensions.
            x = layers.GlobalAveragePooling2D()(x)
        elif task_type == "tabular":
            for i in range(depth):
                # Conv1D scans local windows of the processed feature sequence.
                x = layers.Conv1D(filters[i], 3, padding="same", use_bias=False)(x)
                x = layers.BatchNormalization()(x)
                x = layers.ReLU()(x)
                if i in pool_after:
                    x = layers.MaxPooling1D(2)(x)
            # Global pooling converts the final sequence representation into one
            ↪vector per sample.
            x = layers.GlobalAveragePooling1D()(x)
        else:
            raise ValueError("task_type must be image or tabular")

        # Dropout regularizes the classifier input during training.
        x = layers.Dropout(0.30)(x)
        # The output layer returns logits; softmax is applied later during
        ↪evaluation.
        outputs = layers.Dense(num_classes)(x)
        return keras.Model(inputs, outputs)

def train_tf(model, X_train, y_train, X_val, y_val, class_weight, model_path,
            ↪log_dir):
    """Train a Keras model with validation monitoring and best-checkpoint
    ↪recovery.

    Purpose:
        Encapsulate compilation, optimization callbacks, timing, and checkpoint
        ↪selection
        while keeping the training configuration consistent across all
        ↪TensorFlow experiments.

    Parameters:
        model (keras.Model): CNN model returned by `build_tf_cnn`.
        X_train, y_train: Training tensors and labels.

```

```

X_val, y_val: Validation tensors and labels.
class_weight (dict/None): Training-derived class weights, if needed.
model_path (Path): Destination for the best `.keras` checkpoint.
log_dir (Path): TensorBoard directory when TensorBoard is available.

Returns:
dict: Training/validation history plus measured training time.
"""
# Compile uses the same learning rate, loss family, and optimizer policy
↳ for all models.
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
)

callbacks = [
    # Stop after repeated validation stagnation and restore the best
    ↳ validation weights.
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
    ↳ restore_best_weights=True),
    # Reduce the learning rate when progress stalls instead of continuing
    ↳ with large updates.
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
    ↳ patience=2, min_lr=1e-6, verbose=1),
    # Persist the best model so inference uses the selected checkpoint
    ↳ rather than the last epoch.
    keras.callbacks.ModelCheckpoint(model_path, monitor="val_loss",
    ↳ save_best_only=True),
]

# TensorBoard is a monitoring tool, not a training requirement, so it is
↳ optional.
try:
    import tensorboard # noqa: F401
    callbacks.append(keras.callbacks.TensorBoard(log_dir=log_dir))
    print("TensorBoard logging enabled.")
except ImportError:
    print("TensorBoard is not installed; continuing without TensorBoard
    ↳ logging.")

start = time.perf_counter()
hist = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,

```

```

        batch_size=BATCH_SIZE,
        shuffle=True,
        class_weight=class_weight,
        callbacks=callbacks,
        verbose=2,
    )
    out = {k:[float(v) for v in vals] for k, vals in hist.history.items()}
    out["training_seconds"] = float(time.perf_counter() - start)
    out["best_epoch"] = int(np.argmin(out["val_loss"]) + 1)
    return out

def predict_tf(model, X):
    """Run TensorFlow inference and convert logits to NumPy probabilities.

    Purpose:
        Standardize the output format so TensorFlow predictions can use the same
        framework-independent evaluation functions as PyTorch predictions.

    Parameters:
        model (keras.Model): Trained Keras model.
        X (np.ndarray): Test input tensor.

    Returns:
        tuple: Logits array and softmax probability array.
    """
    # Predict in batches to avoid unnecessary peak memory usage on larger
    ↪ datasets.
    logits = model.predict(X, batch_size=TEST_BATCH_SIZE, verbose=0)
    probs = tf.nn.softmax(logits, axis=1).numpy()
    return logits, probs

```

8.1.4 8.0.2 TensorFlow model summary before training

Before running the long training loop, inspect `model.summary()` for CNN3 and CNN5. The summary provides a transparent record of:

- input/output shape at each layer;
- parameter count per layer;
- the total number of trainable parameters.

This is particularly useful when checking whether the intended depth really produced the expected architectural difference.

8.2 8.4 What TensorFlow/Keras hides

In this notebook, a call such as:

```
history = model.fit(...)
```

implicitly covers repeated steps such as:

```
Batch selection
  ↓
Forward pass
  ↓
Loss calculation
  ↓
Automatic differentiation
  ↓
Optimizer update
  ↓
Metric calculation
  ↓
Validation pass
```

The important point is not that Keras “does less computation.” The computation still exists. Keras simply provides a higher-level API that hides more implementation detail from the student. ###
TensorBoard handling

TensorBoard logging is treated as optional. If the TensorBoard package is not installed, training continues without it rather than failing the entire experiment. This is a practical engineering improvement because TensorBoard is a monitoring convenience, not a requirement for the CNN itself.

8.3 8.0 Tensor-shape planning before training

Lecture 05 explicitly requires tensor shapes to be calculated before the final classifier is written. This avoids one of the most common CNN implementation mistakes: giving a **Dense** or **Linear** layer an incorrect input dimension.

For example, for a 28 x 28 Fashion-MNIST image, a 3-layer model that performs pooling after the first two convolutional blocks follows the spatial transformation:

```
Input           : 28 x 28
Conv 3, padding 1 : 28 x 28
Pool 2          : 14 x 14
Conv 3, padding 1 : 14 x 14
Pool 2          : 7 x 7
Conv 3, padding 1 : 7 x 7
GlobalAvgPool    : 1 x 1 per feature map
```

Because global average pooling is used, the classifier input is the number of final filters rather than **channels x height x width**. This makes the architecture less sensitive to the exact spatial size and keeps the parameter count moderate.

For CIFAR-100 the same principle starts from 32 x 32 x 3, and for Diabetes the Conv1D input starts from **sequence_length x 1**. The notebook prints shape plans before training so that architecture assumptions are visible.

```
[19]: # Purpose: Execute the main computation for this notebook section.
# Inspect the exact layer sequence and parameter counts before training.
for depth in (3, 5):
    model = build_tf_cnn((28, 28, 1), 10, depth, "image")
    print(f"Fashion-MNIST TensorFlow CNN{depth} parameters: {model.
↪count_params():,}")
    for layer in model.layers:
        print(f"{layer.name:24s} -> {layer.output.shape}")
    print()
```

Fashion-MNIST TensorFlow CNN3 parameters: 94,634

```
input_layer          -> (None, 28, 28, 1)
random_flip          -> (None, 28, 28, 1)
conv2d               -> (None, 28, 28, 32)
batch_normalization  -> (None, 28, 28, 32)
re_lu                -> (None, 28, 28, 32)
max_pooling2d        -> (None, 14, 14, 32)
conv2d_1             -> (None, 14, 14, 64)
batch_normalization_1 -> (None, 14, 14, 64)
re_lu_1              -> (None, 14, 14, 64)
max_pooling2d_1      -> (None, 7, 7, 64)
conv2d_2             -> (None, 7, 7, 128)
batch_normalization_2 -> (None, 7, 7, 128)
re_lu_2              -> (None, 7, 7, 128)
global_average_pooling2d -> (None, 128)
dropout              -> (None, 128)
dense                -> (None, 10)
```

Fashion-MNIST TensorFlow CNN5 parameters: 427,882

```
input_layer_1        -> (None, 28, 28, 1)
random_flip_1        -> (None, 28, 28, 1)
conv2d_3             -> (None, 28, 28, 32)
batch_normalization_3 -> (None, 28, 28, 32)
re_lu_3              -> (None, 28, 28, 32)
max_pooling2d_2      -> (None, 14, 14, 32)
conv2d_4             -> (None, 14, 14, 64)
batch_normalization_4 -> (None, 14, 14, 64)
re_lu_4              -> (None, 14, 14, 64)
max_pooling2d_3      -> (None, 7, 7, 64)
conv2d_5             -> (None, 7, 7, 128)
batch_normalization_5 -> (None, 7, 7, 128)
re_lu_5              -> (None, 7, 7, 128)
conv2d_6             -> (None, 7, 7, 128)
batch_normalization_6 -> (None, 7, 7, 128)
re_lu_6              -> (None, 7, 7, 128)
max_pooling2d_4      -> (None, 3, 3, 128)
conv2d_7             -> (None, 3, 3, 160)
batch_normalization_7 -> (None, 3, 3, 160)
```

```

re_lu_7                -> (None, 3, 3, 160)
global_average_pooling2d_1 -> (None, 160)
dropout_1              -> (None, 160)
dense_1                -> (None, 10)

```

8.4 8.1 TensorFlow - Fashion-MNIST CNN3 and CNN5

The Fashion-MNIST experiment trains two models that differ only in convolutional depth within the defined architecture family.

8.4.1 CNN3

- convolutional filters: 32, 64, 128;
- kernel size: 3 x 3;
- BatchNorm and ReLU after each convolution;
- pooling after the first two blocks;
- global average pooling;
- dropout 0.30;
- 10-class output layer.

8.4.2 CNN5

- convolutional filters: 32, 64, 128, 128, 160;
- kernel size: 3 x 3;
- BatchNorm and ReLU after each convolution;
- pooling after blocks 1, 2, and 4;
- global average pooling;
- dropout 0.30;
- 10-class output layer.

The code records parameter count before training and stores the best validation checkpoint. The final test set is evaluated only after training and checkpoint restoration.

```

[20]: # Purpose: Inspect TensorFlow CNN3/CNN5 architecture and parameter counts
      ↪ before training.
      # This is a structural verification step, not a training step.
      for depth in (3, 5):
          demo_model = build_tf_cnn((28, 28, 1), 10, depth, "image")
          print(f"\nFashion-MNIST TensorFlow CNN{depth}")
          demo_model.summary()

```

Fashion-MNIST TensorFlow CNN3

Model: "functional_2"

Layer (type)	Output Shape	
↪Param #		

input_layer_2 (InputLayer)	(None, 28, 28, 1)	└
↪ 0		
random_flip_2 (RandomFlip)	(None, 28, 28, 1)	└
↪ 0		
conv2d_8 (Conv2D)	(None, 28, 28, 32)	└
↪288		
batch_normalization_8	(None, 28, 28, 32)	└
↪128		
(BatchNormalization)		└
↪		
re_lu_8 (ReLU)	(None, 28, 28, 32)	└
↪ 0		
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 32)	└
↪ 0		
conv2d_9 (Conv2D)	(None, 14, 14, 64)	└
↪18,432		
batch_normalization_9	(None, 14, 14, 64)	└
↪256		
(BatchNormalization)		└
↪		
re_lu_9 (ReLU)	(None, 14, 14, 64)	└
↪ 0		
max_pooling2d_6 (MaxPooling2D)	(None, 7, 7, 64)	└
↪ 0		
conv2d_10 (Conv2D)	(None, 7, 7, 128)	└
↪73,728		
batch_normalization_10	(None, 7, 7, 128)	└
↪512		
(BatchNormalization)		└
↪		
re_lu_10 (ReLU)	(None, 7, 7, 128)	└
↪ 0		

global_average_pooling2d_2	(None, 128)	⌵
↪ 0		
(GlobalAveragePooling2D)		⌵
↪		

dropout_2 (Dropout)	(None, 128)	⌵
↪ 0		

dense_2 (Dense)	(None, 10)	⌵
↪ 1,290		

Total params: 94,634 (369.66 KB)

Trainable params: 94,186 (367.91 KB)

Non-trainable params: 448 (1.75 KB)

Fashion-MNIST TensorFlow CNN5

Model: "functional_3"

Layer (type)	Output Shape	⌵
↪ Param #		
input_layer_3 (InputLayer)	(None, 28, 28, 1)	⌵
↪ 0		
random_flip_3 (RandomFlip)	(None, 28, 28, 1)	⌵
↪ 0		
conv2d_11 (Conv2D)	(None, 28, 28, 32)	⌵
↪ 288		
batch_normalization_11	(None, 28, 28, 32)	⌵
↪ 128		
(BatchNormalization)		⌵
↪		
re_lu_11 (ReLU)	(None, 28, 28, 32)	⌵
↪ 0		

max_pooling2d_7 (MaxPooling2D)	(None, 14, 14, 32)	└
↪ 0		
conv2d_12 (Conv2D)	(None, 14, 14, 64)	└
↪18,432		
batch_normalization_12	(None, 14, 14, 64)	└
↪256		
(BatchNormalization)		└
↪		
re_lu_12 (ReLU)	(None, 14, 14, 64)	└
↪ 0		
max_pooling2d_8 (MaxPooling2D)	(None, 7, 7, 64)	└
↪ 0		
conv2d_13 (Conv2D)	(None, 7, 7, 128)	└
↪73,728		
batch_normalization_13	(None, 7, 7, 128)	└
↪512		
(BatchNormalization)		└
↪		
re_lu_13 (ReLU)	(None, 7, 7, 128)	└
↪ 0		
conv2d_14 (Conv2D)	(None, 7, 7, 128)	└
↪147,456		
batch_normalization_14	(None, 7, 7, 128)	└
↪512		
(BatchNormalization)		└
↪		
re_lu_14 (ReLU)	(None, 7, 7, 128)	└
↪ 0		
max_pooling2d_9 (MaxPooling2D)	(None, 3, 3, 128)	└
↪ 0		
conv2d_15 (Conv2D)	(None, 3, 3, 160)	└
↪184,320		

batch_normalization_15	(None, 3, 3, 160)	↳
↳ 640		
(BatchNormalization)		↳
↳		
re_lu_15 (ReLU)	(None, 3, 3, 160)	↳
↳ 0		
global_average_pooling2d_3	(None, 160)	↳
↳ 0		
(GlobalAveragePooling2D)		↳
↳		
dropout_3 (Dropout)	(None, 160)	↳
↳ 0		
dense_3 (Dense)	(None, 10)	↳
↳ 1,610		

Total params: 427,882 (1.63 MB)

Trainable params: 426,858 (1.63 MB)

Non-trainable params: 1,024 (4.00 KB)

```
[21]: # Purpose: Run the computation defined by this section. The detailed comments
      ↳ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 36.
      TF_RESULTS = {}
      for depth in (3,5):
          path = MODELS_DIR / f"fashion_tensorflow_cnn{depth}.keras"
          model = build_tf_cnn((28,28,1), 10, depth, "image")
          print(f"\nTensorFlow Fashion CNN{depth} | params: {model.count_params():,}")
          history = train_tf(
              model,
              X_fashion_tr[...,None], y_fashion_tr,
              X_fashion_val[...,None], y_fashion_val,
              None, path,
              str(OUTPUTS_DIR / "tensorboard" / f"fashion_tf_cnn{depth}"),
          )
          _, probs = predict_tf(model, X_fashion_test[...,None])
          metrics, _ = evaluate_and_visualize("fashion_mnist", "tensorflow", depth,
      ↳ y_fashion_test, probs, FASHION_NAMES, X_fashion_test, history)
```

```

metrics["parameters"] = int(model.count_params())
metrics["training_seconds"] = float(history["training_seconds"])
TF_RESULTS[("fashion", depth)] = metrics

# Re-save the final metric record after runtime and parameter fields are
↳attached.
save_experiment("fashion_mnist", "tensorflow", depth, y_fashion_test, probs,
↳metrics)

```

TensorFlow Fashion CNN3 | params: 94,634

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.

Please use WSL2 or the TensorFlow-DirectML plugin.

TensorBoard is not installed; continuing without TensorBoard logging.

Epoch 1/12

422/422 - 30s - 71ms/step - accuracy: 0.7599 - loss: 0.7117 - val_accuracy: 0.5072 - val_loss: 1.4469 - learning_rate: 0.0010

Epoch 2/12

422/422 - 36s - 86ms/step - accuracy: 0.8402 - loss: 0.4554 - val_accuracy: 0.8217 - val_loss: 0.5073 - learning_rate: 0.0010

Epoch 3/12

422/422 - 38s - 90ms/step - accuracy: 0.8598 - loss: 0.3988 - val_accuracy: 0.8147 - val_loss: 0.5140 - learning_rate: 0.0010

Epoch 4/12

422/422 - 38s - 91ms/step - accuracy: 0.8692 - loss: 0.3698 - val_accuracy: 0.8407 - val_loss: 0.4311 - learning_rate: 0.0010

Epoch 5/12

422/422 - 38s - 91ms/step - accuracy: 0.8773 - loss: 0.3455 - val_accuracy: 0.8630 - val_loss: 0.3820 - learning_rate: 0.0010

Epoch 6/12

422/422 - 39s - 91ms/step - accuracy: 0.8817 - loss: 0.3300 - val_accuracy: 0.8638 - val_loss: 0.3858 - learning_rate: 0.0010

Epoch 7/12

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

422/422 - 38s - 91ms/step - accuracy: 0.8873 - loss: 0.3162 - val_accuracy: 0.8620 - val_loss: 0.3952 - learning_rate: 0.0010

Epoch 8/12

422/422 - 38s - 90ms/step - accuracy: 0.8988 - loss: 0.2892 - val_accuracy: 0.8770 - val_loss: 0.3289 - learning_rate: 5.0000e-04

Epoch 9/12

422/422 - 38s - 90ms/step - accuracy: 0.8995 - loss: 0.2831 - val_accuracy: 0.8908 - val_loss: 0.3068 - learning_rate: 5.0000e-04

Epoch 10/12

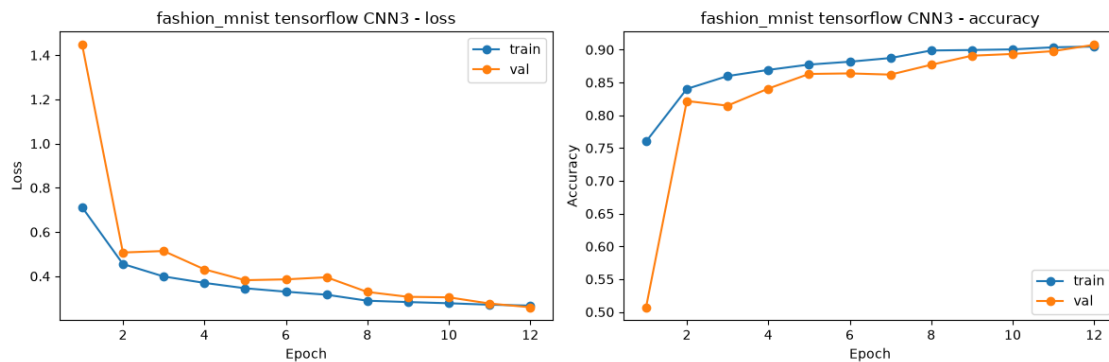
422/422 - 39s - 93ms/step - accuracy: 0.9005 - loss: 0.2778 - val_accuracy: 0.8935 - val_loss: 0.3044 - learning_rate: 5.0000e-04

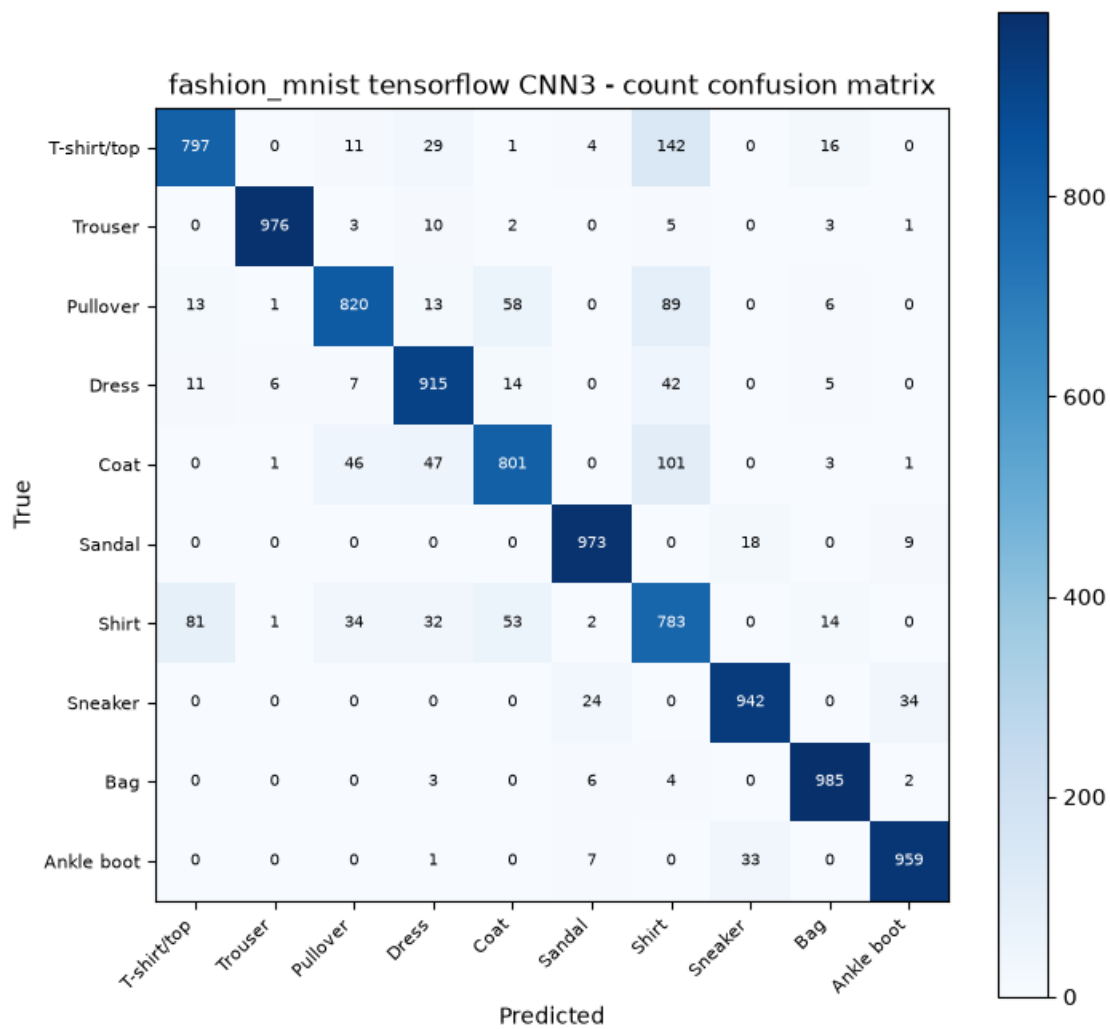
Epoch 11/12

422/422 - 38s - 90ms/step - accuracy: 0.9037 - loss: 0.2712 - val_accuracy:
0.8978 - val_loss: 0.2765 - learning_rate: 5.0000e-04
Epoch 12/12
422/422 - 38s - 90ms/step - accuracy: 0.9048 - loss: 0.2671 - val_accuracy:
0.9075 - val_loss: 0.2596 - learning_rate: 5.0000e-04

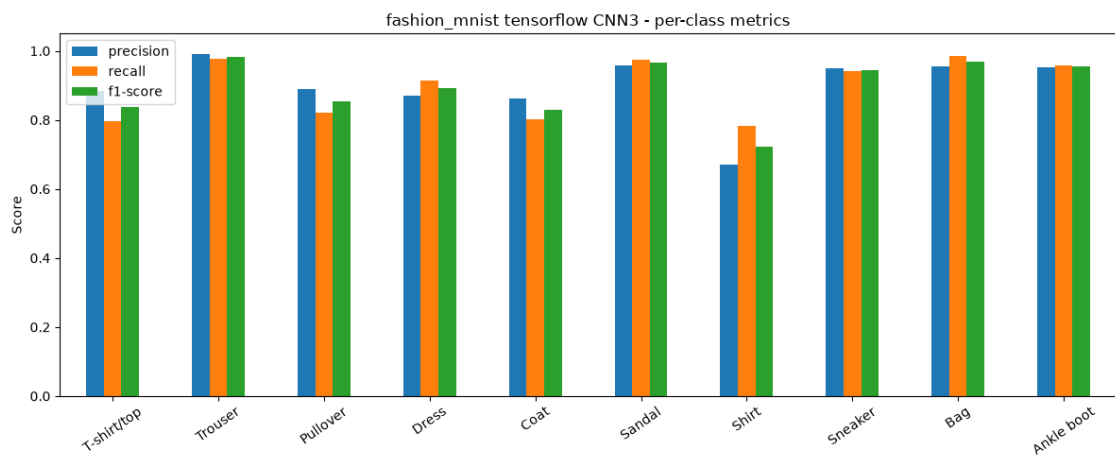
fashion_mnist | tensorflow | CNN3

	value
accuracy	0.8951
precision_macro	0.898402
recall_macro	0.8951
f1_macro	0.895753
precision_weighted	0.898402
recall_weighted	0.8951
f1_weighted	0.895753
log_loss	0.294354
mean_confidence	0.89921
ece	0.00463
top5_accuracy	0.9978
dataset	fashion_mnist
framework	tensorflow
conv_layers	3

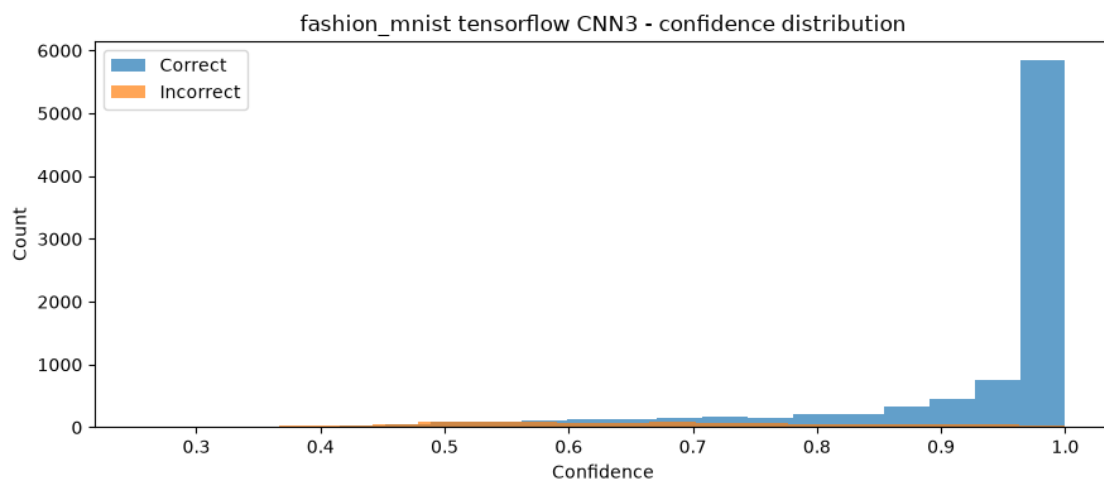


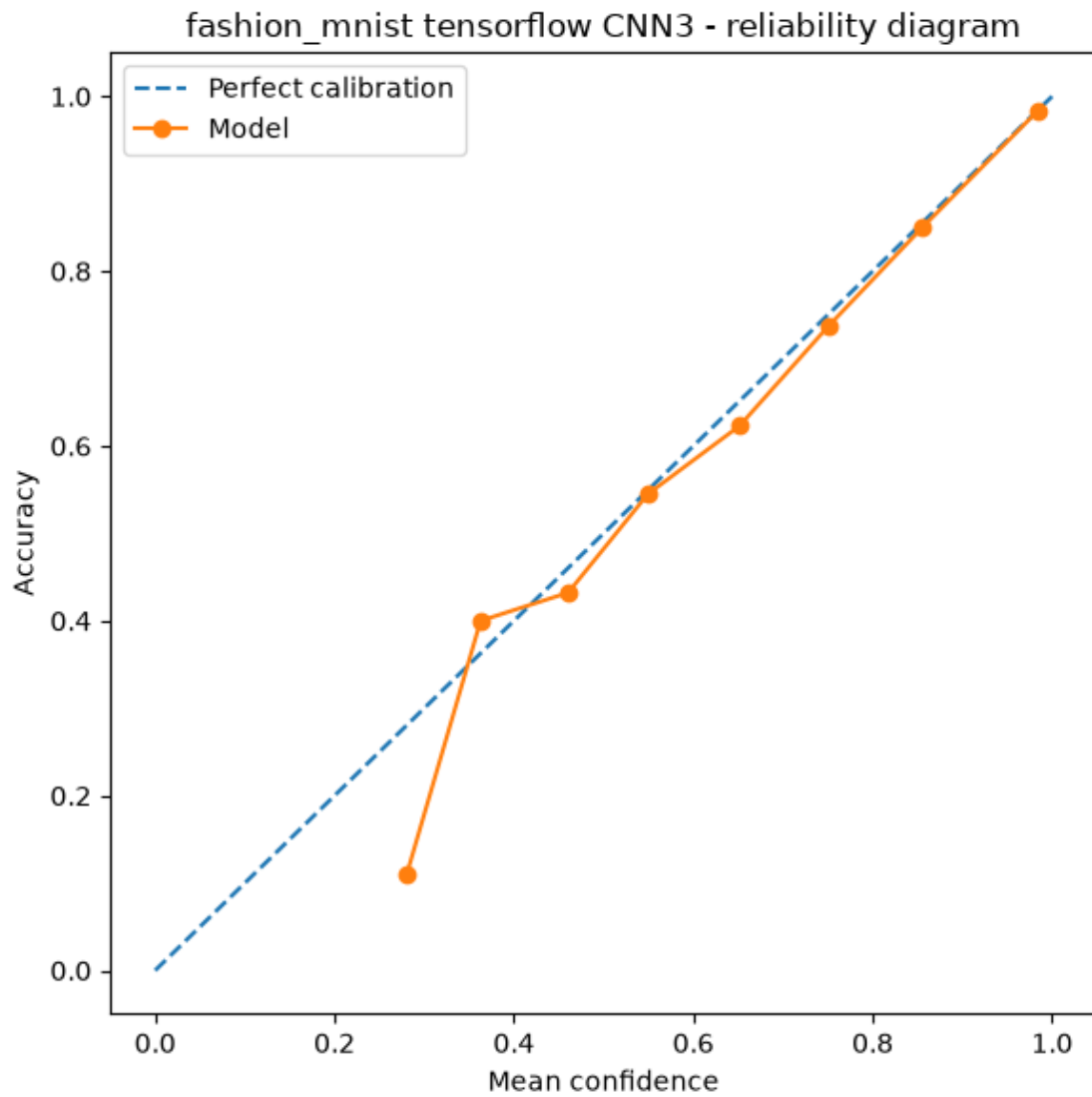


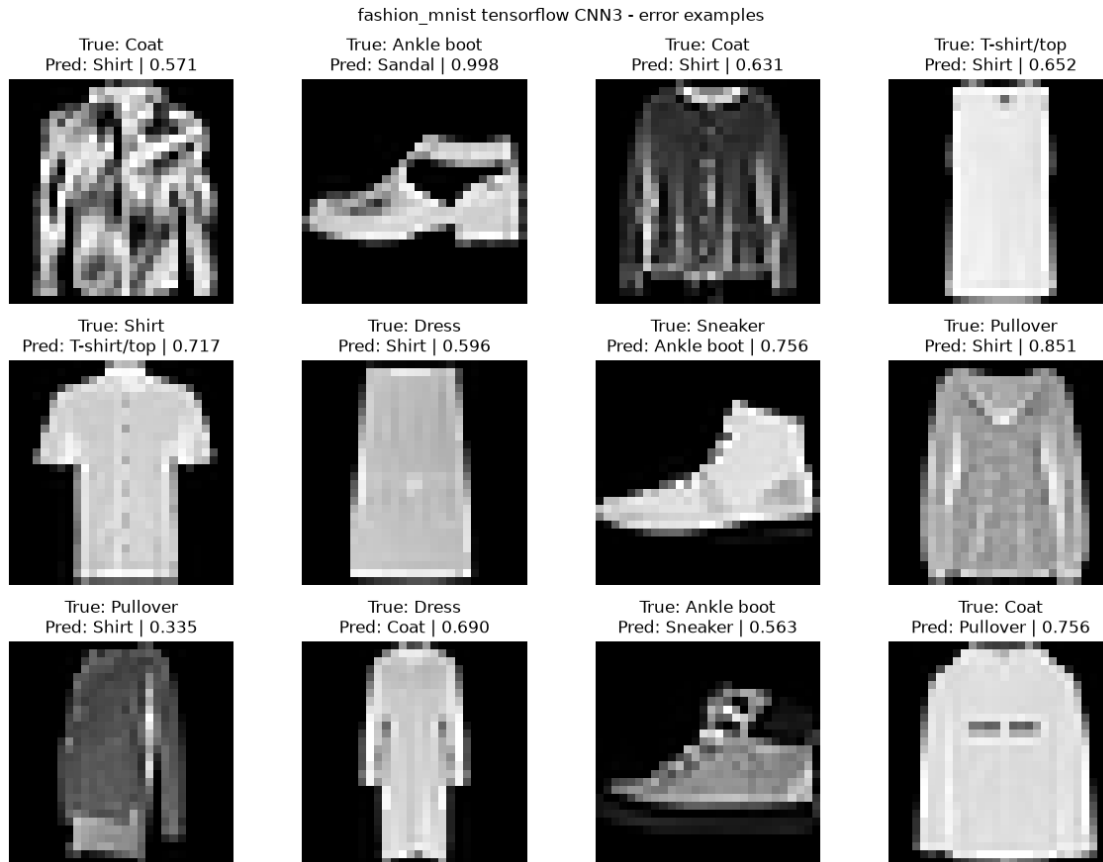
Per-class metrics:



	precision	recall	f1-score	support
T-shirt/top	0.883592	0.797	0.838065	1000.0
Trouser	0.990863	0.976	0.983375	1000.0
Pullover	0.890337	0.820	0.853722	1000.0
Dress	0.871429	0.915	0.892683	1000.0
Coat	0.862217	0.801	0.830482	1000.0
Sandal	0.957677	0.973	0.965278	1000.0
Shirt	0.671527	0.783	0.722992	1000.0
Sneaker	0.948640	0.942	0.945309	1000.0
Bag	0.954457	0.985	0.969488	1000.0
Ankle boot	0.953280	0.959	0.956132	1000.0







Top confusion pairs:

	count	true_class	predicted_class
0	142	T-shirt/top	Shirt
1	101	Coat	Shirt
2	89	Pullover	Shirt
3	81	Shirt	T-shirt/top
4	58	Pullover	Coat
5	53	Shirt	Coat
6	47	Coat	Dress
7	46	Coat	Pullover
8	42	Dress	Shirt
9	34	Sneaker	Ankle boot
10	34	Shirt	Pullover
11	33	Ankle boot	Sneaker
12	32	Shirt	Dress
13	29	T-shirt/top	Dress
14	24	Sneaker	Sandal

TensorFlow Fashion CNN5 | params: 427,882

TensorBoard is not installed; continuing without TensorBoard logging.

Epoch 1/12

422/422 - 60s - 141ms/step - accuracy: 0.8450 - loss: 0.4370 - val_accuracy: 0.6522 - val_loss: 1.0457 - learning_rate: 0.0010

Epoch 2/12

422/422 - 59s - 139ms/step - accuracy: 0.8965 - loss: 0.2871 - val_accuracy: 0.8885 - val_loss: 0.3086 - learning_rate: 0.0010

Epoch 3/12

422/422 - 59s - 141ms/step - accuracy: 0.9113 - loss: 0.2441 - val_accuracy: 0.8767 - val_loss: 0.3371 - learning_rate: 0.0010

Epoch 4/12

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

422/422 - 59s - 141ms/step - accuracy: 0.9219 - loss: 0.2155 - val_accuracy: 0.8860 - val_loss: 0.3090 - learning_rate: 0.0010

Epoch 5/12

422/422 - 59s - 139ms/step - accuracy: 0.9373 - loss: 0.1741 - val_accuracy: 0.9137 - val_loss: 0.2326 - learning_rate: 5.0000e-04

Epoch 6/12

422/422 - 59s - 140ms/step - accuracy: 0.9444 - loss: 0.1570 - val_accuracy: 0.9158 - val_loss: 0.2282 - learning_rate: 5.0000e-04

Epoch 7/12

422/422 - 60s - 142ms/step - accuracy: 0.9472 - loss: 0.1452 - val_accuracy: 0.9260 - val_loss: 0.2015 - learning_rate: 5.0000e-04

Epoch 8/12

422/422 - 58s - 138ms/step - accuracy: 0.9526 - loss: 0.1320 - val_accuracy: 0.9167 - val_loss: 0.2300 - learning_rate: 5.0000e-04

Epoch 9/12

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

422/422 - 60s - 141ms/step - accuracy: 0.9569 - loss: 0.1203 - val_accuracy: 0.9095 - val_loss: 0.2698 - learning_rate: 5.0000e-04

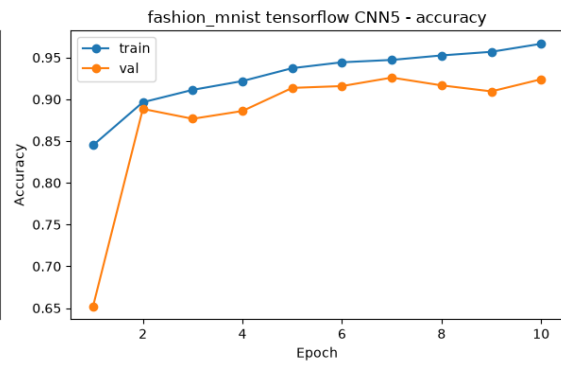
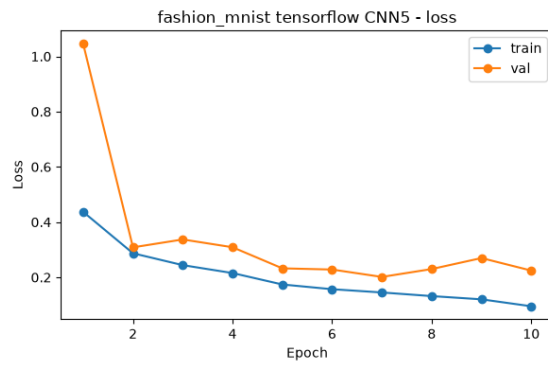
Epoch 10/12

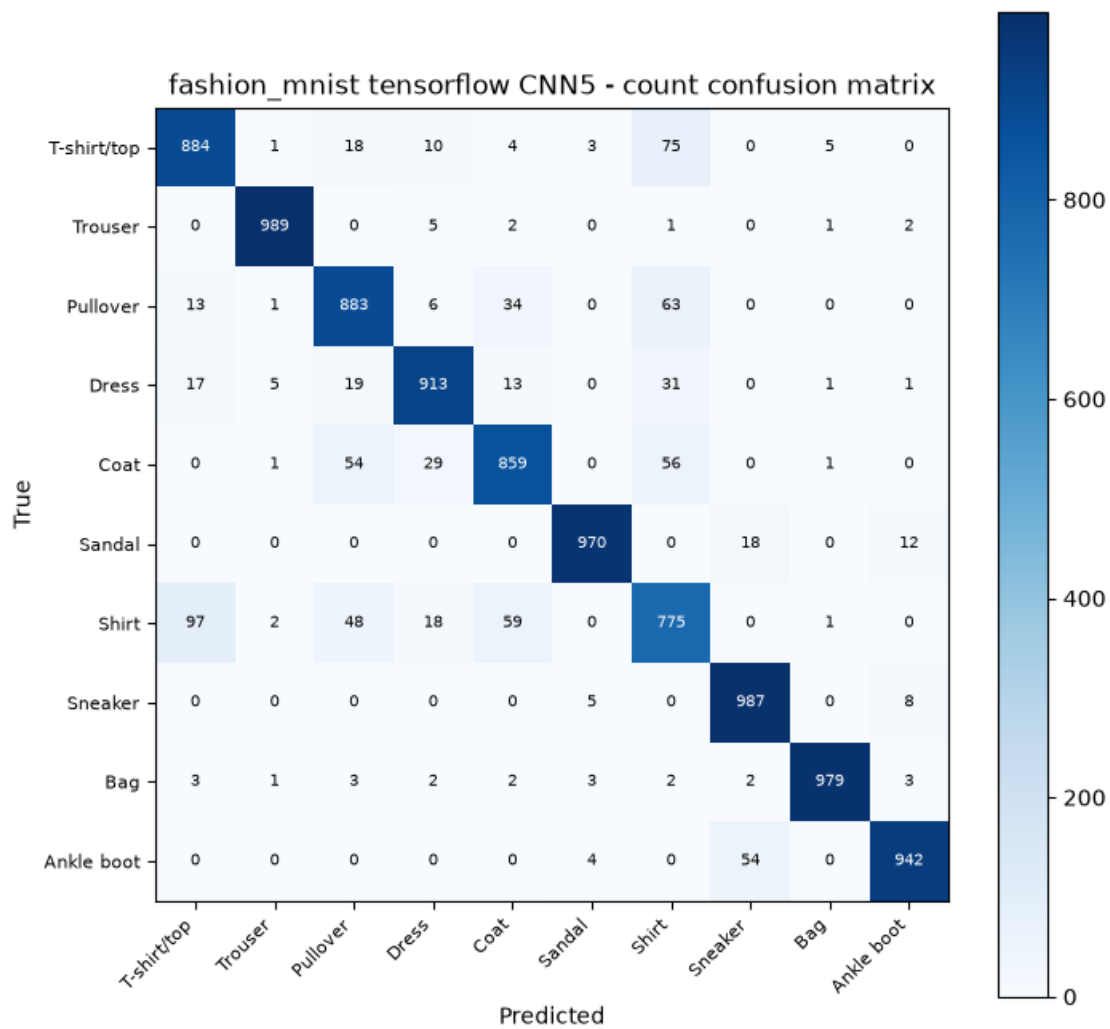
422/422 - 71s - 169ms/step - accuracy: 0.9665 - loss: 0.0953 - val_accuracy: 0.9240 - val_loss: 0.2245 - learning_rate: 2.5000e-04

fashion_mnist | tensorflow | CNN5

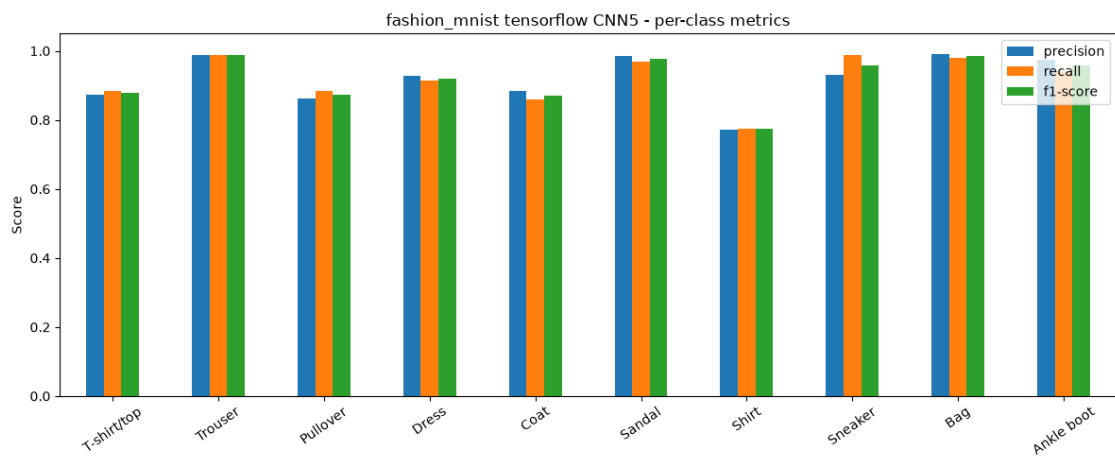
	value
accuracy	0.9181
precision_macro	0.918562
recall_macro	0.9181
f1_macro	0.918172
precision_weighted	0.918562
recall_weighted	0.9181
f1_weighted	0.918172
log_loss	0.229283
mean_confidence	0.934951

ece 0.017132
top5_accuracy 0.9992
dataset fashion_mnist
framework tensorflow
conv_layers 5

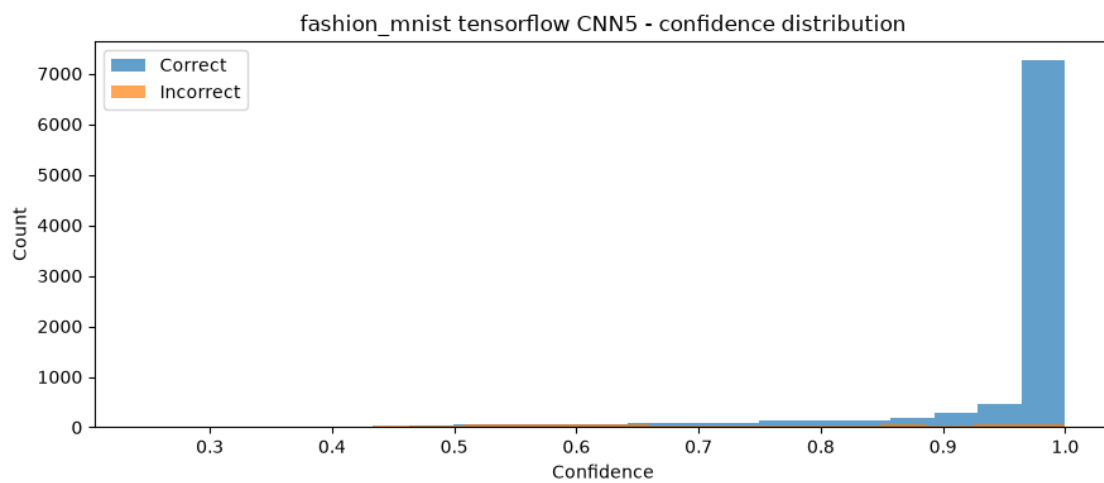


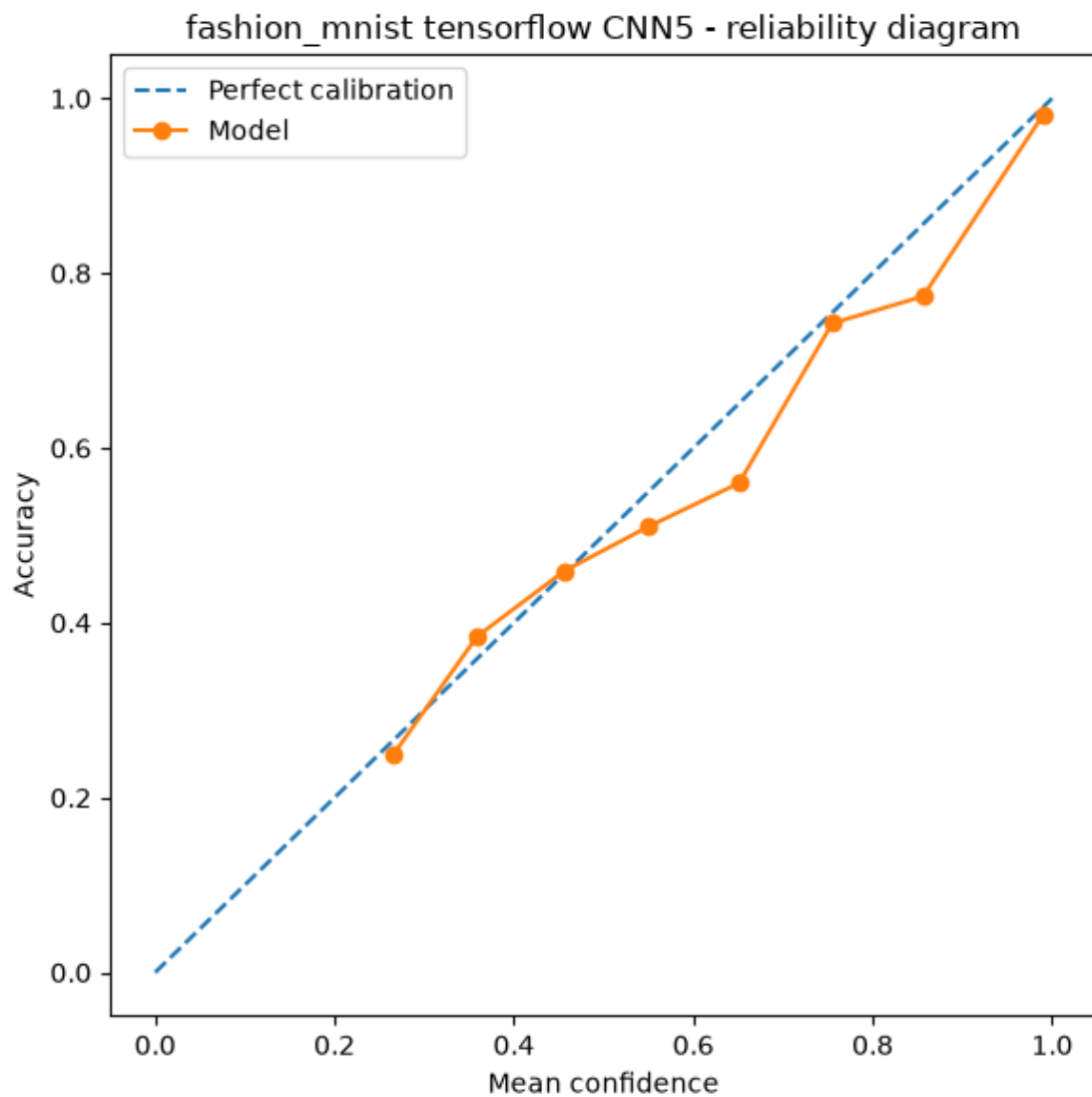


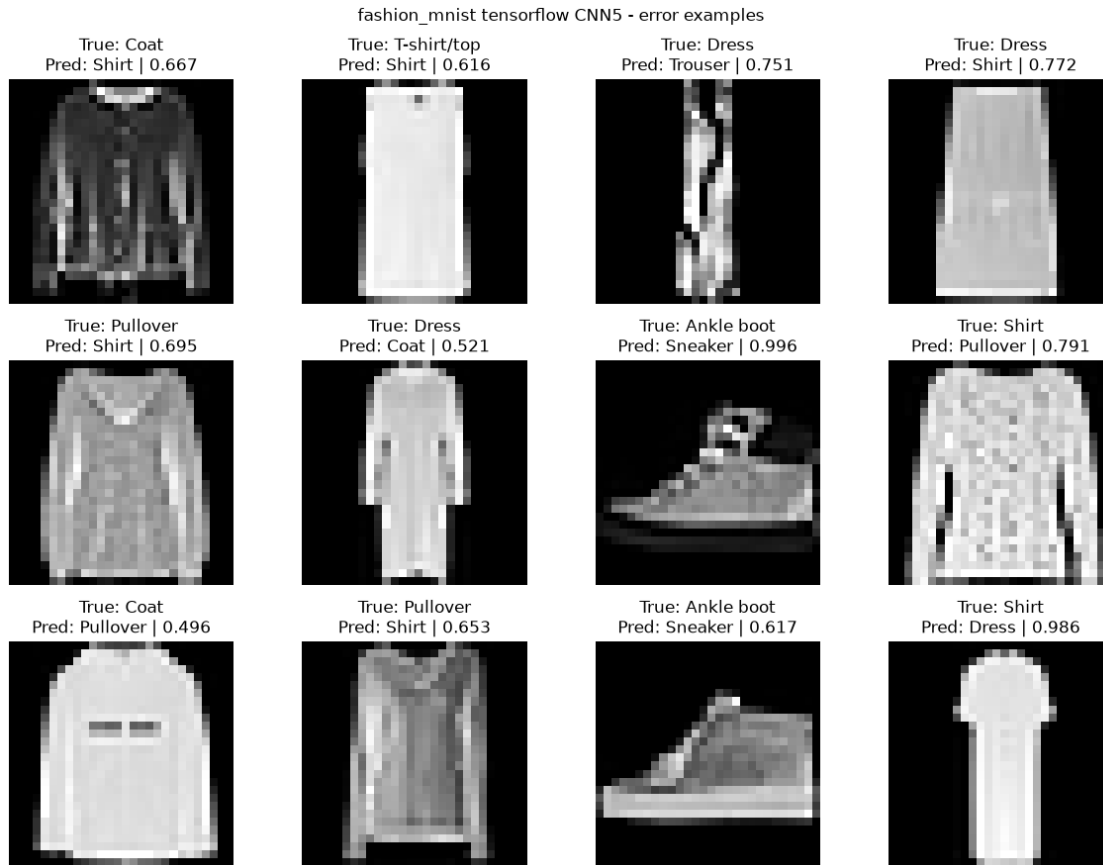
Per-class metrics:



	precision	recall	f1-score	support
T-shirt/top	0.871795	0.884	0.877855	1000.0
Trouser	0.989000	0.989	0.989000	1000.0
Pullover	0.861463	0.883	0.872099	1000.0
Dress	0.928789	0.913	0.920827	1000.0
Coat	0.882837	0.859	0.870755	1000.0
Sandal	0.984772	0.970	0.977330	1000.0
Shirt	0.772682	0.775	0.773839	1000.0
Sneaker	0.930254	0.987	0.957787	1000.0
Bag	0.990891	0.979	0.984909	1000.0
Ankle boot	0.973140	0.942	0.957317	1000.0







Top confusion pairs:

	count	true_class	predicted_class
0	97	Shirt	T-shirt/top
1	75	T-shirt/top	Shirt
2	63	Pullover	Shirt
3	59	Shirt	Coat
4	56	Coat	Shirt
5	54	Coat	Pullover
6	54	Ankle boot	Sneaker
7	48	Shirt	Pullover
8	34	Pullover	Coat
9	31	Dress	Shirt
10	29	Coat	Dress
11	19	Dress	Pullover
12	18	T-shirt/top	Pullover
13	18	Shirt	Dress
14	18	Sandal	Sneaker

8.5 8.2 TensorFlow - CIFAR-100 CNN3 and CNN5

The CIFAR-100 TensorFlow models reuse the same depth definitions as Fashion-MNIST so that the comparison remains about representation complexity rather than an unrelated redesign.

The key input difference is:

```
Fashion-MNIST : channels = 1, classes = 10
CIFAR-100     : channels = 3, classes = 100
```

The final classifier therefore produces 100 logits for CIFAR-100. The output dimension is determined by the number of classes, not by the convolutional depth.

Because CIFAR-100 is a 100-class problem, the evaluation section also reports top-5 accuracy and produces a readable top-confusion-pair summary in addition to the full confusion matrix.

```
[22]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 38.
      for depth in (3,5):
          path = MODELS_DIR / f"cifar100_tensorflow_cnn{depth}.keras"
          model = build_tf_cnn((32,32,3), 100, depth, "image")
          print(f"\nTensorFlow CIFAR-100 CNN{depth} | params: {model.count_params():
          ↪ ,}")
          history = train_tf(
              model,
              X_cifar_tr, y_cifar_tr,
              X_cifar_val, y_cifar_val,
              None, path,
              str(OUTPUTS_DIR / "tensorboard" / f"cifar100_tf_cnn{depth}"),
          )
          _, probs = predict_tf(model, X_cifar_test)
          metrics, _ = evaluate_and_visualize("cifar100", "tensorflow", depth,
          ↪ y_cifar_test, probs, CIFAR100_NAMES, X_cifar_test, history)
          metrics["parameters"] = int(model.count_params())
          metrics["training_seconds"] = float(history["training_seconds"])
          TF_RESULTS[("cifar100", depth)] = metrics

      # Re-save the final metric record after runtime and parameter fields are
      ↪ attached.
      save_experiment("cifar100", "tensorflow", depth, y_cifar_test, probs, metrics)
```

TensorFlow CIFAR-100 CNN3 | params: 106,820

TensorBoard is not installed; continuing without TensorBoard logging.

Epoch 1/12

352/352 - 44s - 125ms/step - accuracy: 0.0959 - loss: 3.9840 - val_accuracy:
0.0350 - val_loss: 4.8192 - learning_rate: 0.0010

Epoch 2/12

352/352 - 41s - 117ms/step - accuracy: 0.1722 - loss: 3.4627 - val_accuracy:

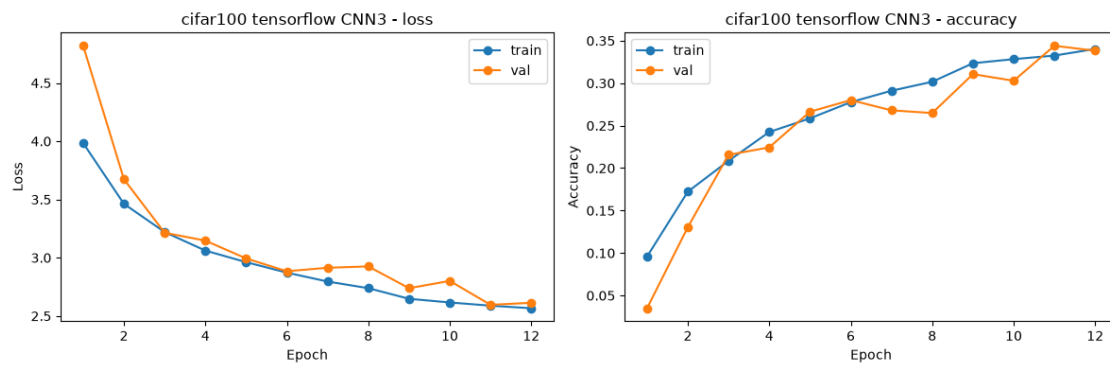
0.1304 - val_loss: 3.6740 - learning_rate: 0.0010
Epoch 3/12
352/352 - 43s - 121ms/step - accuracy: 0.2086 - loss: 3.2204 - val_accuracy: 0.2156 - val_loss: 3.2144 - learning_rate: 0.0010
Epoch 4/12
352/352 - 41s - 117ms/step - accuracy: 0.2425 - loss: 3.0615 - val_accuracy: 0.2244 - val_loss: 3.1469 - learning_rate: 0.0010
Epoch 5/12
352/352 - 44s - 125ms/step - accuracy: 0.2586 - loss: 2.9623 - val_accuracy: 0.2664 - val_loss: 2.9939 - learning_rate: 0.0010
Epoch 6/12
352/352 - 43s - 122ms/step - accuracy: 0.2776 - loss: 2.8723 - val_accuracy: 0.2800 - val_loss: 2.8842 - learning_rate: 0.0010
Epoch 7/12
352/352 - 40s - 114ms/step - accuracy: 0.2911 - loss: 2.7959 - val_accuracy: 0.2680 - val_loss: 2.9141 - learning_rate: 0.0010
Epoch 8/12

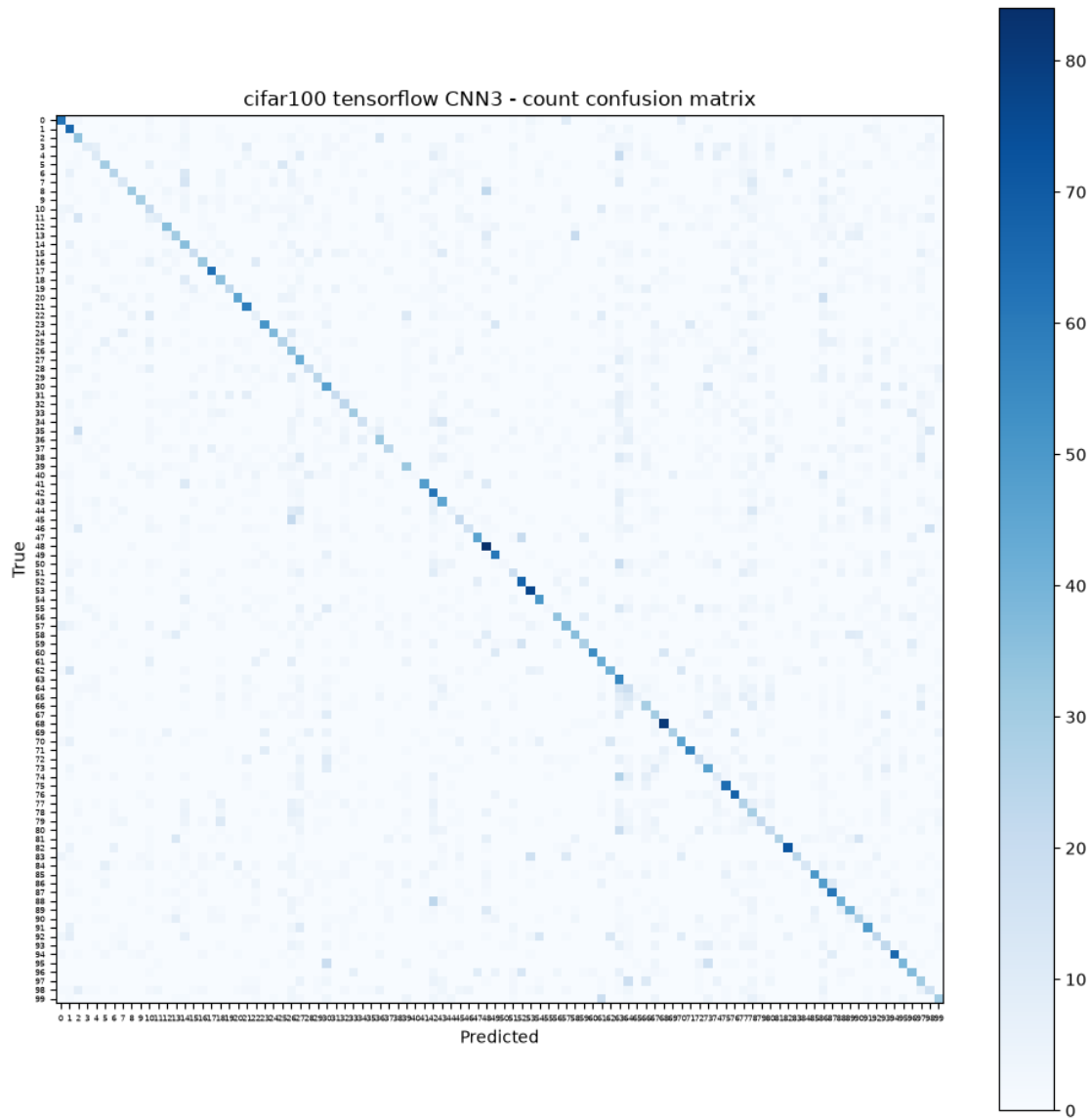
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
352/352 - 41s - 117ms/step - accuracy: 0.3016 - loss: 2.7388 - val_accuracy: 0.2648 - val_loss: 2.9255 - learning_rate: 0.0010
Epoch 9/12
352/352 - 42s - 119ms/step - accuracy: 0.3234 - loss: 2.6477 - val_accuracy: 0.3106 - val_loss: 2.7386 - learning_rate: 5.0000e-04
Epoch 10/12
352/352 - 46s - 132ms/step - accuracy: 0.3283 - loss: 2.6161 - val_accuracy: 0.3028 - val_loss: 2.8013 - learning_rate: 5.0000e-04
Epoch 11/12
352/352 - 48s - 136ms/step - accuracy: 0.3324 - loss: 2.5888 - val_accuracy: 0.3440 - val_loss: 2.5961 - learning_rate: 5.0000e-04
Epoch 12/12
352/352 - 41s - 116ms/step - accuracy: 0.3402 - loss: 2.5665 - val_accuracy: 0.3382 - val_loss: 2.6134 - learning_rate: 5.0000e-04

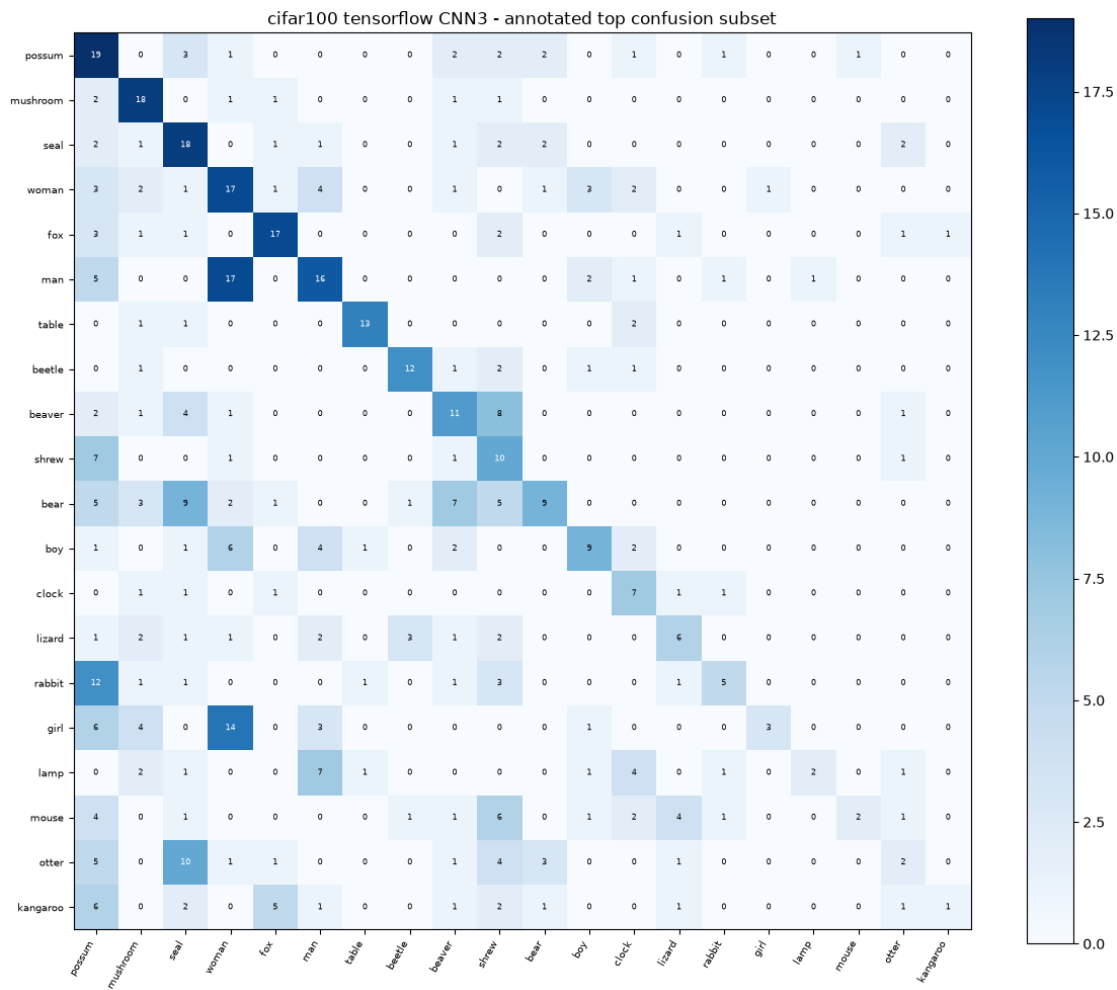
cifar100 | tensorflow | CNN3

	value
accuracy	0.3437
precision_macro	0.380055
recall_macro	0.3437
f1_macro	0.336275
precision_weighted	0.380055
recall_weighted	0.3437
f1_weighted	0.336275
log_loss	2.566965
mean_confidence	0.286036
ece	0.057664
top5_accuracy	0.6577

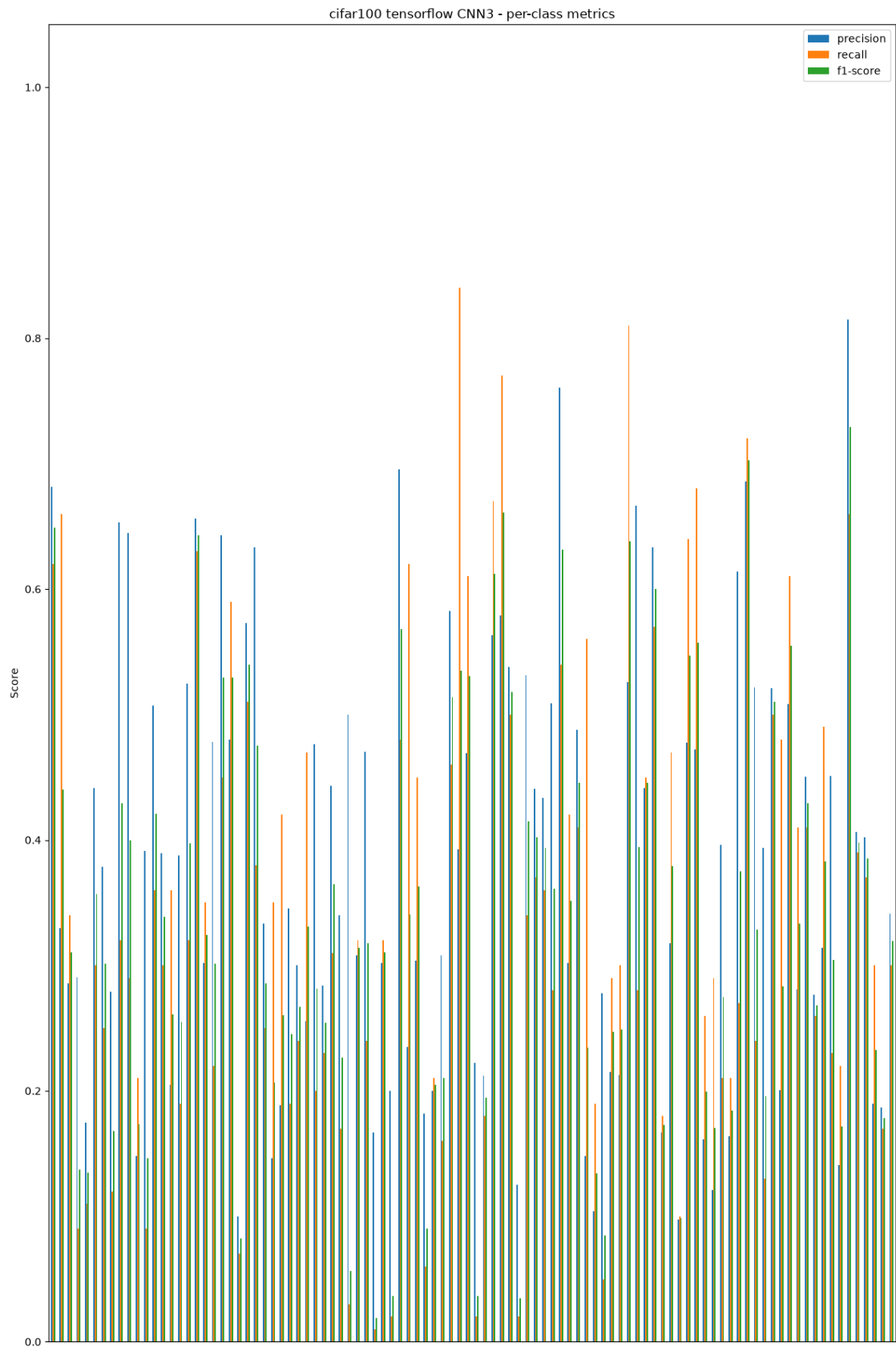
dataset cifar100
framework tensorflow
conv_layers 3





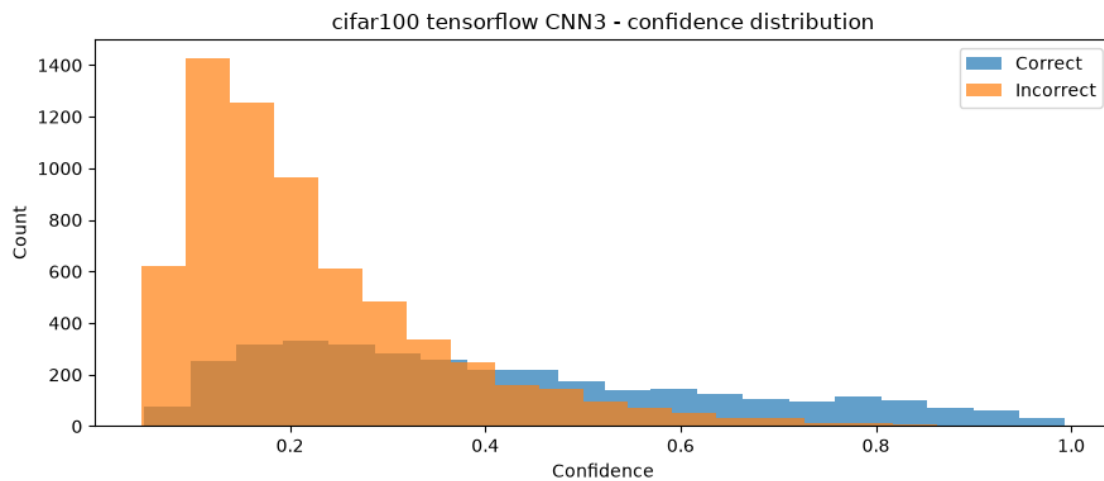


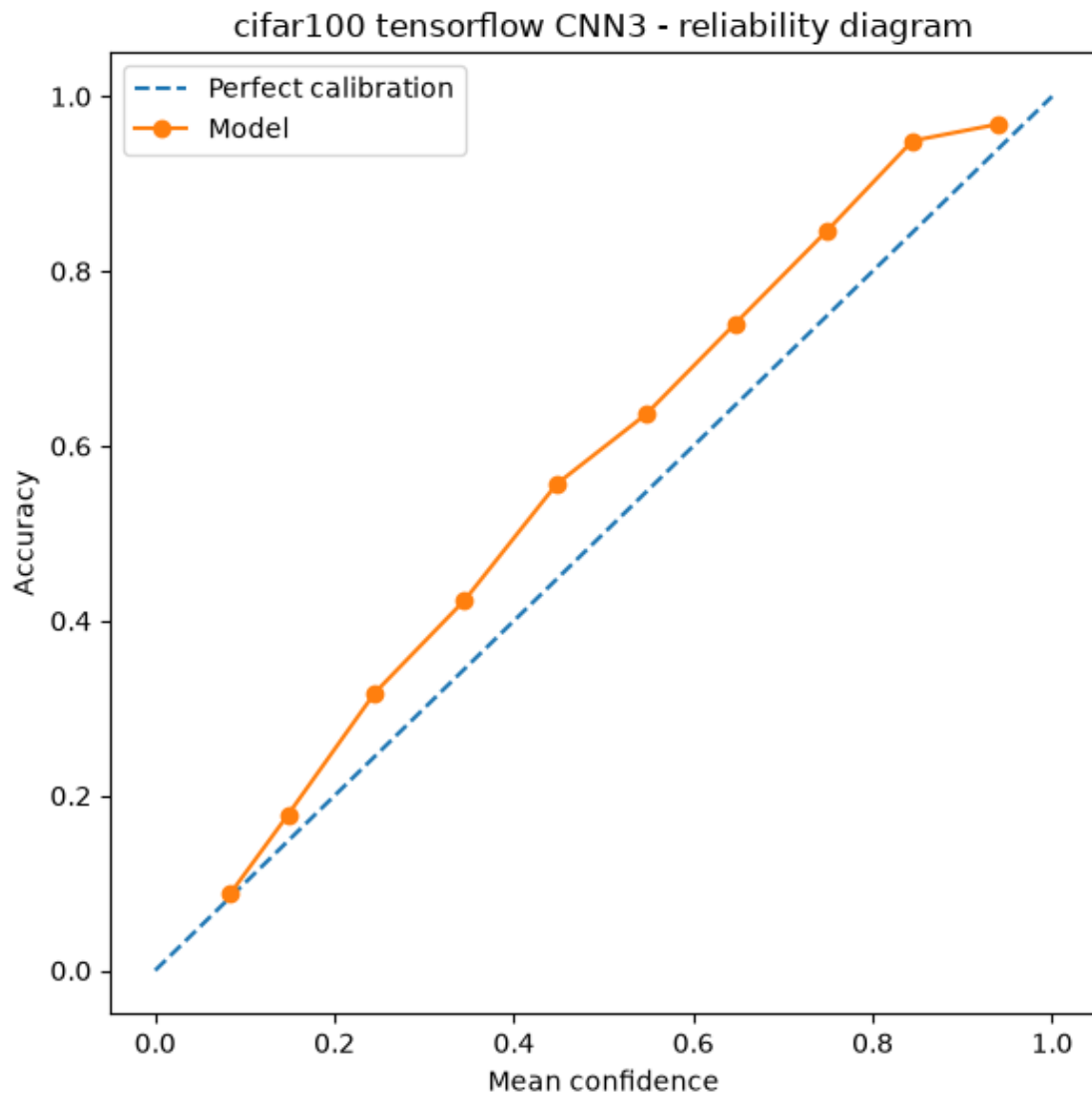
Per-class metrics:



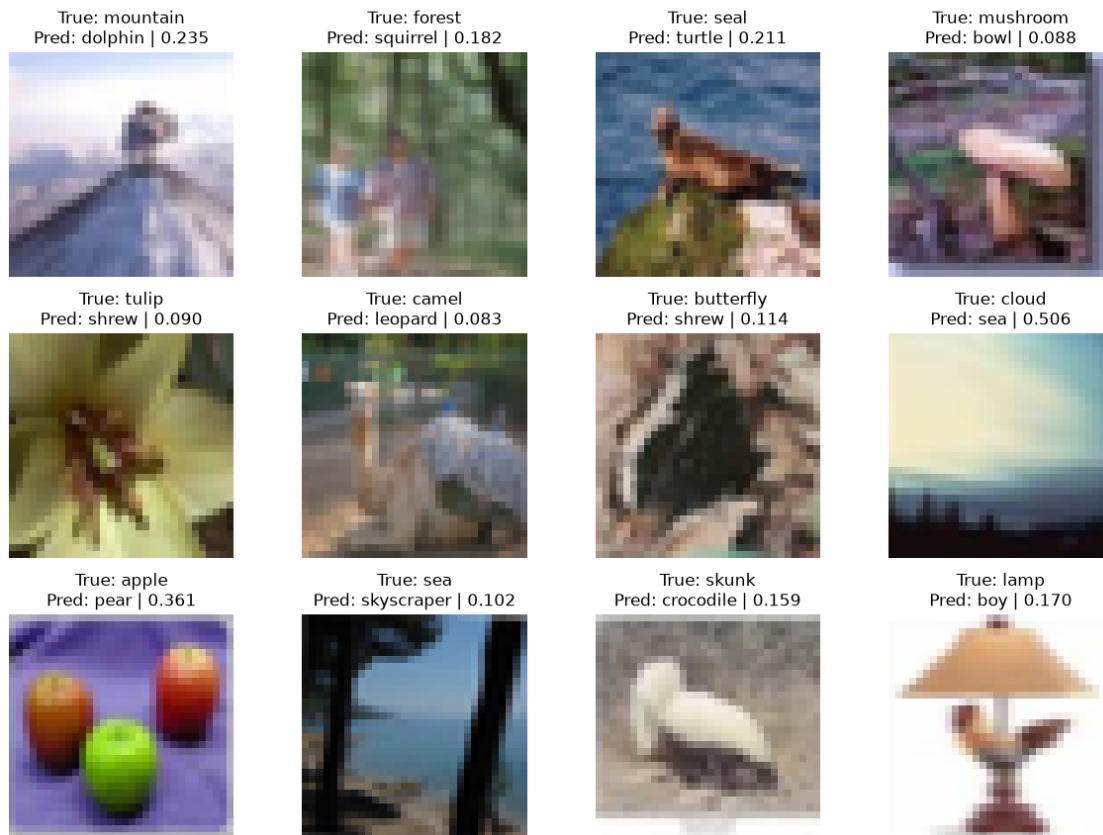
	precision	recall	f1-score	support
apple	0.681319	0.62	0.649215	100.0
aquarium_fish	0.330000	0.66	0.440000	100.0
baby	0.285714	0.34	0.310502	100.0
bear	0.290323	0.09	0.137405	100.0
beaver	0.174603	0.11	0.134969	100.0
...	...	...	...	...
whale	0.406250	0.39	0.397959	100.0
willow_tree	0.402174	0.37	0.385417	100.0
wolf	0.189873	0.30	0.232558	100.0
woman	0.186813	0.17	0.178010	100.0
worm	0.340909	0.30	0.319149	100.0

[100 rows x 4 columns]





cifar100 tensorflow CNN3 - error examples



Top confusion pairs:

	count	true_class	predicted_class
0	26	shrew	porcupine
1	23	tiger	leopard
2	22	bicycle	motorcycle
3	21	lobster	crab
4	20	maple_tree	oak_tree
5	20	bus	pickup_truck
6	20	beaver	porcupine
7	19	worm	plate
8	19	whale	dolphin
9	19	squirrel	porcupine
10	19	mouse	porcupine
11	19	girl	baby
12	18	wolf	possum
13	18	chair	telephone
14	17	whale	shark

TensorFlow CIFAR-100 CNN5 | params: 442,948

TensorBoard is not installed; continuing without TensorBoard logging.

Epoch 1/12
 352/352 - 71s - 202ms/step - accuracy: 0.1454 - loss: 3.6689 - val_accuracy: 0.0376 - val_loss: 4.7690 - learning_rate: 0.0010

Epoch 2/12
 352/352 - 67s - 190ms/step - accuracy: 0.2642 - loss: 2.9178 - val_accuracy: 0.2382 - val_loss: 3.1349 - learning_rate: 0.0010

Epoch 3/12
 352/352 - 64s - 183ms/step - accuracy: 0.3346 - loss: 2.5504 - val_accuracy: 0.2500 - val_loss: 3.0393 - learning_rate: 0.0010

Epoch 4/12
 352/352 - 65s - 183ms/step - accuracy: 0.3849 - loss: 2.3180 - val_accuracy: 0.3238 - val_loss: 2.6617 - learning_rate: 0.0010

Epoch 5/12
 352/352 - 65s - 183ms/step - accuracy: 0.4204 - loss: 2.1552 - val_accuracy: 0.3646 - val_loss: 2.4653 - learning_rate: 0.0010

Epoch 6/12
 352/352 - 64s - 182ms/step - accuracy: 0.4526 - loss: 2.0230 - val_accuracy: 0.3166 - val_loss: 2.6871 - learning_rate: 0.0010

Epoch 7/12
 352/352 - 66s - 186ms/step - accuracy: 0.4741 - loss: 1.9218 - val_accuracy: 0.3716 - val_loss: 2.4341 - learning_rate: 0.0010

Epoch 8/12
 352/352 - 65s - 185ms/step - accuracy: 0.4954 - loss: 1.8373 - val_accuracy: 0.3960 - val_loss: 2.3061 - learning_rate: 0.0010

Epoch 9/12
 352/352 - 66s - 187ms/step - accuracy: 0.5133 - loss: 1.7516 - val_accuracy: 0.4102 - val_loss: 2.2554 - learning_rate: 0.0010

Epoch 10/12
 352/352 - 65s - 183ms/step - accuracy: 0.5291 - loss: 1.6843 - val_accuracy: 0.3530 - val_loss: 2.7295 - learning_rate: 0.0010

Epoch 11/12

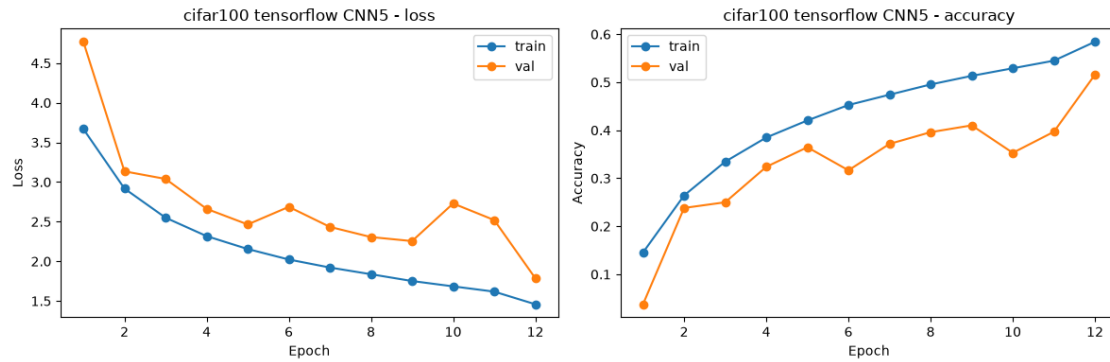
Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
 352/352 - 64s - 180ms/step - accuracy: 0.5449 - loss: 1.6177 - val_accuracy: 0.3968 - val_loss: 2.5205 - learning_rate: 0.0010

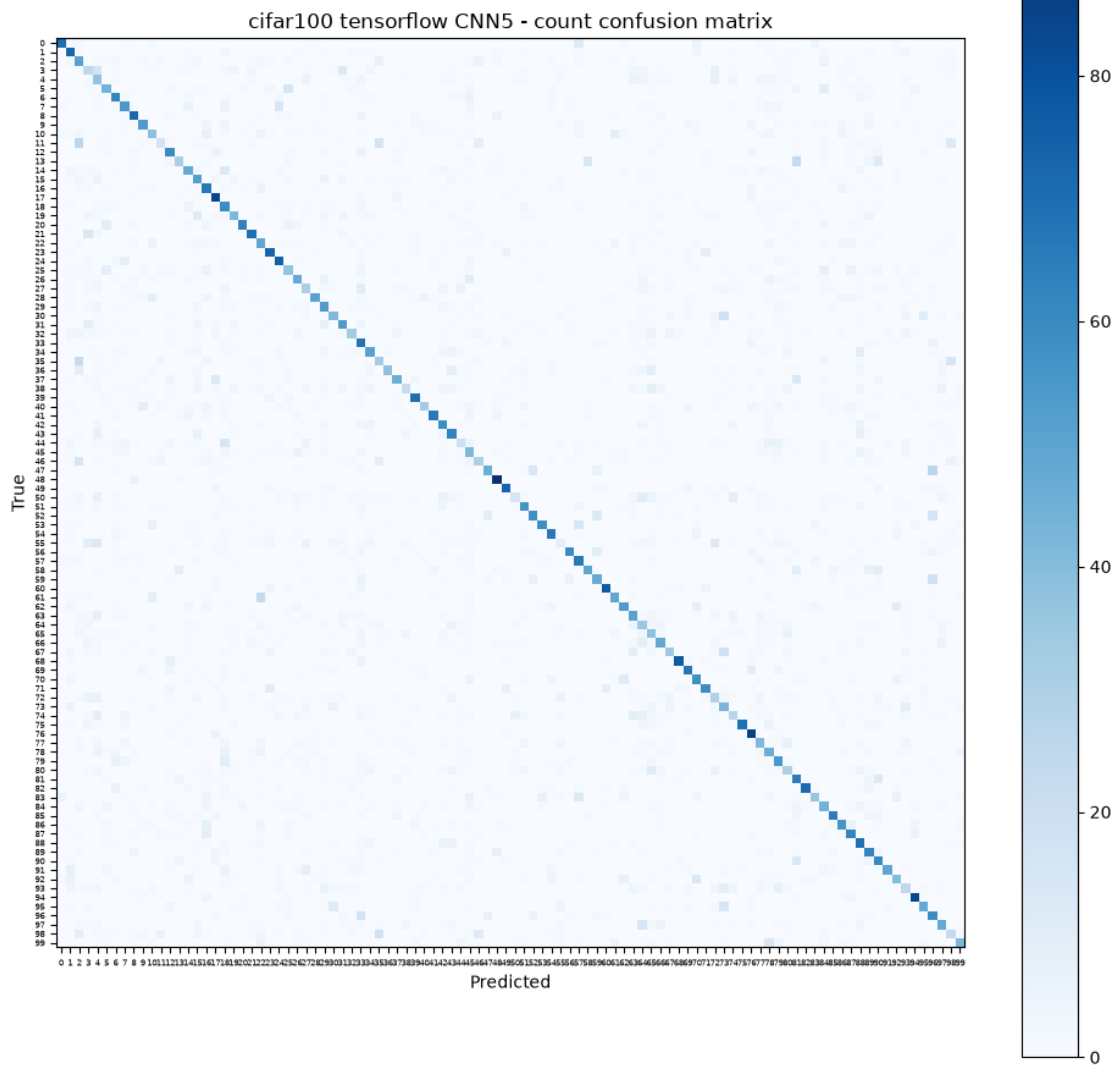
Epoch 12/12
 352/352 - 64s - 182ms/step - accuracy: 0.5843 - loss: 1.4587 - val_accuracy: 0.5158 - val_loss: 1.7871 - learning_rate: 5.0000e-04

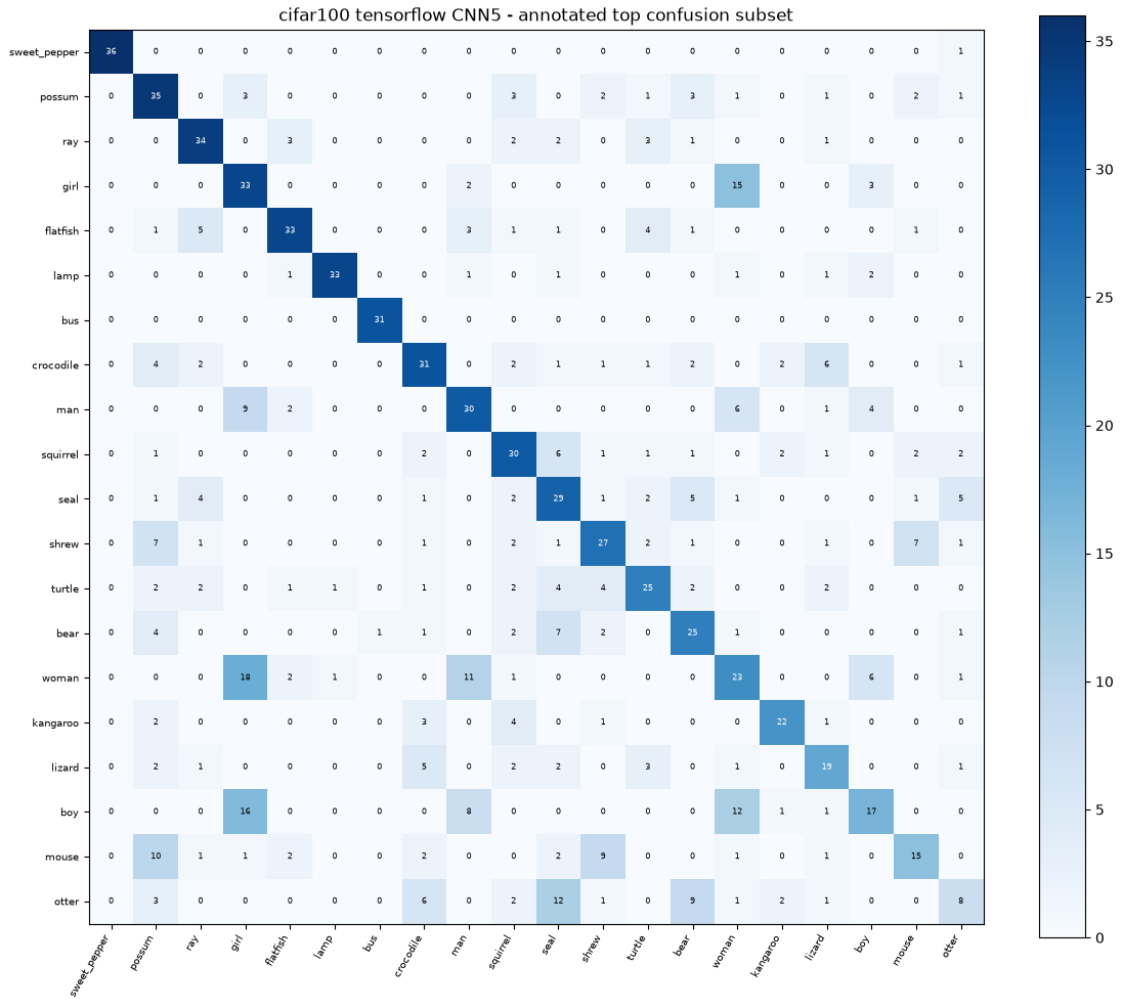
cifar100 | tensorflow | CNN5

	value
accuracy	0.5134
precision_macro	0.536683
recall_macro	0.5134
f1_macro	0.512011
precision_weighted	0.536683

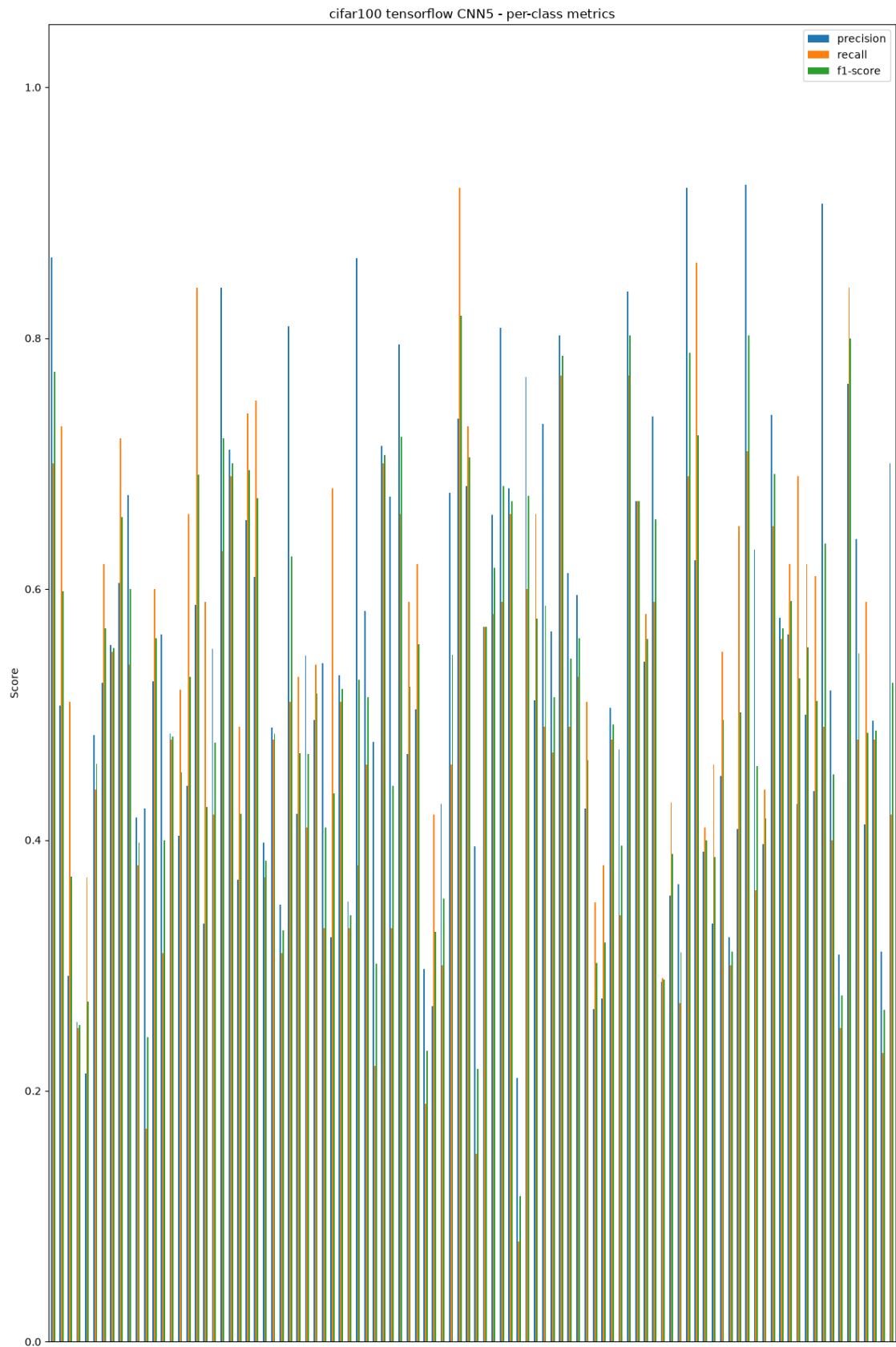
recall_weighted 0.5134
f1_weighted 0.512011
log_loss 1.79677
mean_confidence 0.52327
ece 0.01154
top5_accuracy 0.8112
dataset cifar100
framework tensorflow
conv_layers 5





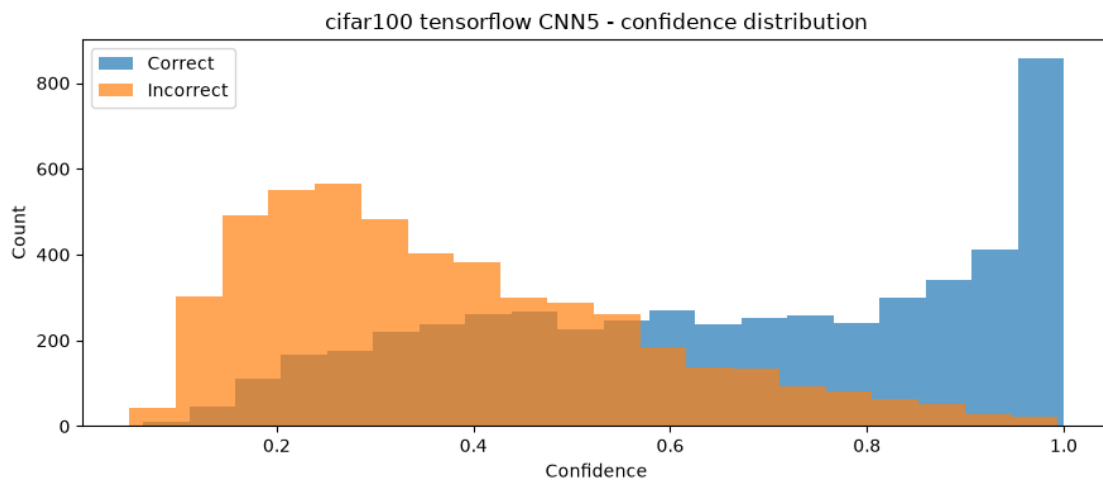


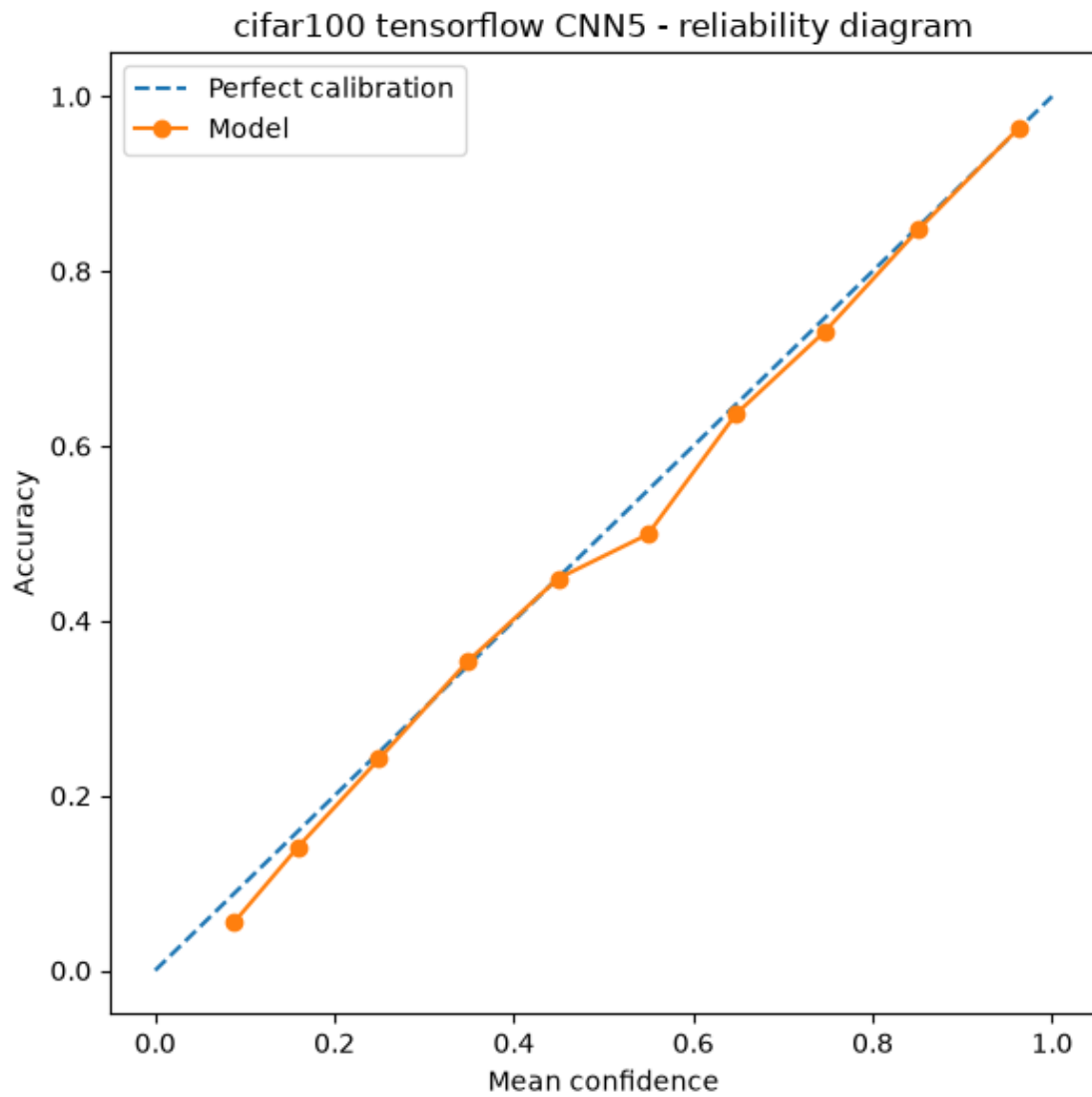
Per-class metrics:



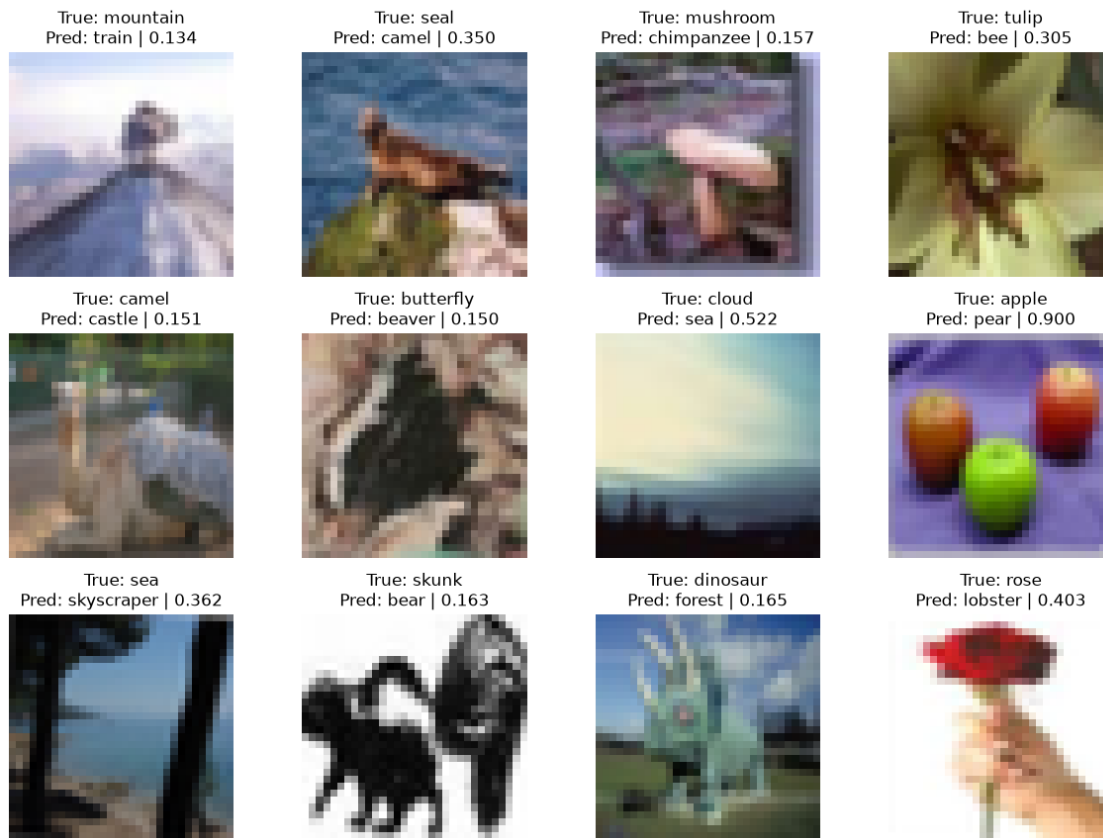
	precision	recall	f1-score	support
apple	0.864198	0.70	0.773481	100.0
aquarium_fish	0.506944	0.73	0.598361	100.0
baby	0.291429	0.51	0.370909	100.0
bear	0.255102	0.25	0.252525	100.0
beaver	0.213873	0.37	0.271062	100.0
...	...	...	...	...
whale	0.640000	0.48	0.548571	100.0
willow_tree	0.412587	0.59	0.485597	100.0
wolf	0.494845	0.48	0.487310	100.0
woman	0.310811	0.23	0.264368	100.0
worm	0.700000	0.42	0.525000	100.0

[100 rows x 4 columns]





cifar100 tensorflow CNN5 - error examples



Top confusion pairs:

	count	true_class	predicted_class
0	26	maple_tree	willow_tree
1	26	boy	baby
2	23	plate	clock
3	23	bus	streetcar
4	22	girl	baby
5	19	ray	shark
6	18	woman	girl
7	18	pine_tree	willow_tree
8	18	dolphin	shark
9	17	willow_tree	forest
10	17	bear	beaver
11	16	oak_tree	willow_tree
12	16	boy	girl
13	15	wolf	possum
14	15	whale	shark

8.6 8.3 TensorFlow - Diabetes Conv1D CNN3 and CNN5

The TensorFlow experiment uses Conv1D because the input has been transformed into a one-dimensional sequence of processed features.

The input tensor has the form:

```
(batch, feature_count, 1)
```

The convolution operates over local windows of the feature sequence. The report must be explicit that this is a modeling adaptation: the feature order is an imposed sequence and does not carry the same spatial semantics as image pixels.

The clinical target has three classes, so the final classifier produces three logits. Training uses class weights computed from the training partition to reduce the tendency to optimize mainly for the largest target class.

The saved preprocessing object is as important as the model itself for later inference. A future web demo must reproduce the exact same imputation, encoding, scaling, and feature ordering before calling the trained CNN.

```
[23]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 40.
      import joblib

      for depth in (3,5):
          path = MODELS_DIR / f"diabetes_tensorflow_cnn{depth}.keras"
          model = build_tf_cnn((X_diab_tr.shape[1],1), 3, depth, "tabular")
          print(f"\nTensorFlow Diabetes CNN{depth} | params: {model.count_params():
            ↪,}")
          history = train_tf(
              model,
              X_diab_tr, y_diab_tr,
              X_diab_val, y_diab_val,
              diabetes_data["class_weight"], path,
              str(OUTPUTS_DIR / "tensorboard" / f"diabetes_tf_cnn{depth}"),
          )
          _, probs = predict_tf(model, X_diab_test)
          metrics, _ = evaluate_and_visualize("diabetes", "tensorflow", depth,
            ↪y_diab_test, probs, DIABETES_NAMES, None, history)
          metrics["parameters"] = int(model.count_params())
          metrics["training_seconds"] = float(history["training_seconds"])
          TF_RESULTS[("diabetes", depth)] = metrics

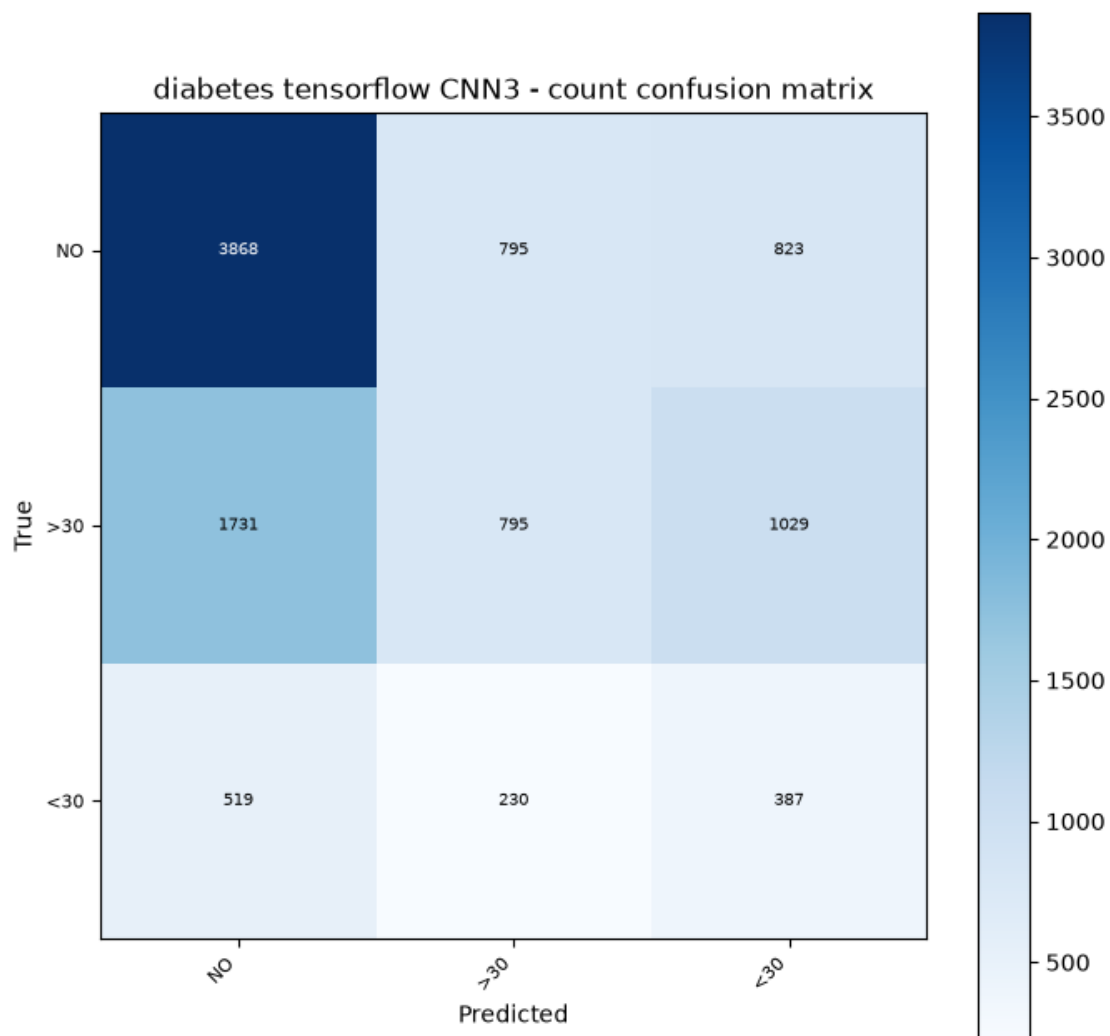
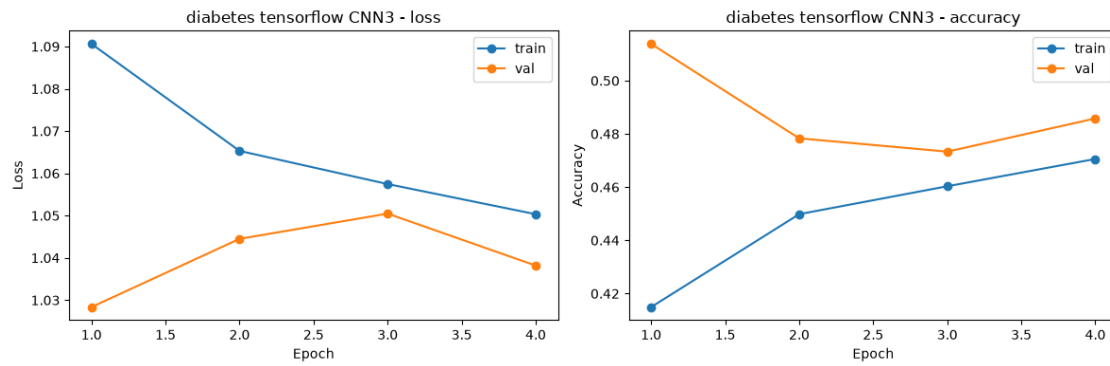
      joblib.dump(diabetes_data["preprocessor"], MODELS_DIR / "diabetes_preprocessor.
        ↪joblib")
      with open(MODELS_DIR / "diabetes_label_names.json", "w", encoding="utf-8") as f:
          json.dump(DIABETES_NAMES, f, ensure_ascii=False, indent=2)
```

```
# Re-save the final metric record after runtime and parameter fields are  
↪attached.  
save_experiment("diabetes", "tensorflow", depth, y_diab_test, probs, metrics)
```

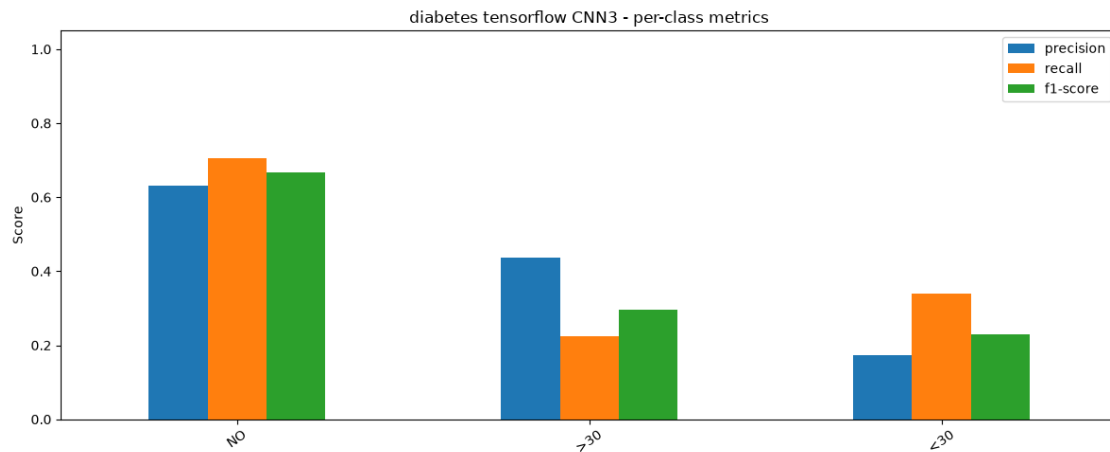
```
TensorFlow Diabetes CNN3 | params: 32,099  
TensorBoard is not installed; continuing without TensorBoard logging.  
Epoch 1/12  
637/637 - 10s - 16ms/step - accuracy: 0.4149 - loss: 1.0906 - val_accuracy:  
0.5138 - val_loss: 1.0284 - learning_rate: 0.0010  
Epoch 2/12  
637/637 - 8s - 13ms/step - accuracy: 0.4498 - loss: 1.0653 - val_accuracy:  
0.4783 - val_loss: 1.0445 - learning_rate: 0.0010  
Epoch 3/12  
  
Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.  
637/637 - 8s - 13ms/step - accuracy: 0.4603 - loss: 1.0575 - val_accuracy:  
0.4733 - val_loss: 1.0505 - learning_rate: 0.0010  
Epoch 4/12  
637/637 - 9s - 14ms/step - accuracy: 0.4706 - loss: 1.0503 - val_accuracy:  
0.4858 - val_loss: 1.0382 - learning_rate: 5.0000e-04
```

```
diabetes | tensorflow | CNN3
```

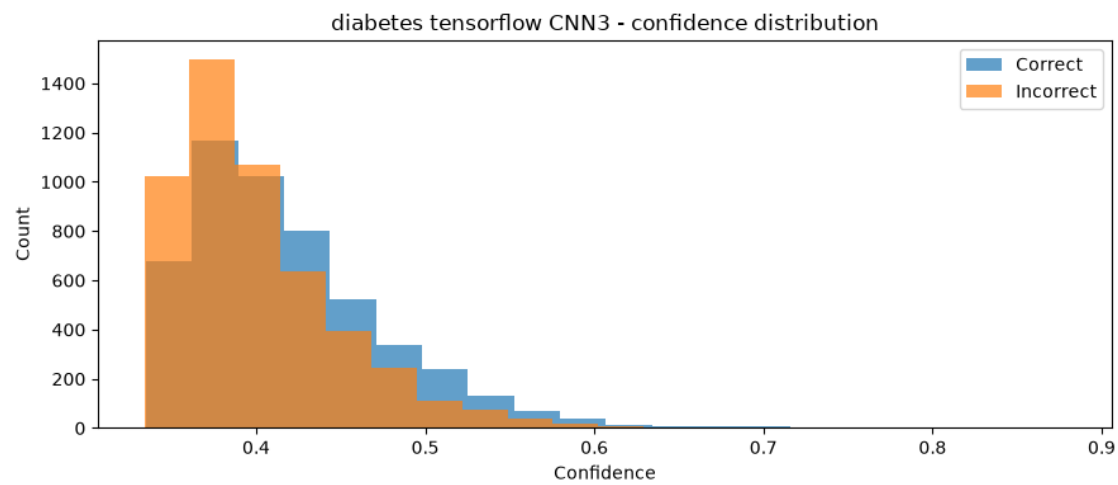
	value
accuracy	0.496217
precision_macro	0.413964
recall_macro	0.423122
f1_macro	0.397271
precision_weighted	0.512691
recall_weighted	0.496217
f1_weighted	0.488304
log_loss	1.034184
mean_confidence	0.409995
ece	0.08709
dataset	diabetes
framework	tensorflow
conv_layers	3

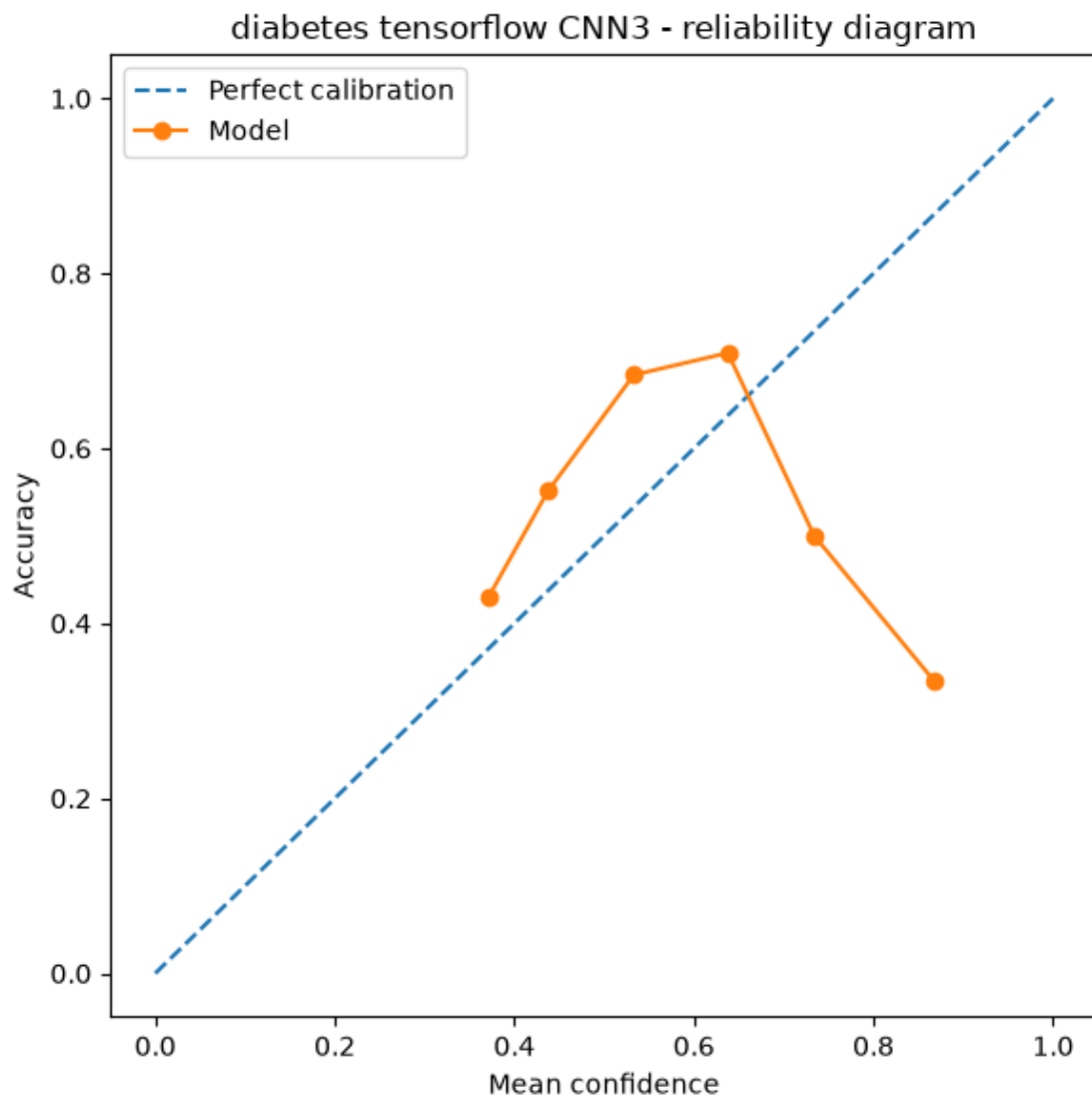


Per-class metrics:



	precision	recall	f1-score	support
NO	0.632233	0.705067	0.666667	5486.0
>30	0.436813	0.223629	0.295814	3555.0
<30	0.172845	0.340669	0.229333	1136.0





Top confusion pairs:

	count	true_class	predicted_class
0	1731	>30	NO
1	1029	>30	<30
2	823	NO	<30
3	795	NO	>30
4	519	<30	NO
5	230	<30	>30

TensorFlow Diabetes CNN5 | params: 143,939

TensorBoard is not installed; continuing without TensorBoard logging.

Epoch 1/12

637/637 - 20s - 31ms/step - accuracy: 0.4346 - loss: 1.0874 - val_accuracy: 0.5113 - val_loss: 1.0200 - learning_rate: 0.0010

Epoch 2/12

637/637 - 17s - 26ms/step - accuracy: 0.4658 - loss: 1.0566 - val_accuracy: 0.4937 - val_loss: 1.0247 - learning_rate: 0.0010

Epoch 3/12

Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

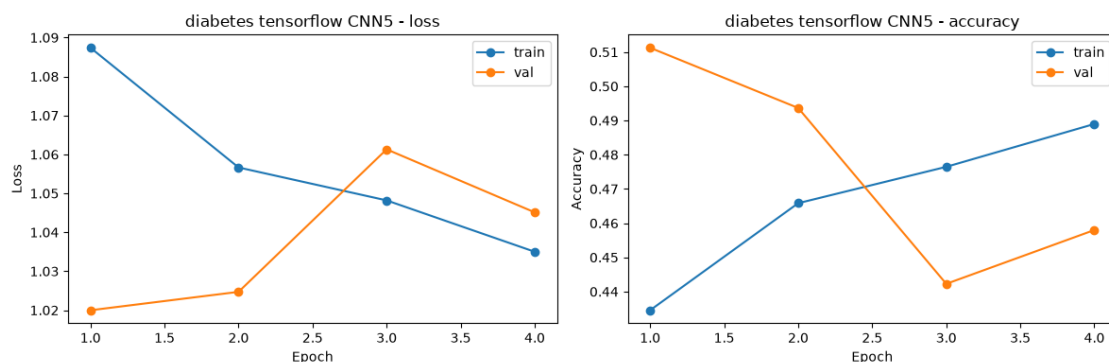
637/637 - 16s - 26ms/step - accuracy: 0.4765 - loss: 1.0482 - val_accuracy: 0.4423 - val_loss: 1.0613 - learning_rate: 0.0010

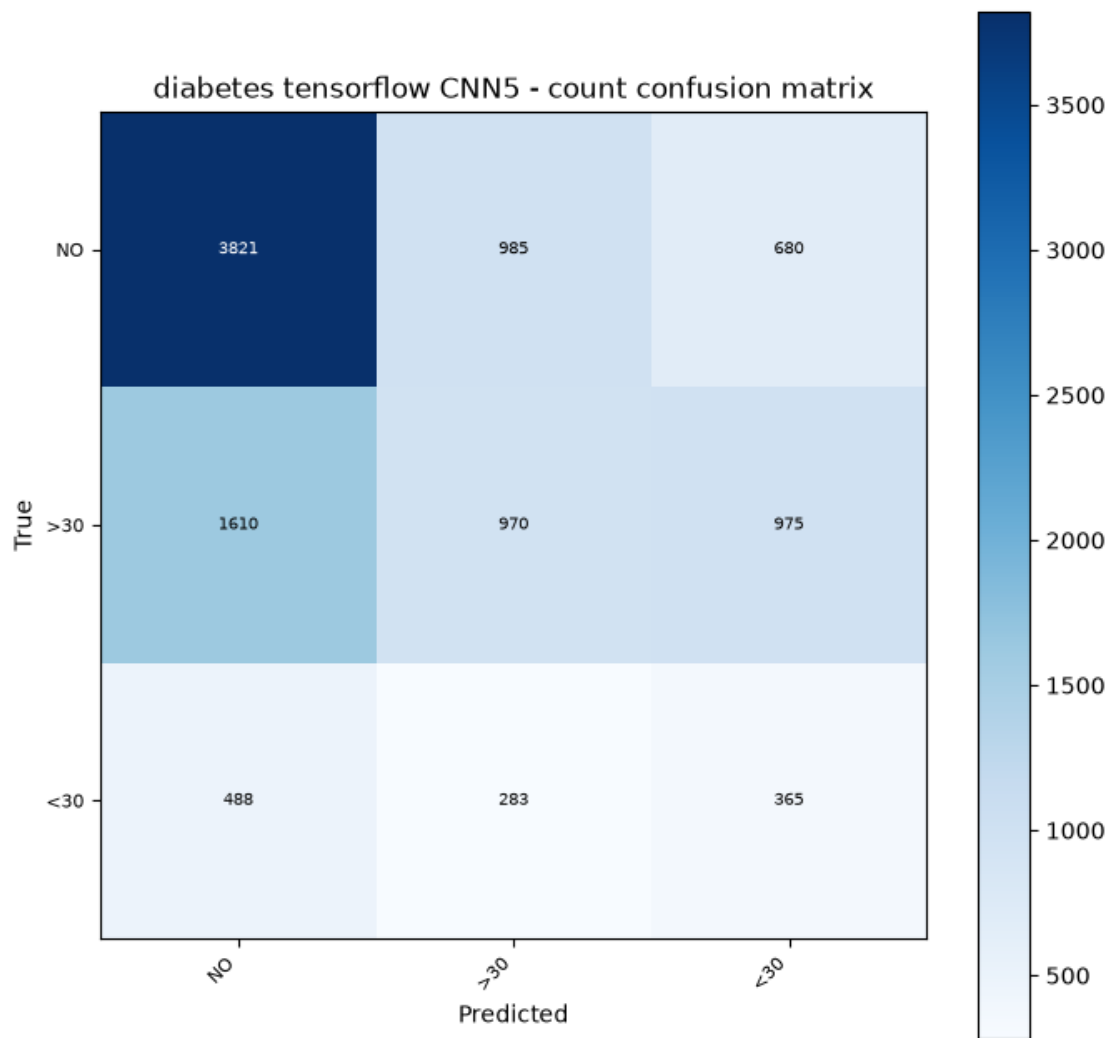
Epoch 4/12

637/637 - 16s - 26ms/step - accuracy: 0.4890 - loss: 1.0350 - val_accuracy: 0.4580 - val_loss: 1.0451 - learning_rate: 5.0000e-04

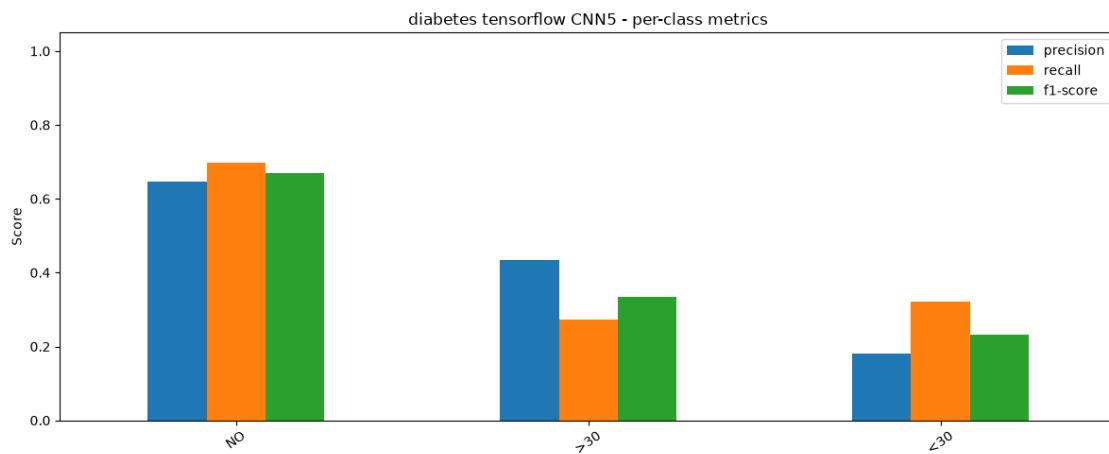
diabetes | tensorflow | CNN5

	value
accuracy	0.506633
precision_macro	0.419888
recall_macro	0.430219
f1_macro	0.412083
precision_weighted	0.51956
recall_weighted	0.506633
f1_weighted	0.504001
log_loss	1.021818
mean_confidence	0.425098
ece	0.081683
dataset	diabetes
framework	tensorflow
conv_layers	5

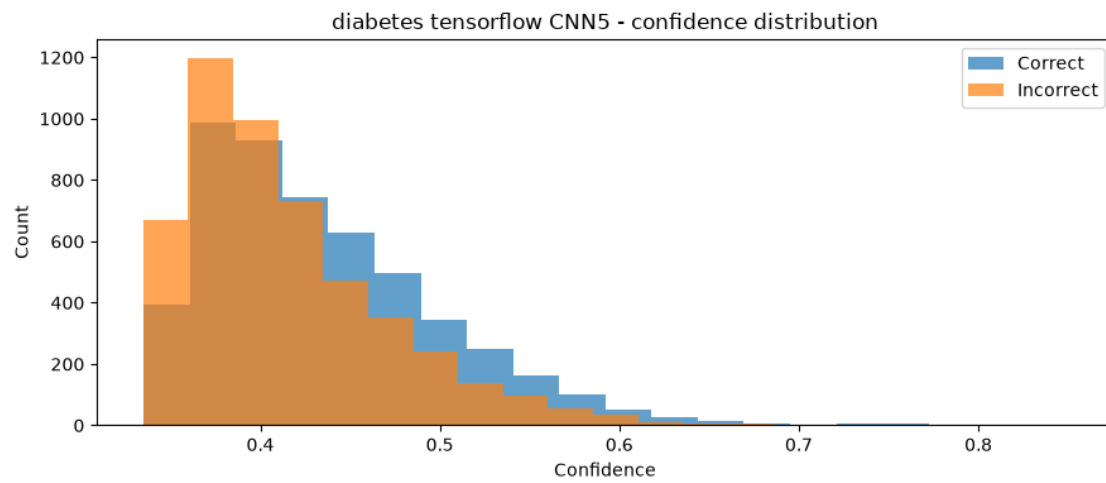


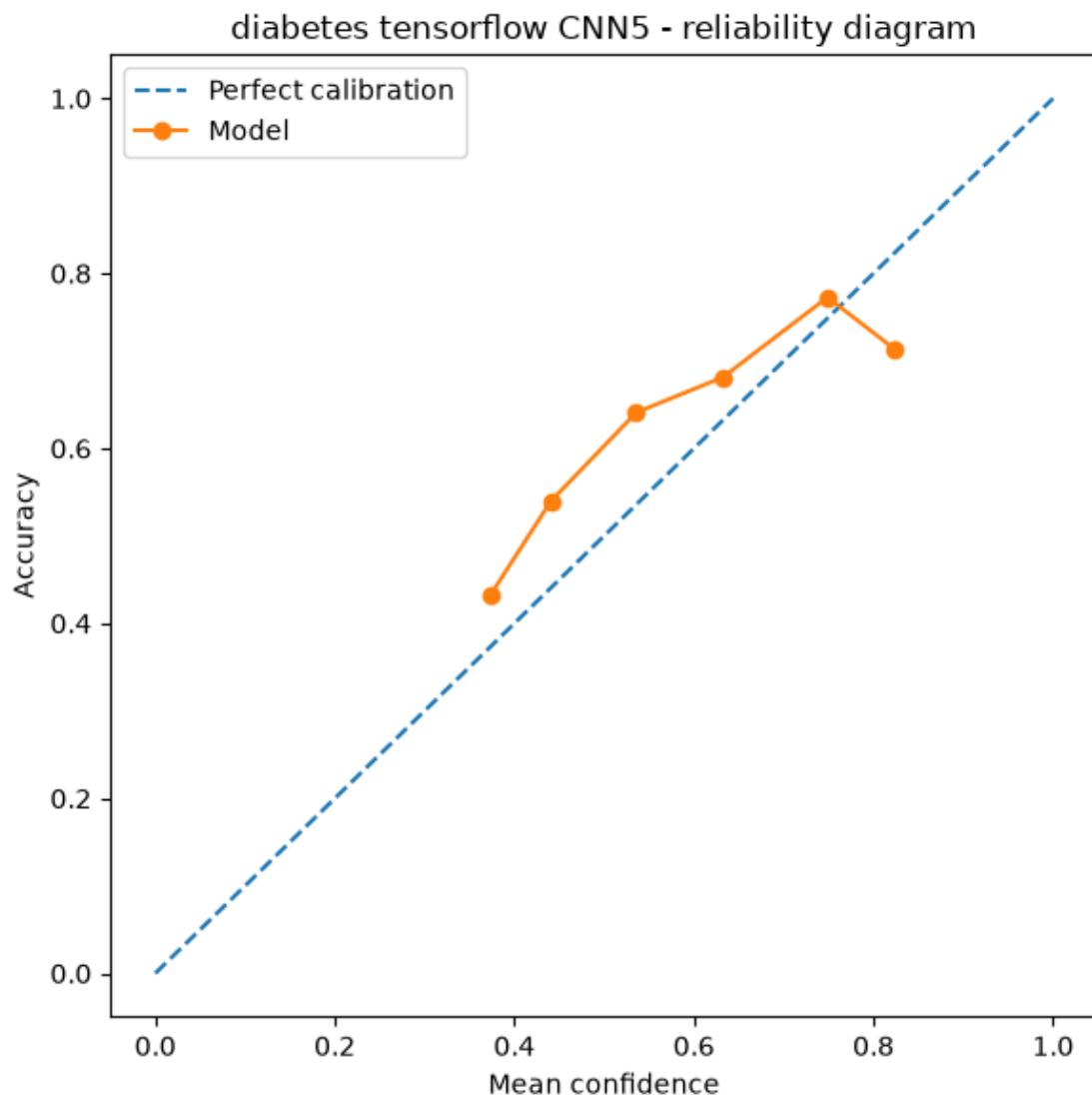


Per-class metrics:



	precision	recall	f1-score	support
NO	0.645548	0.696500	0.670057	5486.0
>30	0.433423	0.272855	0.334887	3555.0
<30	0.180693	0.321303	0.231305	1136.0





Top confusion pairs:

	count	true_class	predicted_class
0	1610	>30	NO
1	985	NO	>30
2	975	>30	<30
3	680	NO	<30
4	488	<30	NO
5	283	<30	>30

9 Part G - PyTorch implementation

9.1 9. PyTorch CNN architecture and abstraction level

PyTorch uses `nn.Module` to express the same conceptual CNN architecture. This corresponds directly to the second framework implementation described in Lecture 04.

For an image model:

```
nn.Conv2d
→ nn.BatchNorm2d
→ nn.ReLU
→ nn.MaxPool2d
→ ...
→ nn.AdaptiveAvgPool2d
→ nn.Dropout
→ nn.Linear
```

For the Diabetes experiment, the same pattern uses the 1-D equivalents.

9.1.1 Tensor layout difference

TensorFlow/Keras normally receives image tensors as:

(batch, height, width, channels)

PyTorch convolution layers expect:

(batch, channels, height, width)

This is a representation difference, not a different learning problem. The notebook converts the NumPy arrays at the Dataset boundary so that the model itself receives the convention expected by PyTorch.

```
[24]: # Purpose: Define the PyTorch datasets, models, validation loop, inference_
      ↪ path, and training loop.
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", DEVICE)

try:
    from torchvision import transforms
    HAS_TORCHVISION = True
except Exception as exc:
    HAS_TORCHVISION = False
    print("torchvision warning:", exc)

class NumpyImageDataset(Dataset):
```

```

"""Wrap a NumPy image array in the Dataset interface expected by PyTorch.

Purpose:
    Convert NumPy HxW/HxWxC images into PyTorch channel-first tensors and
    optionally
    apply training-only augmentation.
"""
def __init__(self, X, y, train=False):
    """Initialize the image dataset wrapper and optional training
    ↪ augmentation.

    Purpose:
        Store image/label arrays and configure augmentation without
    ↪ modifying validation/test data.
    """

    # Store the arrays without changing the underlying labels.
    self.X = X
    self.y = y.astype(np.int64)
    self.augment = None
    if HAS_TORCHVISION and train:
        # Match the TensorFlow image policy with a light horizontal flip
    ↪ only.
        self.augment = transforms.Compose([
            transforms.RandomHorizontalFlip(0.5),
        ])

def __len__(self):
    """Return the number of image examples in this dataset."""
    return len(self.y)

def __getitem__(self, idx):
    """Return one image tensor and its integer target label.

    Purpose:
        Perform the NumPy-to-PyTorch channel conversion at the data
    ↪ boundary.
    """

    img = self.X[idx]
    if img.ndim == 2:
        # Add the single grayscale channel before transposing to CxHxW.
        img = img[...,None]
    tensor = torch.from_numpy(img.transpose(2,0,1)).float()
    if self.augment is not None:
        # Augmentation is applied only to the training dataset.
        tensor = self.augment(tensor)
    return tensor, torch.tensor(self.y[idx], dtype=torch.long)

```

```

class NumpyTabularDataset(Dataset):
    """Wrap the processed Diabetes Conv1D tensor and labels as a PyTorch
    Dataset."""
    def __init__(self, X, y):
        """Initialize a tabular dataset wrapper around processed Conv1D tensors.

        Purpose:
            Convert processed NumPy arrays into PyTorch tensors once for
            efficient batching.
        """
        # Convert the complete arrays once so __getitem__ remains lightweight.
        self.X = torch.from_numpy(X).float()
        self.y = torch.from_numpy(y.astype(np.int64)).long()

    def __len__(self):
        """Return the number of tabular training/evaluation examples."""
        return len(self.y)

    def __getitem__(self, idx):
        """Return one Conv1D-ready feature tensor and its class label."""
        return self.X[idx], self.y[idx]

class ImageCNN(nn.Module):
    """Build the A5 CNN3/CNN5 architecture for 2-D image datasets."""
    def __init__(self, in_channels, num_classes, depth=3):
        """Initialize the image CNN modules for the requested convolutional
        depth.

        Purpose:
            Build the shared CNN3/CNN5 structure used in the PyTorch image
            experiments.
        """
        super().__init__()
        # Keep the filter schedule identical to TensorFlow so depth is the main
        architectural variable.
        filters = [32,64,128,128,160] if depth == 5 else [32,64,128]
        pool_after = {0,1} if depth == 3 else {0,1,3}
        blocks=[]
        for i in range(depth):
            blocks += [
                nn.Conv2d(in_channels if i==0 else filters[i-1], filters[i], 3,
                padding=1, bias=False),
                nn.BatchNorm2d(filters[i]),
                nn.ReLU(inplace=True),

```

```

    ]
    if i in pool_after:
        # Pooling reduces spatial resolution and computation after
        ↪ selected blocks.
        blocks.append(nn.MaxPool2d(2))
    self.features = nn.Sequential(*blocks)
    # Adaptive pooling makes the classifier input equal to the final number
    ↪ of filters.
    self.pool = nn.AdaptiveAvgPool2d(1)
    self.dropout = nn.Dropout(0.30)
    self.classifier = nn.Linear(filters[-1], num_classes)

def forward(self, x):
    """Define the forward data flow from image tensor to class logits."""
    # Feature extractor learns local-to-global visual representations.
    x = self.features(x)
    # Global average pooling removes dependence on the exact remaining
    ↪ spatial dimensions.
    x = self.pool(x).flatten(1)
    # Dropout regularizes the classifier during training.
    return self.classifier(self.dropout(x))

class TabularCNN(nn.Module):
    """Build the assignment-driven Conv1D CNN3/CNN5 architecture for Diabetes.
    ↪ """
    def __init__(self, num_classes, depth=3):
        """Initialize the assignment-driven Conv1D CNN for the Diabetes
        ↪ experiment."""
        super().__init__()

        filters = [32, 64, 128, 128, 160] if depth == 5 else [32, 64, 128]
        pool_after = {0, 1} if depth == 3 else {0, 1, 3}

        blocks = []

        for i in range(depth):
            blocks += [
                nn.Conv1d(
                    1 if i == 0 else filters[i - 1],
                    filters[i],
                    kernel_size=3,
                    padding=1,
                    bias=False
                ),
                nn.BatchNorm1d(filters[i]),
                nn.ReLU(inplace=True),

```

```

    ]

    if i in pool_after:
        blocks.append(nn.MaxPool1d(2))

    self.features = nn.Sequential(*blocks)
    self.pool = nn.AdaptiveAvgPool1d(1)
    self.dropout = nn.Dropout(0.30)
    self.classifier = nn.Linear(filters[-1], num_classes)

def forward(self, x):
    """Forward pass for the tabular Conv1D model."""

    # Input from the preprocessing pipeline is:
    # (batch, sequence_length, channels) = (batch, 45, 1)
    #
    # PyTorch Conv1d expects:
    # (batch, channels, sequence_length) = (batch, 1, 45)
    x = x.transpose(1, 2)

    x = self.features(x)
    x = self.pool(x).flatten(1)

    return self.classifier(self.dropout(x))

def loader(ds, shuffle=False):
    """Create a reproducible DataLoader for training, validation, or testing.

    Purpose:
        Provide consistent batching and optional shuffling across every PyTorch
        ↪ experiment.

    Parameters:
        ds (Dataset): Dataset instance to batch.
        shuffle (bool): Whether training samples should be shuffled each epoch.

    Returns:
        DataLoader: Batched PyTorch data iterator.
    """
    # Pin memory only when CUDA is available because it can speed
    ↪ host-to-device transfers.
    return DataLoader(ds, batch_size=BATCH_SIZE, shuffle=shuffle,
    ↪ num_workers=0, pin_memory=torch.cuda.is_available())

def val_loss_acc(model, dl, criterion):

```

```

"""Evaluate validation loss and accuracy without updating model parameters.

Purpose:
    Provide the validation signal used by the scheduler and
    ↪ checkpoint-selection logic.

Parameters:
    model (nn.Module): Current PyTorch model.
    dl (DataLoader): Validation data loader.
    criterion: Loss function.

Returns:
    tuple: Mean validation loss and validation accuracy.
    """
model.eval()
total_loss=0.0; total=0; correct=0
with torch.no_grad():
    for xb,yb in dl:
        xb,yb=xb.to(DEVICE),yb.to(DEVICE)
        logits=model(xb); loss=criterion(logits,yb)
        # Accumulate sample-weighted loss so batches of different sizes are
    ↪ handled correctly.
        total_loss += loss.item()*yb.size(0)
        total += yb.size(0)
        correct += (logits.argmax(1)==yb).sum().item()
    return total_loss/max(total,1), correct/max(total,1)

def predict_torch(model, dl):
    """Run PyTorch inference and return labels, logits, and probabilities as
    ↪ NumPy arrays.

    Purpose:
        Convert framework-specific model output into the common format used by
    ↪ the evaluation code.

    Parameters:
        model (nn.Module): Trained model.
        dl (DataLoader): Test data loader.

    Returns:
        tuple: y_true, logits, and softmax probabilities as NumPy arrays.
    """
model.eval(); logits_all=[]; y_all=[]
with torch.no_grad():
    for xb,yb in dl:

```

```

        # Disable gradients because test inference does not update model_
        ↪parameters.
        logits_all.append(model(xb.to(DEVICE)).cpu().numpy())
        y_all.append(yb.numpy())
        logits=np.concatenate(logits_all); y_true=np.concatenate(y_all)
        # Apply numerically stable softmax after moving logits back to NumPy.
        p=np.exp(logits-logits.max(axis=1,keepdims=True)); p/=p.
        ↪sum(axis=1,keepdims=True)
        return y_true, logits, p

```

```

def train_torch(model, train_dl, val_dl, class_weight, path):
    """Train a PyTorch CNN with Adam, scheduling, gradient clipping, and_
    ↪checkpointing.

```

Purpose:

Make the learning loop explicit, matching the training-cycle_
 ↪*explanation in Lecture 04.*

Parameters:

model (nn.Module): CNN3 or CNN5 model.
train_dl (DataLoader): Training data.
val_dl (DataLoader): Validation data.
class_weight (dict): Training-derived class weights.
path (Path): Destination for the best `.pth` checkpoint.

Returns:

dict: Training/validation history, elapsed time, and best epoch.

```

    """
    # Convert training-derived class weights into a device-resident tensor for_
    ↪CrossEntropyLoss.
    weights=torch.tensor([class_weight.get(i,1.0) for i in_
    ↪range(len(class_weight))], dtype=torch.float32, device=DEVICE)
    criterion=nn.CrossEntropyLoss(weight=weights)
    optimizer=torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
    scheduler=torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min",_
    ↪factor=0.5, patience=2, min_lr=1e-6)
    history={"loss": [], "val_loss": [], "accuracy": [], "val_accuracy": []}
    best=float("inf"); bad=0; best_epoch=1; start=time.perf_counter()

    for epoch in range(1,EPOCHS+1):
        model.train(); running=0.0; total=0; correct=0
        for xb,yb in train_dl:
            xb,yb=xb.to(DEVICE),yb.to(DEVICE)
            # Clear old gradients before calculating the next batch gradient.
            optimizer.zero_grad(set_to_none=True)

```

```

    # Forward pass produces class logits.
    logits=model(xb)
    # Compute the supervised training objective.
    loss=criterion(logits,yb)
    # Autograd calculates gradients for every learnable parameter.
    loss.backward()
    # Clip very large gradients to improve training stability.
    torch.nn.utils.clip_grad_norm_(model.parameters(),5.0)
    # Apply the optimizer update using the new gradients.
    optimizer.step()
    running += loss.item()*yb.size(0); total += yb.size(0); correct +=
↪(logits.argmax(1)==yb).sum().item()

    tr_loss=running/max(total,1); tr_acc=correct/max(total,1)
    vl_loss,vl_acc=val_loss_acc(model,val_dl,criterion)
    # Scheduler observes validation loss rather than training loss because
↪generalization is the target.
    scheduler.step(vl_loss)
    for k,v in
↪[("loss",tr_loss),("accuracy",tr_acc),("val_loss",vl_loss),("val_accuracy",vl_acc)]:
    ↪ history[k].append(float(v))
        print(f"Epoch {epoch:02d} | train loss {tr_loss:.4f} acc {tr_acc:.4f} |
↪val loss {vl_loss:.4f} acc {vl_acc:.4f}")

    if vl_loss < best-1e-5:
        # Save the best validation checkpoint rather than blindly keeping
↪the last epoch.
        best=vl_loss; bad=0; best_epoch=epoch
        torch.save({"model_state_dict":model.state_dict(),"best_val_loss":
↪best,"best_epoch":best_epoch}, path)
    else:
        bad += 1
        if bad >= 3:
            print("Early stopping.")
            break

    # Restore the best checkpoint before any test-set evaluation.
    ckpt=torch.load(path, map_location=DEVICE)
    model.load_state_dict(ckpt["model_state_dict"])
    history["training_seconds"]=float(time.perf_counter()-start)
    history["best_epoch"]=int(ckpt.get("best_epoch",best_epoch))
    return history

```

Device: cpu

9.1.2 9.0.1 PyTorch model inspection before training

The PyTorch model definition should be read in the same order as the TensorFlow model:

Input → convolutional blocks → pooling → global average pooling → dropout → classifier

Although the syntax differs, the tensor flow should be recognizable. This is the central framework-comparison lesson of Lecture 04.

9.2 9.4 What PyTorch makes visible

The most important contrast with Keras is the training loop.

PyTorch exposes the central cycle explicitly:

```
optimizer.zero_grad()
    ↓
model(x)
    ↓
criterion(outputs, labels)
    ↓
loss.backward()
    ↓
optimizer.step()
```

This corresponds closely to the learning-cycle diagram from Lecture 04:

Zero gradients → Forward → Loss → Backward → Update

The model definition is also explicit through `nn.Module` and `forward()`.

This does not mean PyTorch exposes every low-level operation. Autograd still calculates derivatives automatically, and framework layers still implement convolution, pooling, and batch normalization. The important point is that the programmer sees more of the model data flow and optimization loop than in a one-line `model.fit()` call.

9.3 9.1 PyTorch - Fashion-MNIST CNN3 and CNN5

The PyTorch Fashion-MNIST models use the same filter schedule, pooling positions, dropout rate, kernel size, and optimizer policy as the TensorFlow models.

The training data are wrapped in a `Dataset` and consumed through a `DataLoader`. Augmentation is applied only when the dataset is constructed for training.

During training, the loop records:

- training loss;
- training accuracy;
- validation loss;
- validation accuracy;
- elapsed training time;
- best validation checkpoint.

After the loop, the best saved weights are restored before the test set is evaluated. This prevents the final report from accidentally using the last epoch when an earlier epoch generalized better.

```
[25]: # Purpose: Inspect the PyTorch CNN3/CNN5 module structure before training.
# Printing named modules makes the architectural abstraction visible in the
↳notebook.

for depth in (3, 5):
    demo_model = ImageCNN(1, 10, depth).to(DEVICE)
    trainable = sum(p.numel() for p in demo_model.parameters() if p.
↳requires_grad)
    print(f"\nFashion-MNIST PyTorch CNN{depth}")
    print(demo_model)
    print(f"Trainable parameters: {trainable:,}")
```

Fashion-MNIST PyTorch CNN3

```
ImageCNN(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    (10): ReLU(inplace=True)
  )
  (pool): AdaptiveAvgPool2d(output_size=1)
  (dropout): Dropout(p=0.3, inplace=False)
  (classifier): Linear(in_features=128, out_features=10, bias=True)
)
Trainable parameters: 94,186
```

Fashion-MNIST PyTorch CNN5

```
ImageCNN(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
```

```

        (2): ReLU(inplace=True)
        (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (6): ReLU(inplace=True)
        (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (10): ReLU(inplace=True)
        (11): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (12): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (13): ReLU(inplace=True)
        (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (15): Conv2d(128, 160, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (16): BatchNorm2d(160, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (17): ReLU(inplace=True)
    )
    (pool): AdaptiveAvgPool2d(output_size=1)
    (dropout): Dropout(p=0.3, inplace=False)
    (classifier): Linear(in_features=160, out_features=10, bias=True)
)
Trainable parameters: 426,858

```

```

[26]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 44.
      TORCH_RESULTS={}
      fashion_train_ds=NumpyImageDataset(X_fashion_tr,y_fashion_tr,True)
      fashion_val_ds=NumpyImageDataset(X_fashion_val,y_fashion_val,False)
      fashion_test_ds=NumpyImageDataset(X_fashion_test,y_fashion_test,False)
      fashion_train_dl=loader(fashion_train_ds,True);
      ↪fashion_val_dl=loader(fashion_val_ds);
      ↪fashion_test_dl=loader(fashion_test_ds)
      fashion_weights={i:1.0 for i in range(10)}

      for depth in (3,5):

```

```

    model=ImageCNN(1,10,depth).to(DEVICE); path=MODELS_DIR/
    ↪f"fashion_pytorch_cnn{depth}.pth"
    params=sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"\nPyTorch Fashion CNN{depth} | params: {params:,}")
    ↪
    ↪history=train_torch(model,fashion_train_dl,fashion_val_dl,fashion_weights,path)
    y_true,_,probs=predict_torch(model,fashion_test_dl)
    ↪
    ↪metrics,_=evaluate_and_visualize("fashion_mnist","pytorch",depth,y_true,probs,FASHION_NAMES
    metrics["parameters"]=int(params);↪
    ↪metrics["training_seconds"]=float(history["training_seconds"])
    TORCH_RESULTS[("fashion",depth)]=metrics

# Re-save the final metric record after runtime and parameter fields are↪
↪attached.
save_experiment("fashion_mnist", "pytorch", depth, y_true, probs, metrics)

```

```

PyTorch Fashion CNN3 | params: 94,186
Epoch 01 | train loss 0.7656 acc 0.7507 | val loss 0.4852 acc 0.8327
Epoch 02 | train loss 0.4714 acc 0.8333 | val loss 0.4334 acc 0.8485
Epoch 03 | train loss 0.4093 acc 0.8562 | val loss 0.3847 acc 0.8635
Epoch 04 | train loss 0.3797 acc 0.8656 | val loss 0.3677 acc 0.8678
Epoch 05 | train loss 0.3508 acc 0.8749 | val loss 0.4404 acc 0.8377
Epoch 06 | train loss 0.3389 acc 0.8798 | val loss 0.3272 acc 0.8787
Epoch 07 | train loss 0.3213 acc 0.8854 | val loss 0.3373 acc 0.8775
Epoch 08 | train loss 0.3135 acc 0.8898 | val loss 0.3089 acc 0.8873
Epoch 09 | train loss 0.3051 acc 0.8928 | val loss 0.3083 acc 0.8865
Epoch 10 | train loss 0.2945 acc 0.8961 | val loss 0.2979 acc 0.8915
Epoch 11 | train loss 0.2873 acc 0.8971 | val loss 0.3423 acc 0.8737
Epoch 12 | train loss 0.2819 acc 0.9000 | val loss 0.3039 acc 0.8908

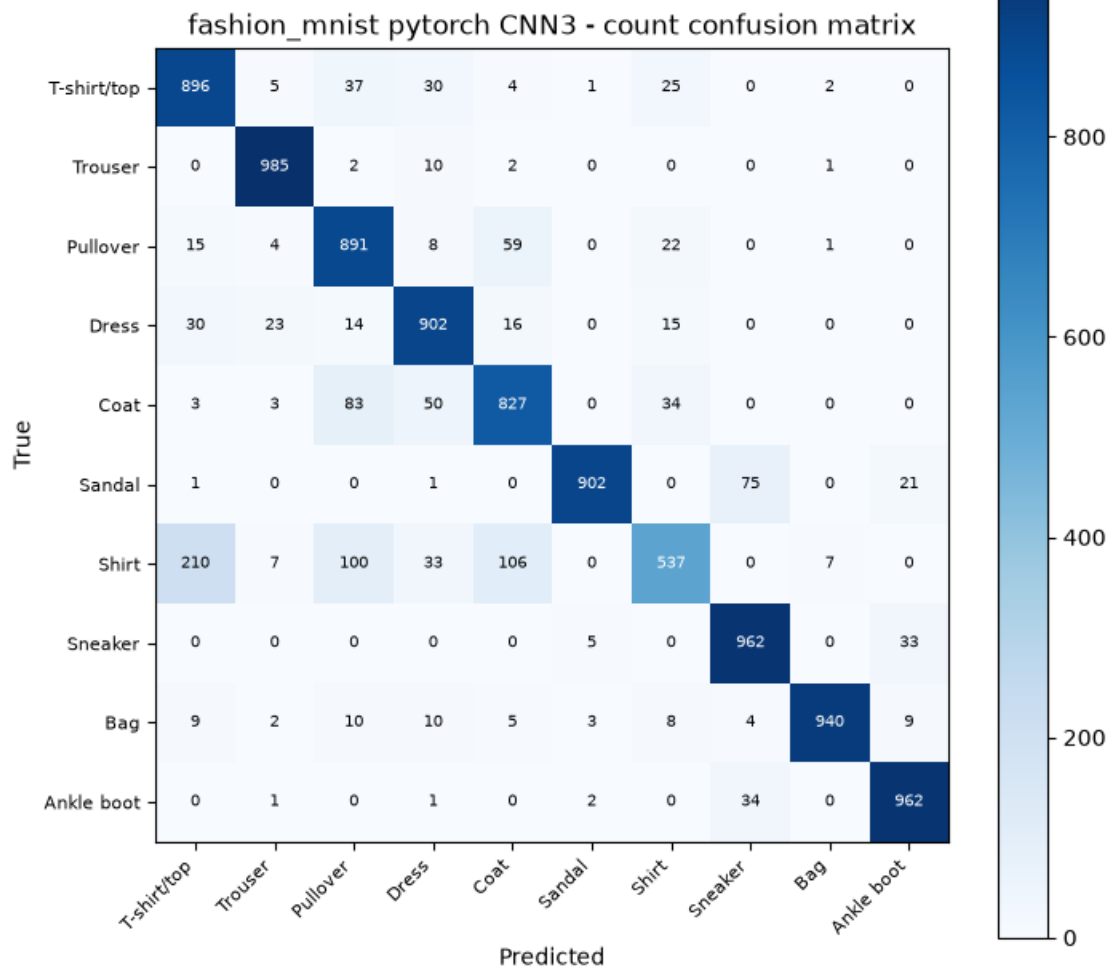
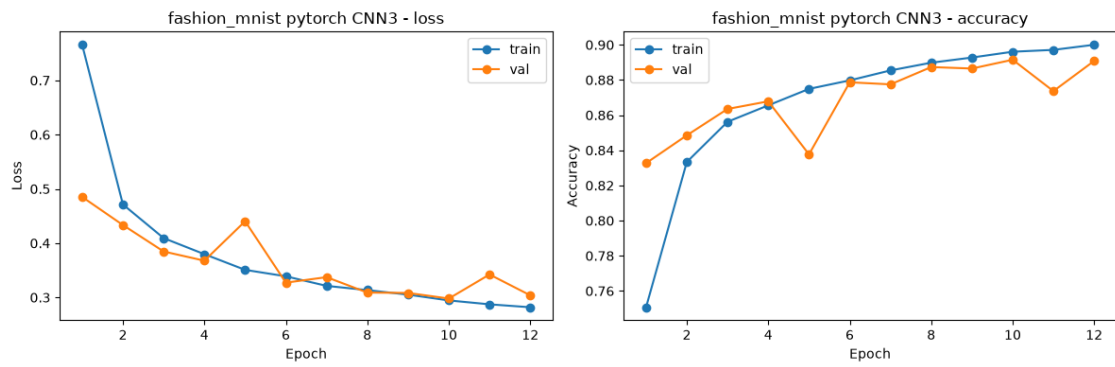
```

fashion_mnist | pytorch | CNN3

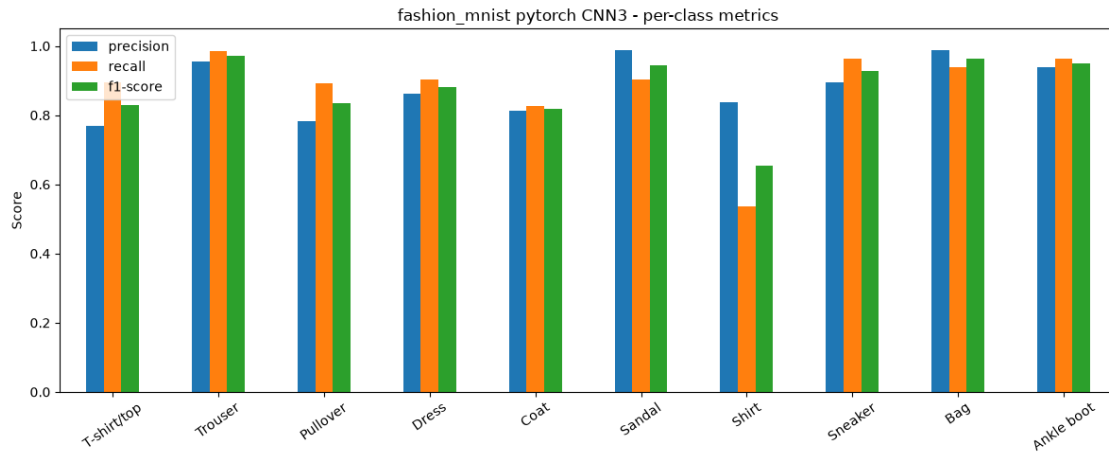
	value
accuracy	0.8804
precision_macro	0.883201
recall_macro	0.8804
f1_macro	0.877225
precision_weighted	0.883201
recall_weighted	0.8804
f1_weighted	0.877225
log_loss	0.342884
mean_confidence	0.894253
ece	0.014683
top5_accuracy	0.9971
dataset	fashion_mnist
framework	pytorch

conv_layers

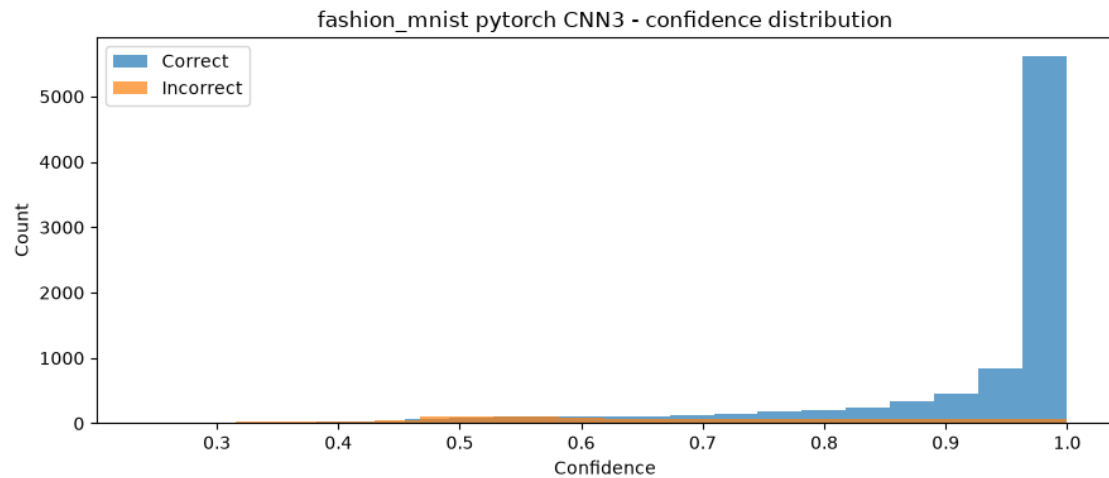
3

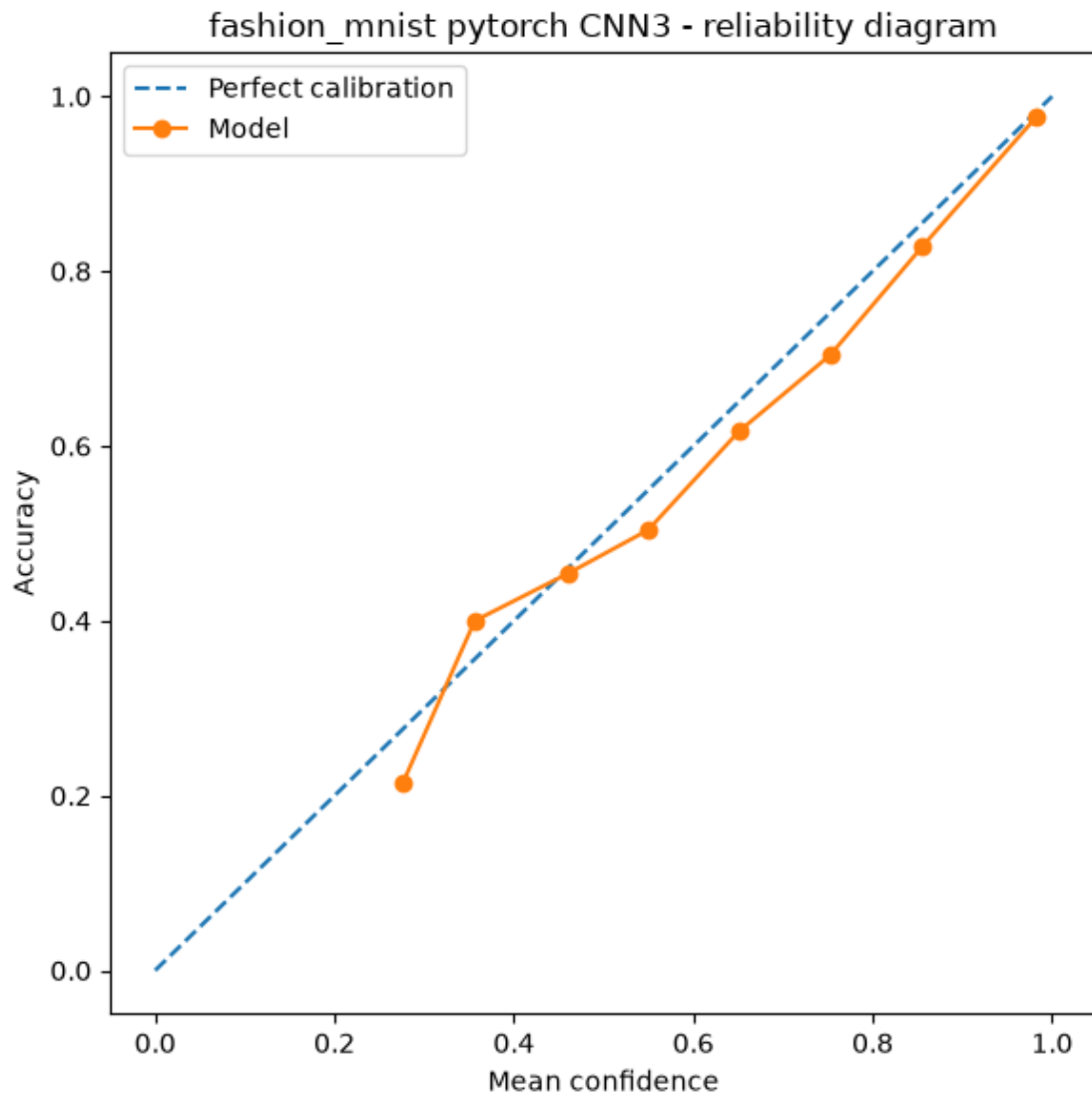


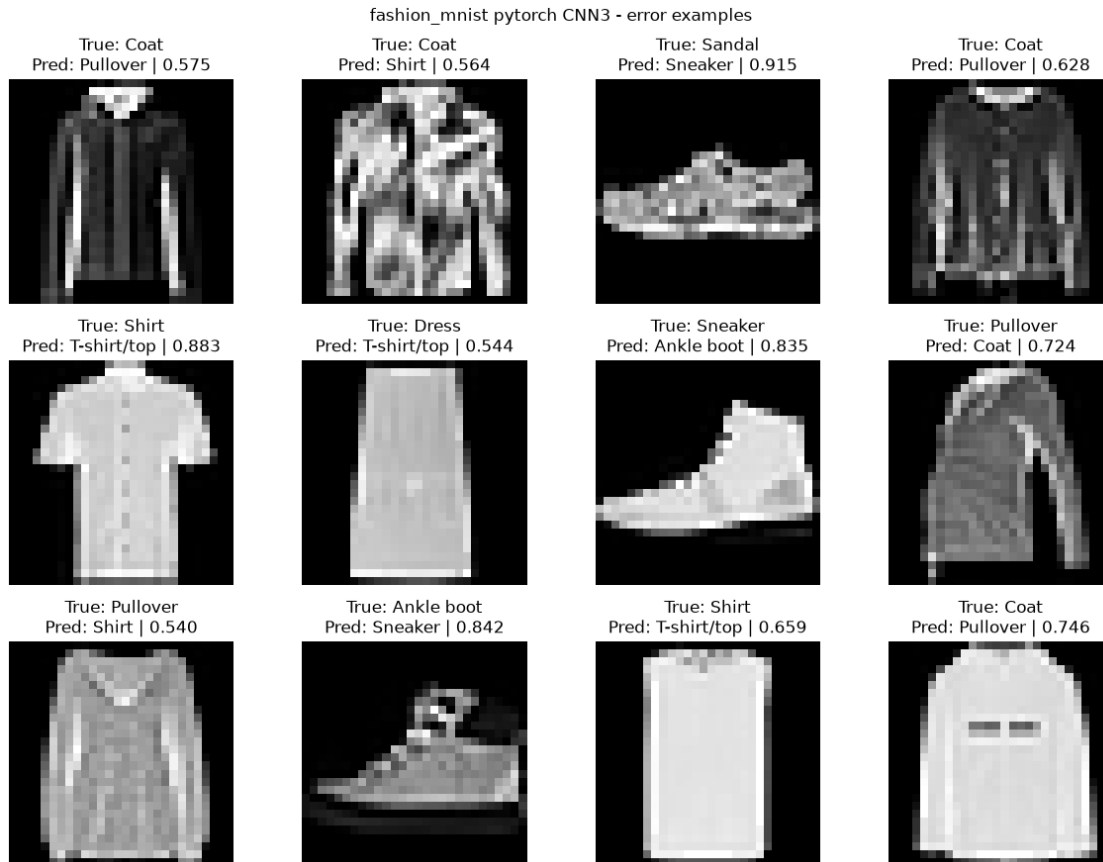
Per-class metrics:



	precision	recall	f1-score	support
T-shirt/top	0.769759	0.896	0.828096	1000.0
Trouser	0.956311	0.985	0.970443	1000.0
Pullover	0.783641	0.891	0.833879	1000.0
Dress	0.863158	0.902	0.882152	1000.0
Coat	0.811580	0.827	0.819217	1000.0
Sandal	0.987952	0.902	0.943021	1000.0
Shirt	0.837754	0.537	0.654479	1000.0
Sneaker	0.894884	0.962	0.927229	1000.0
Bag	0.988433	0.940	0.963608	1000.0
Ankle boot	0.938537	0.962	0.950123	1000.0







Top confusion pairs:

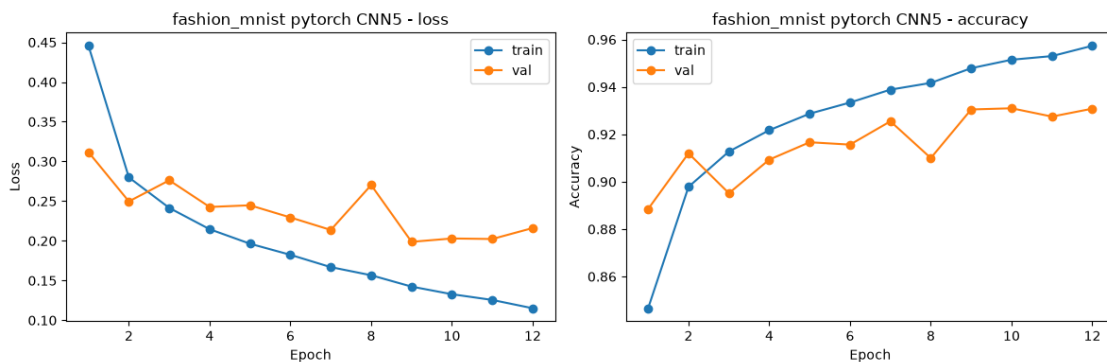
	count	true_class	predicted_class
0	210	Shirt	T-shirt/top
1	106	Shirt	Coat
2	100	Shirt	Pullover
3	83	Coat	Pullover
4	75	Sandal	Sneaker
5	59	Pullover	Coat
6	50	Coat	Dress
7	37	T-shirt/top	Pullover
8	34	Coat	Shirt
9	34	Ankle boot	Sneaker
10	33	Sneaker	Ankle boot
11	33	Shirt	Dress
12	30	T-shirt/top	Dress
13	30	Dress	T-shirt/top
14	25	T-shirt/top	Shirt

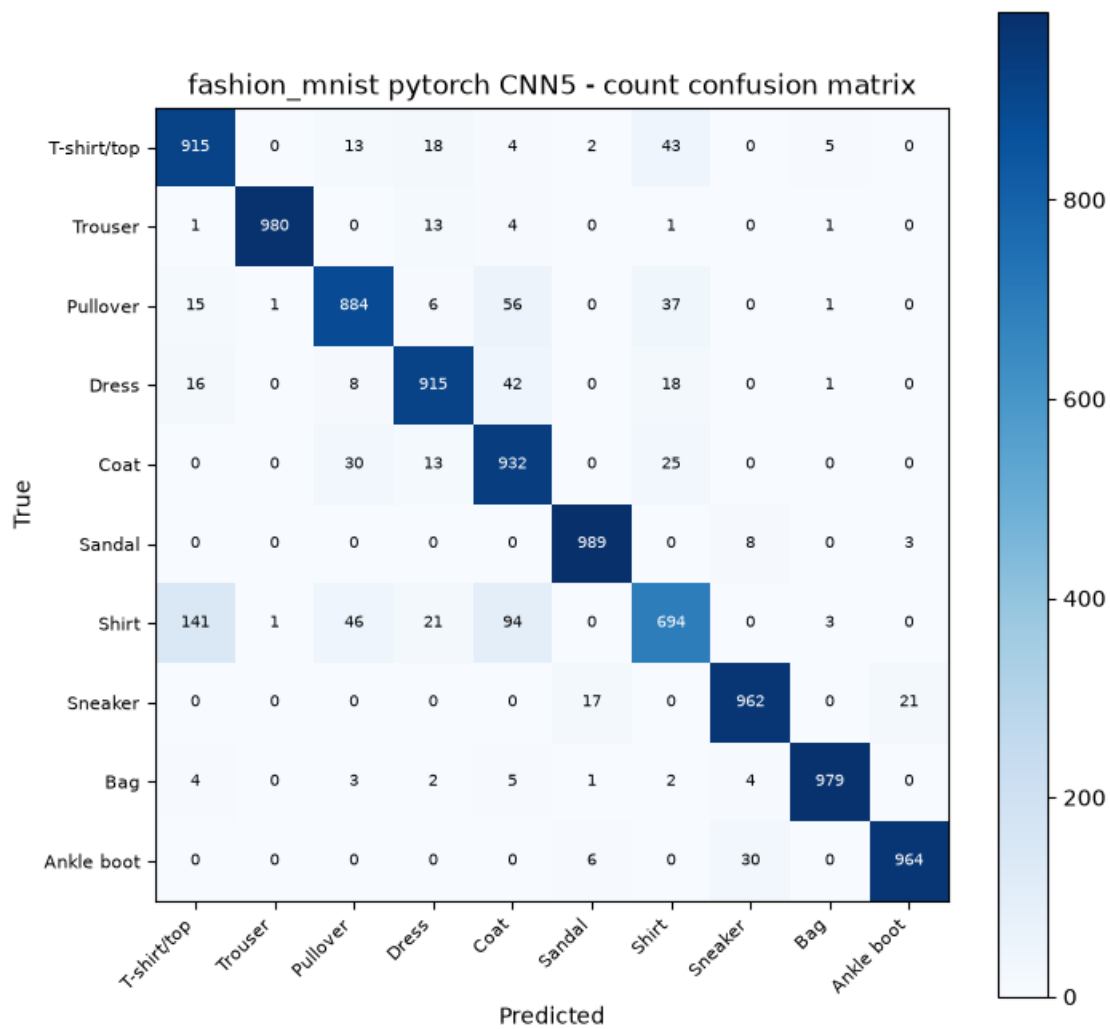
PyTorch Fashion CNN5 | params: 426,858

```
Epoch 01 | train loss 0.4456 acc 0.8466 | val loss 0.3117 acc 0.8885
Epoch 02 | train loss 0.2799 acc 0.8979 | val loss 0.2497 acc 0.9120
Epoch 03 | train loss 0.2415 acc 0.9127 | val loss 0.2762 acc 0.8952
Epoch 04 | train loss 0.2147 acc 0.9218 | val loss 0.2427 acc 0.9093
Epoch 05 | train loss 0.1964 acc 0.9287 | val loss 0.2448 acc 0.9167
Epoch 06 | train loss 0.1824 acc 0.9334 | val loss 0.2295 acc 0.9157
Epoch 07 | train loss 0.1669 acc 0.9389 | val loss 0.2137 acc 0.9255
Epoch 08 | train loss 0.1567 acc 0.9417 | val loss 0.2703 acc 0.9100
Epoch 09 | train loss 0.1424 acc 0.9479 | val loss 0.1988 acc 0.9305
Epoch 10 | train loss 0.1328 acc 0.9515 | val loss 0.2030 acc 0.9310
Epoch 11 | train loss 0.1256 acc 0.9530 | val loss 0.2023 acc 0.9275
Epoch 12 | train loss 0.1151 acc 0.9574 | val loss 0.2161 acc 0.9308
Early stopping.
```

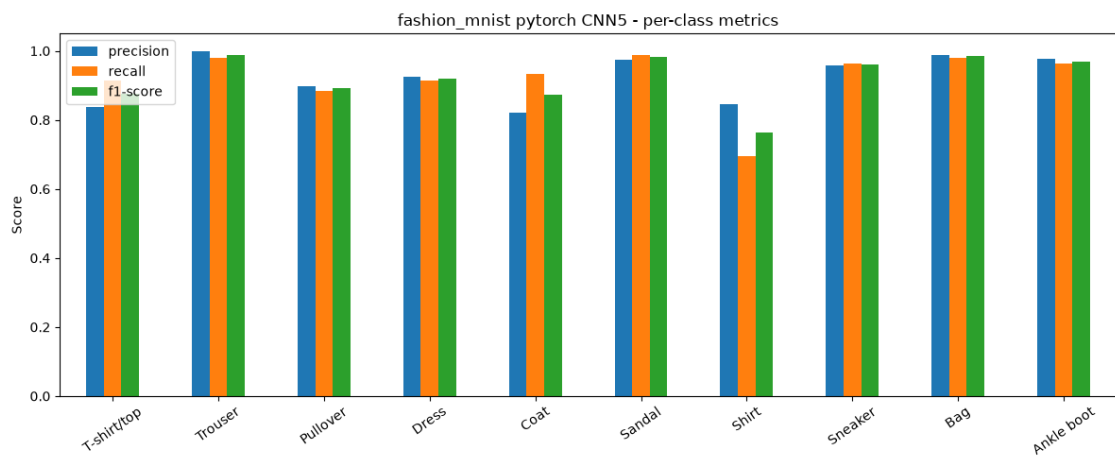
```
fashion_mnist | pytorch | CNN5
```

	value
accuracy	0.9214
precision_macro	0.922355
recall_macro	0.9214
f1_macro	0.920566
precision_weighted	0.922355
recall_weighted	0.9214
f1_weighted	0.920566
log_loss	0.23249
mean_confidence	0.945136
ece	0.023736
top5_accuracy	0.9994
dataset	fashion_mnist
framework	pytorch
conv_layers	5

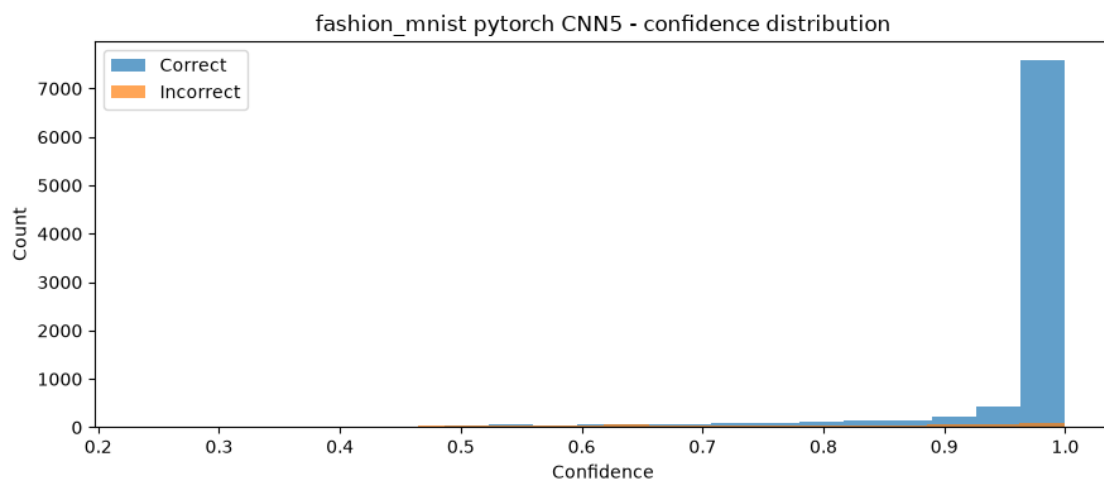


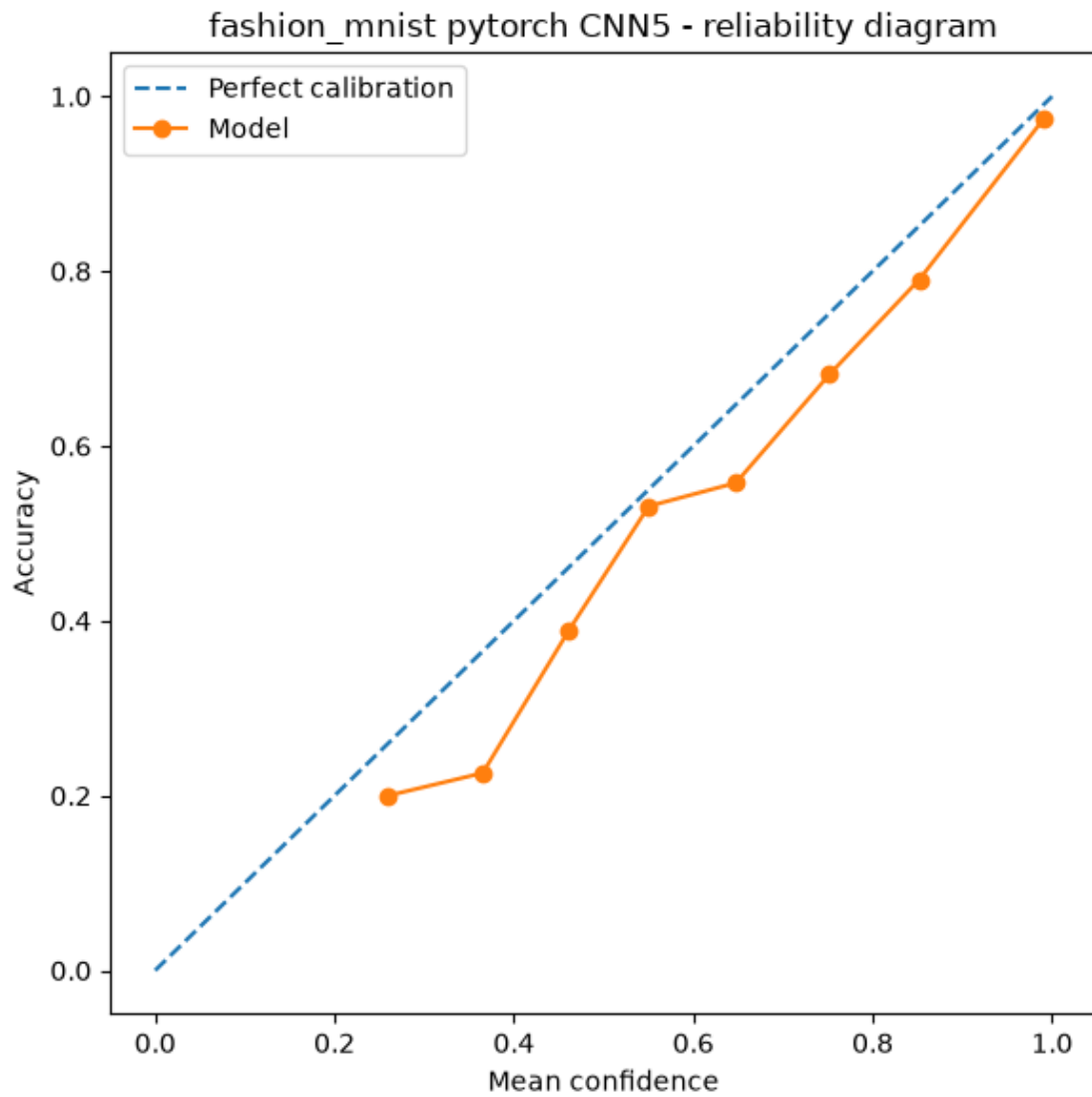


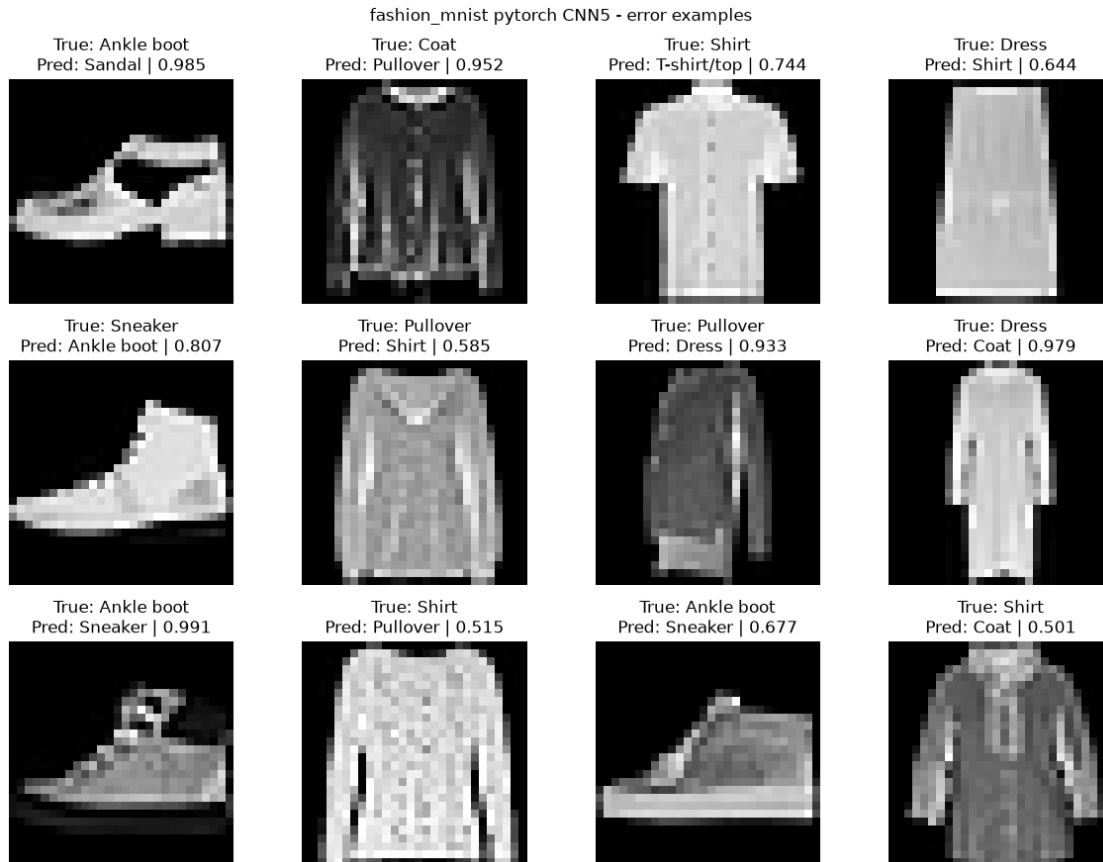
Per-class metrics:



	precision	recall	f1-score	support
T-shirt/top	0.837912	0.915	0.874761	1000.0
Trouser	0.997963	0.980	0.988900	1000.0
Pullover	0.898374	0.884	0.891129	1000.0
Dress	0.926113	0.915	0.920523	1000.0
Coat	0.819701	0.932	0.872251	1000.0
Sandal	0.974384	0.989	0.981638	1000.0
Shirt	0.846341	0.694	0.762637	1000.0
Sneaker	0.958167	0.962	0.960080	1000.0
Bag	0.988889	0.979	0.983920	1000.0
Ankle boot	0.975709	0.964	0.969819	1000.0







Top confusion pairs:

	count	true_class	predicted_class
0	141	Shirt	T-shirt/top
1	94	Shirt	Coat
2	56	Pullover	Coat
3	46	Shirt	Pullover
4	43	T-shirt/top	Shirt
5	42	Dress	Coat
6	37	Pullover	Shirt
7	30	Coat	Pullover
8	30	Ankle boot	Sneaker
9	25	Coat	Shirt
10	21	Sneaker	Ankle boot
11	21	Shirt	Dress
12	18	T-shirt/top	Dress
13	18	Dress	Shirt
14	17	Sneaker	Sandal

9.4 9.2 PyTorch - CIFAR-100 CNN3 and CNN5

The CIFAR-100 PyTorch experiment changes the input channel count from 1 to 3 and the classifier output from 10 to 100.

Everything else remains structurally aligned with the Fashion-MNIST experiment so that the report can separate dataset effects from framework effects.

Because CIFAR-100 is balanced by construction, class weights are not needed for the image model. The emphasis therefore falls on representation capacity, top-1 versus top-5 accuracy, class-level F1, confusion patterns, and computational cost.

```
[27]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 46.
      cifar_train_ds=NumpyImageDataset(X_cifar_tr,y_cifar_tr,True)
      cifar_val_ds=NumpyImageDataset(X_cifar_val,y_cifar_val,False)
      cifar_test_ds=NumpyImageDataset(X_cifar_test,y_cifar_test,False)
      cifar_train_dl=loader(cifar_train_ds,True); cifar_val_dl=loader(cifar_val_ds);
      ↪ cifar_test_dl=loader(cifar_test_ds)
      cifar_weights={i:1.0 for i in range(100)}

      for depth in (3,5):
          model=ImageCNN(3,100,depth).to(DEVICE); path=MODELS_DIR/
          ↪ f"cifar100_pytorch_cnn{depth}.pth"
          params=sum(p.numel() for p in model.parameters() if p.requires_grad)
          print(f"\nPyTorch CIFAR-100 CNN{depth} | params: {params:,}")
          history=train_torch(model,cifar_train_dl,cifar_val_dl,cifar_weights,path)
          y_true,_,probs=predict_torch(model,cifar_test_dl)

          ↪
          ↪ metrics,_,=evaluate_and_visualize("cifar100","pytorch",depth,y_true,probs,CIFAR100_NAMES,X_c
          metrics["parameters"]=int(params);
          ↪ metrics["training_seconds"]=float(history["training_seconds"])
          TORCH_RESULTS[("cifar100",depth)]=metrics

      # Re-save the final metric record after runtime and parameter fields are
      ↪ attached.
      save_experiment("cifar100", "pytorch", depth, y_true, probs, metrics)
```

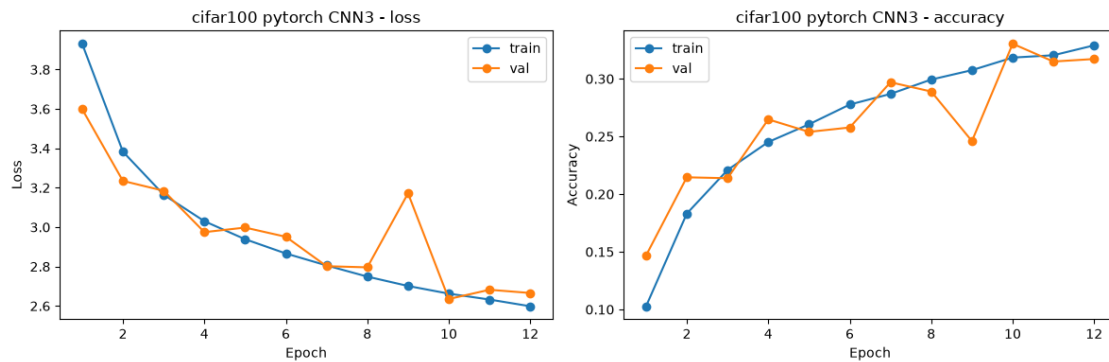
PyTorch CIFAR-100 CNN3 | params: 106,372

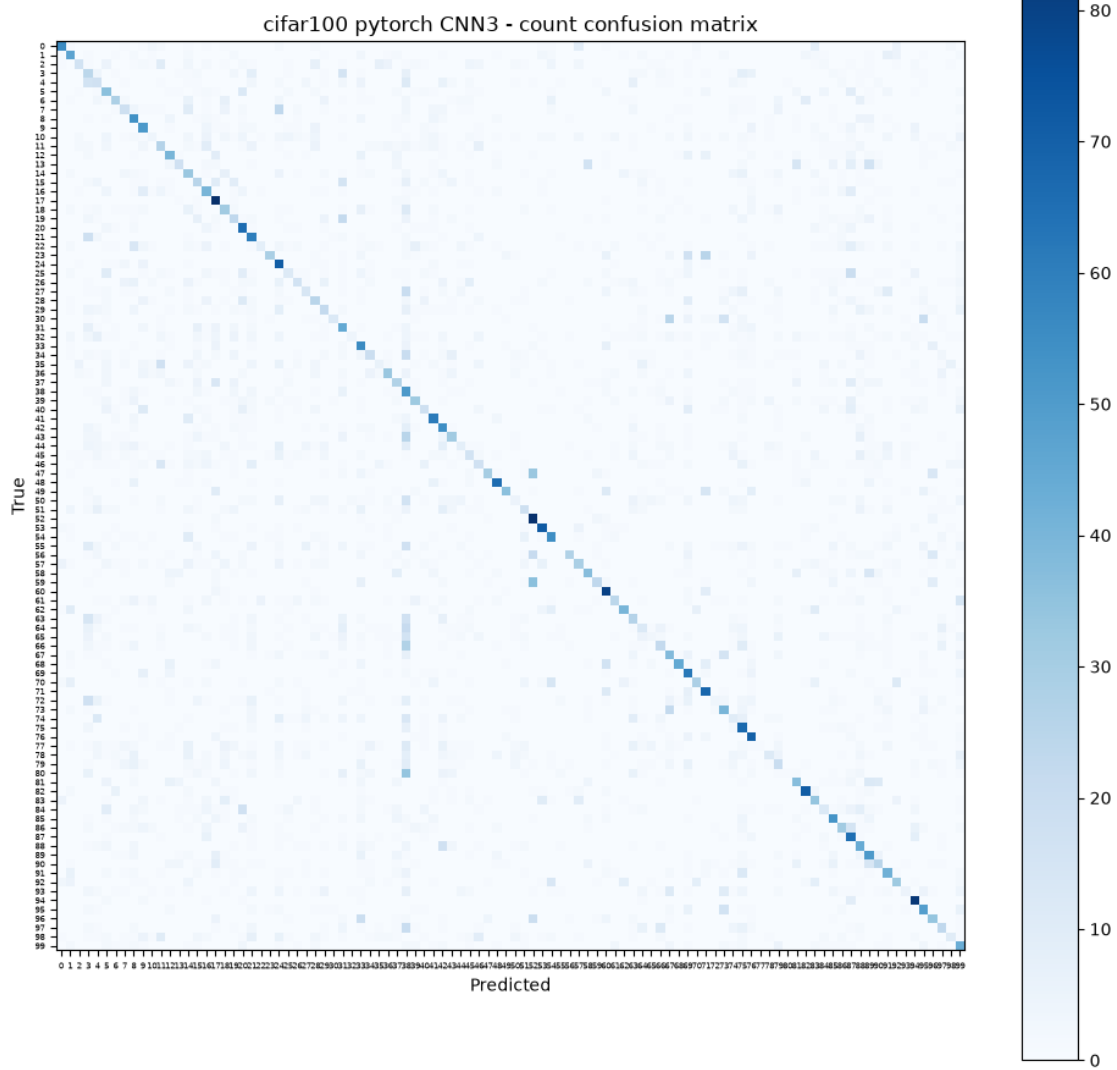
Epoch 01		train loss 3.9309	acc 0.1028		val loss 3.5989	acc 0.1468
Epoch 02		train loss 3.3831	acc 0.1831		val loss 3.2348	acc 0.2146
Epoch 03		train loss 3.1647	acc 0.2209		val loss 3.1847	acc 0.2138
Epoch 04		train loss 3.0302	acc 0.2452		val loss 2.9747	acc 0.2648
Epoch 05		train loss 2.9388	acc 0.2606		val loss 2.9979	acc 0.2540
Epoch 06		train loss 2.8664	acc 0.2777		val loss 2.9512	acc 0.2578
Epoch 07		train loss 2.8057	acc 0.2869		val loss 2.8018	acc 0.2970
Epoch 08		train loss 2.7489	acc 0.2994		val loss 2.7960	acc 0.2890

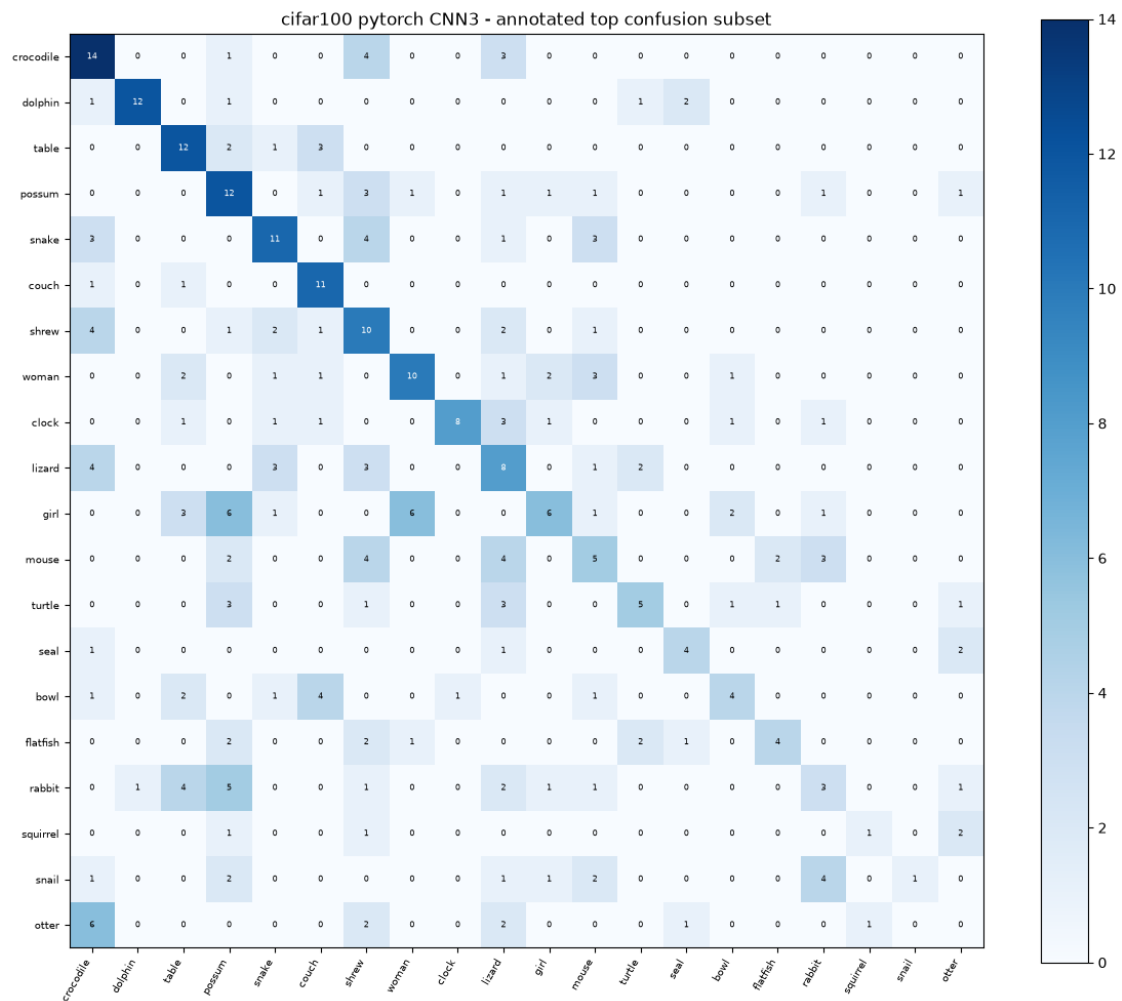
Epoch 09 | train loss 2.7015 acc 0.3075 | val loss 3.1717 acc 0.2460
 Epoch 10 | train loss 2.6625 acc 0.3184 | val loss 2.6353 acc 0.3304
 Epoch 11 | train loss 2.6325 acc 0.3204 | val loss 2.6822 acc 0.3150
 Epoch 12 | train loss 2.5985 acc 0.3290 | val loss 2.6658 acc 0.3172

cifar100 | pytorch | CNN3

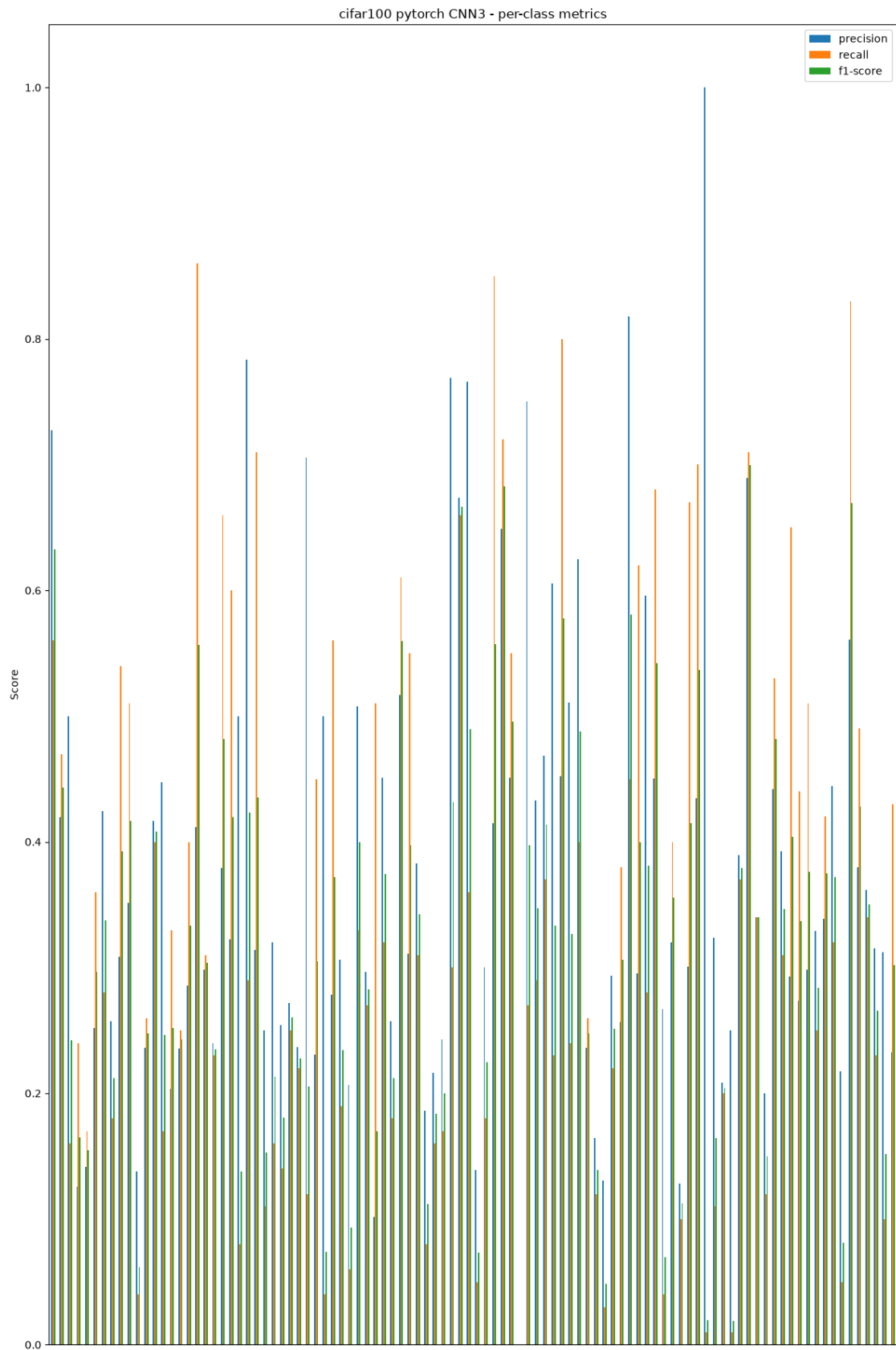
	value
accuracy	0.3384
precision_macro	0.370387
recall_macro	0.3384
f1_macro	0.317223
precision_weighted	0.370387
recall_weighted	0.3384
f1_weighted	0.317223
log_loss	2.590736
mean_confidence	0.269867
ece	0.068533
top5_accuracy	0.6578
dataset	cifar100
framework	pytorch
conv_layers	3





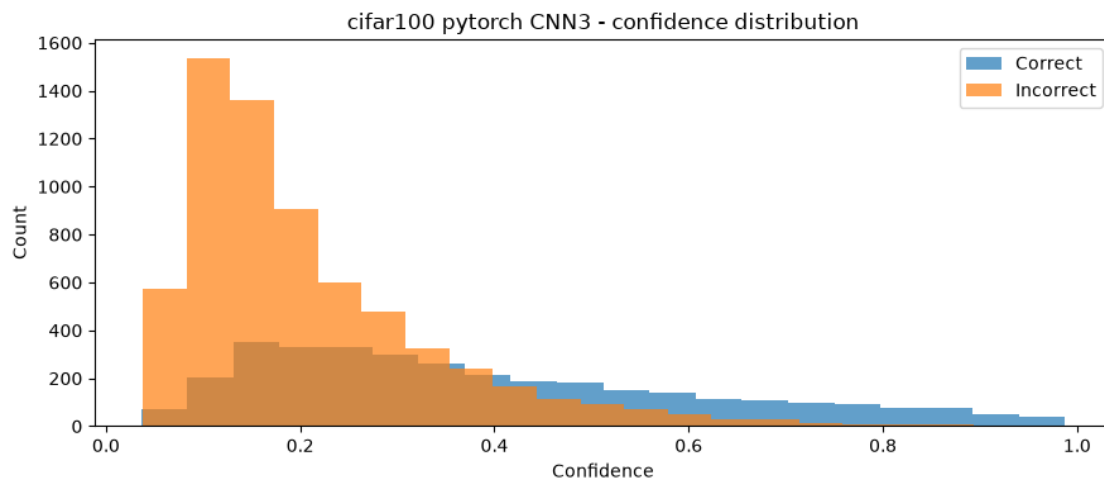


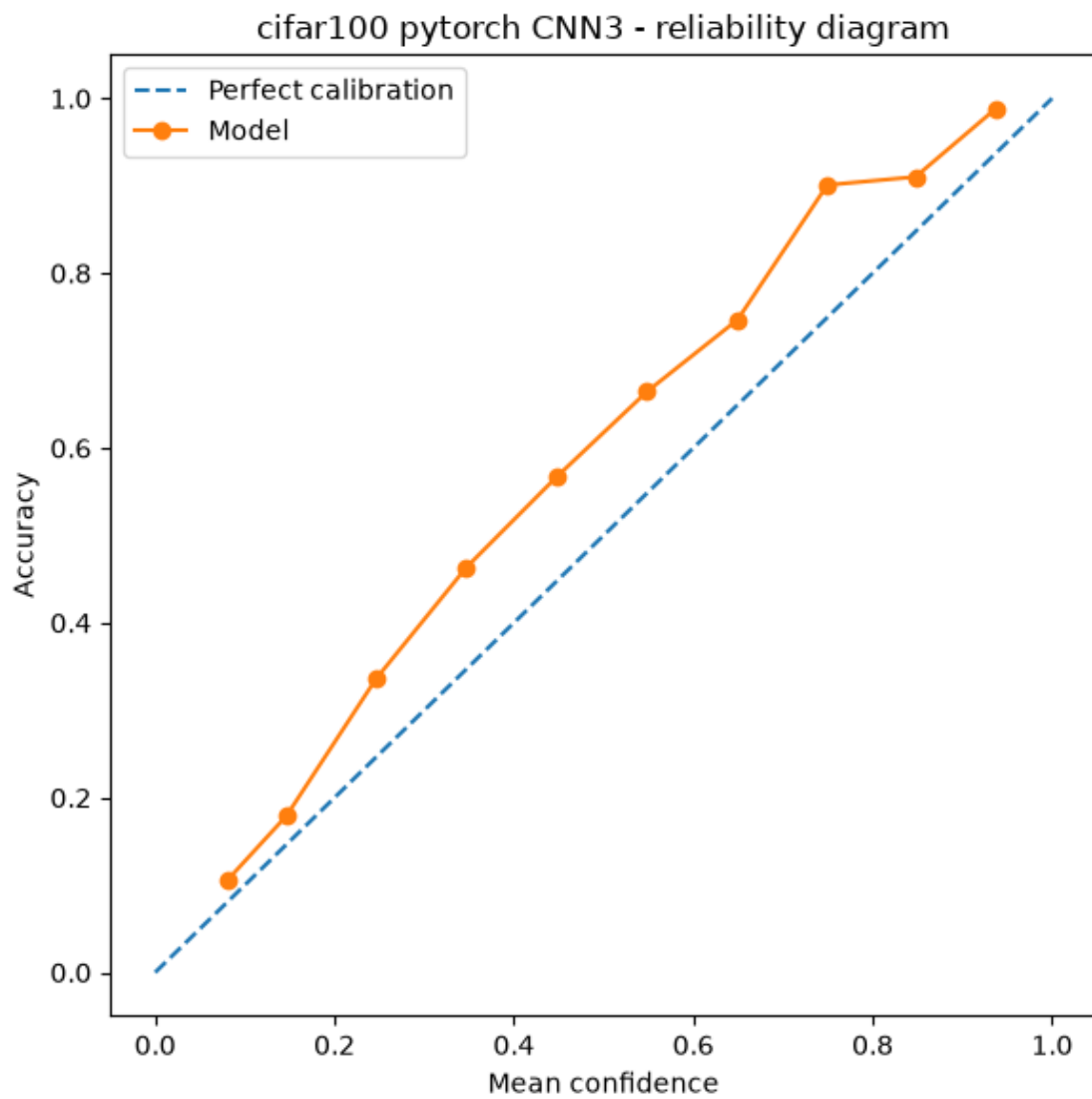
Per-class metrics:



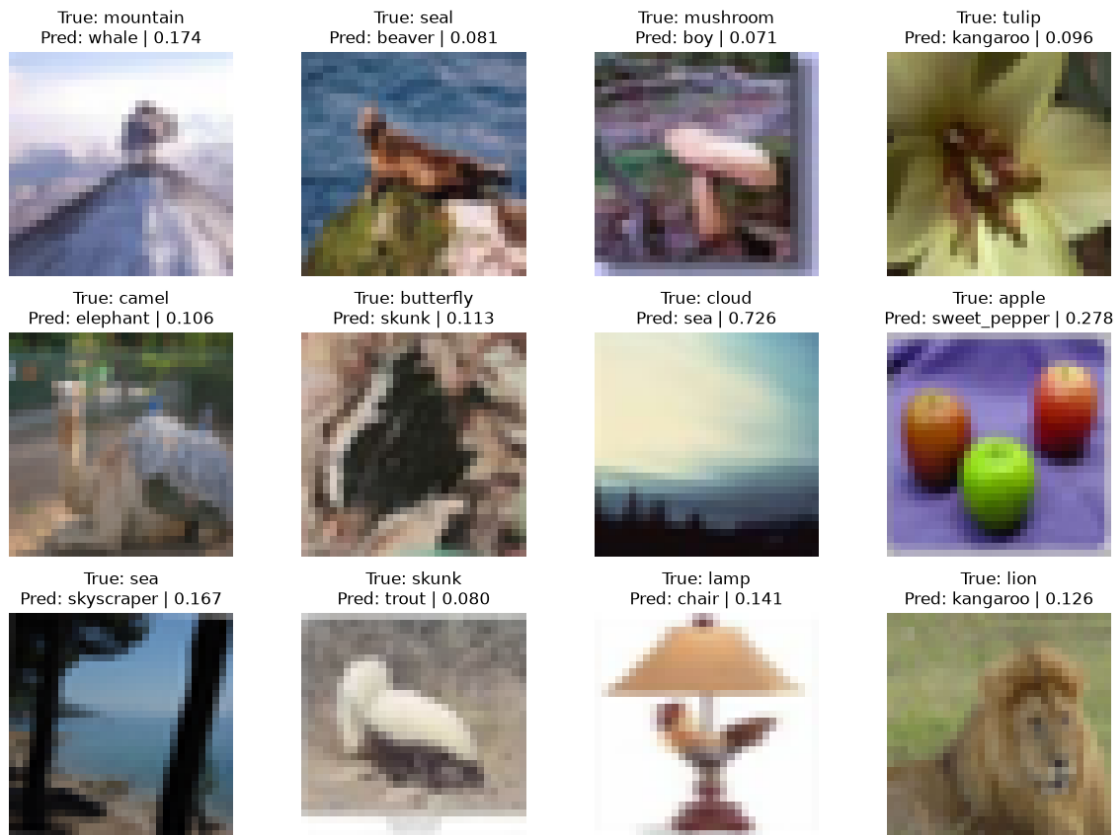
	precision	recall	f1-score	support
apple	0.727273	0.56	0.632768	100.0
aquarium_fish	0.419643	0.47	0.443396	100.0
baby	0.500000	0.16	0.242424	100.0
bear	0.125654	0.24	0.164948	100.0
beaver	0.141667	0.17	0.154545	100.0
...	...	...	...	...
whale	0.379845	0.49	0.427948	100.0
willow_tree	0.361702	0.34	0.350515	100.0
wolf	0.315068	0.23	0.265896	100.0
woman	0.312500	0.10	0.151515	100.0
worm	0.232432	0.43	0.301754	100.0

[100 rows x 4 columns]





cifar100 pytorch CNN3 - error examples



Top confusion pairs:

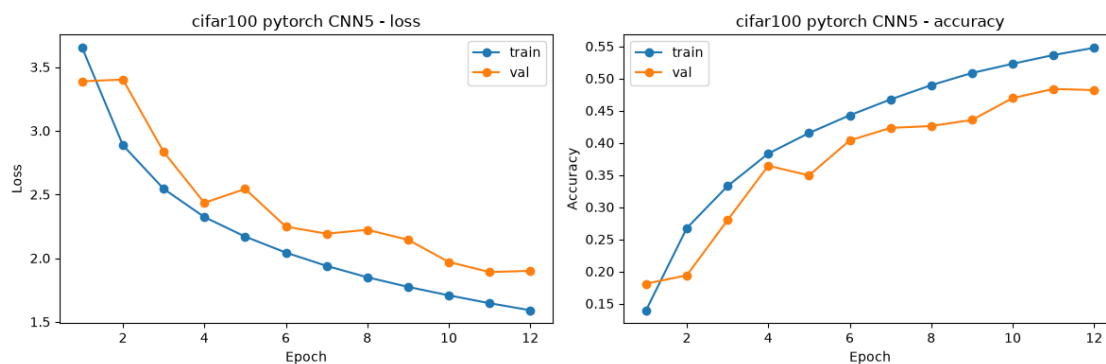
	count	true_class	predicted_class
0	36	pine_tree	oak_tree
1	34	squirrel	kangaroo
2	33	maple_tree	oak_tree
3	27	raccoon	kangaroo
4	25	lion	kangaroo
5	25	dolphin	ray
6	25	cloud	sea
7	23	beetle	cockroach
8	22	shark	ray
9	22	cattle	elephant
10	21	possum	kangaroo
11	21	palm_tree	oak_tree
12	20	wolf	kangaroo
13	20	fox	kangaroo
14	20	dolphin	whale

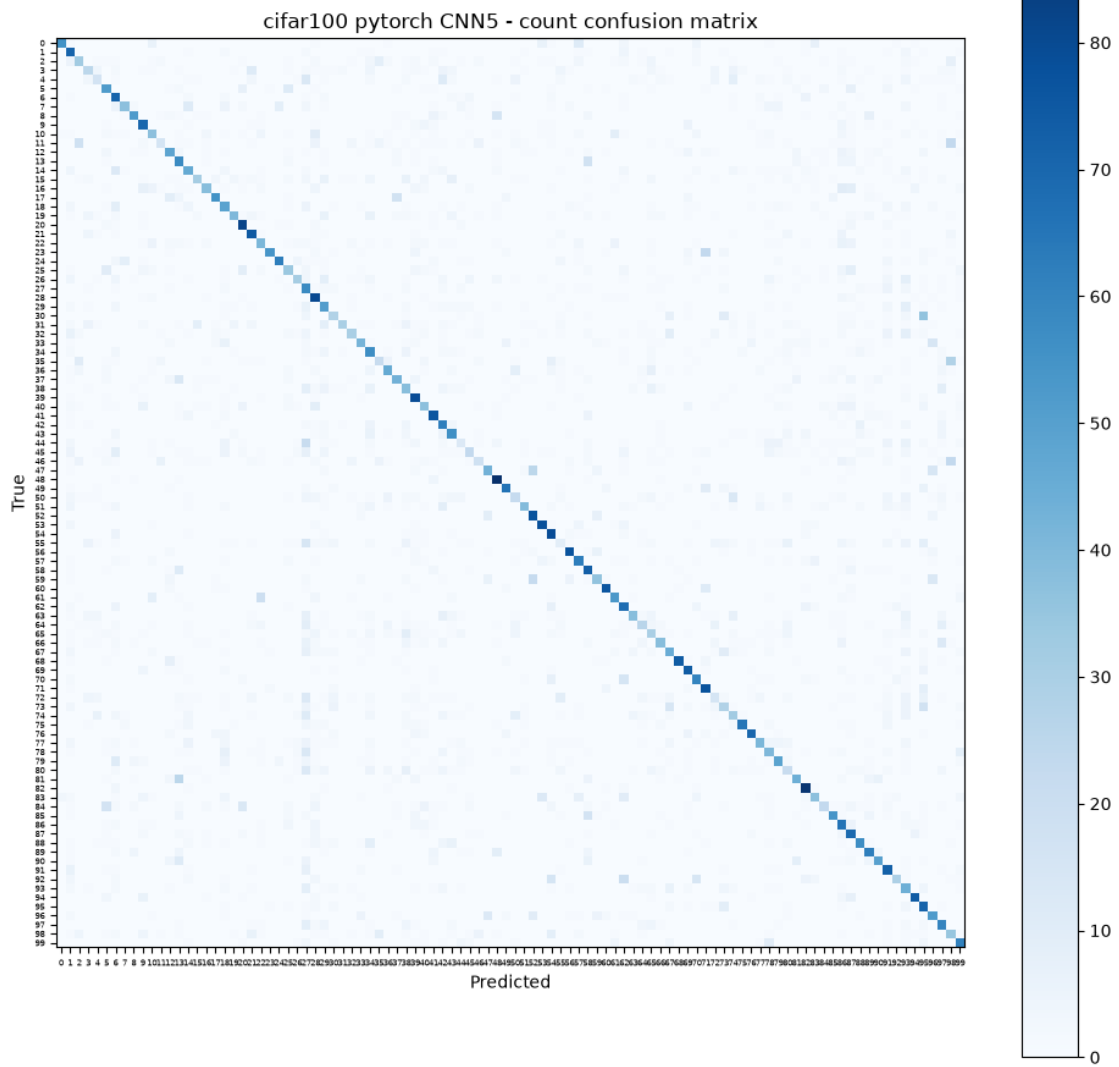
PyTorch CIFAR-100 CNN5 | params: 441,924

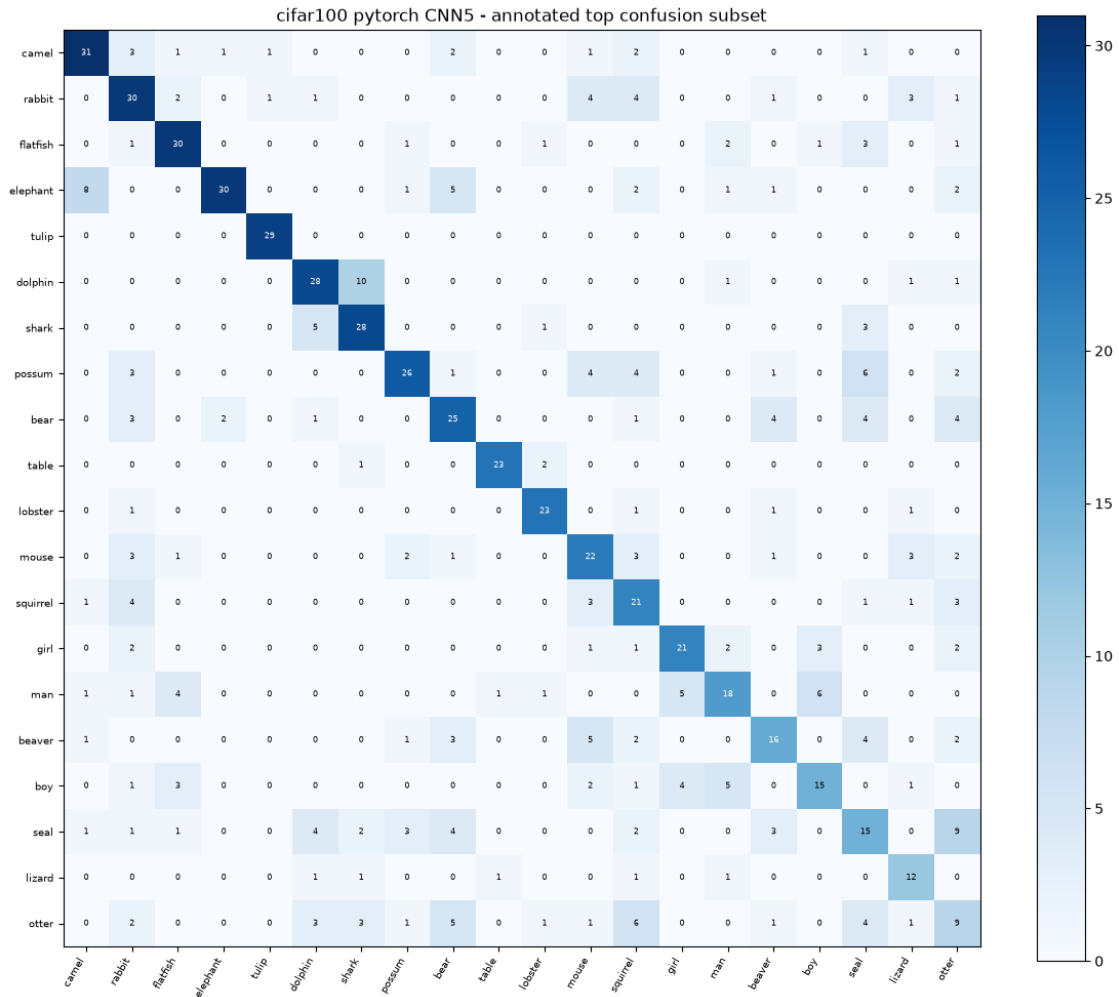
Epoch 01		train loss	3.6510	acc	0.1395		val loss	3.3881	acc	0.1808
Epoch 02		train loss	2.8862	acc	0.2672		val loss	3.4014	acc	0.1938
Epoch 03		train loss	2.5438	acc	0.3326		val loss	2.8334	acc	0.2796
Epoch 04		train loss	2.3224	acc	0.3831		val loss	2.4338	acc	0.3642
Epoch 05		train loss	2.1691	acc	0.4151		val loss	2.5431	acc	0.3492
Epoch 06		train loss	2.0436	acc	0.4426		val loss	2.2472	acc	0.4036
Epoch 07		train loss	1.9390	acc	0.4673		val loss	2.1921	acc	0.4230
Epoch 08		train loss	1.8498	acc	0.4895		val loss	2.2222	acc	0.4260
Epoch 09		train loss	1.7743	acc	0.5084		val loss	2.1443	acc	0.4354
Epoch 10		train loss	1.7078	acc	0.5228		val loss	1.9698	acc	0.4694
Epoch 11		train loss	1.6463	acc	0.5363		val loss	1.8907	acc	0.4836
Epoch 12		train loss	1.5909	acc	0.5476		val loss	1.9002	acc	0.4818

cifar100 | pytorch | CNN5

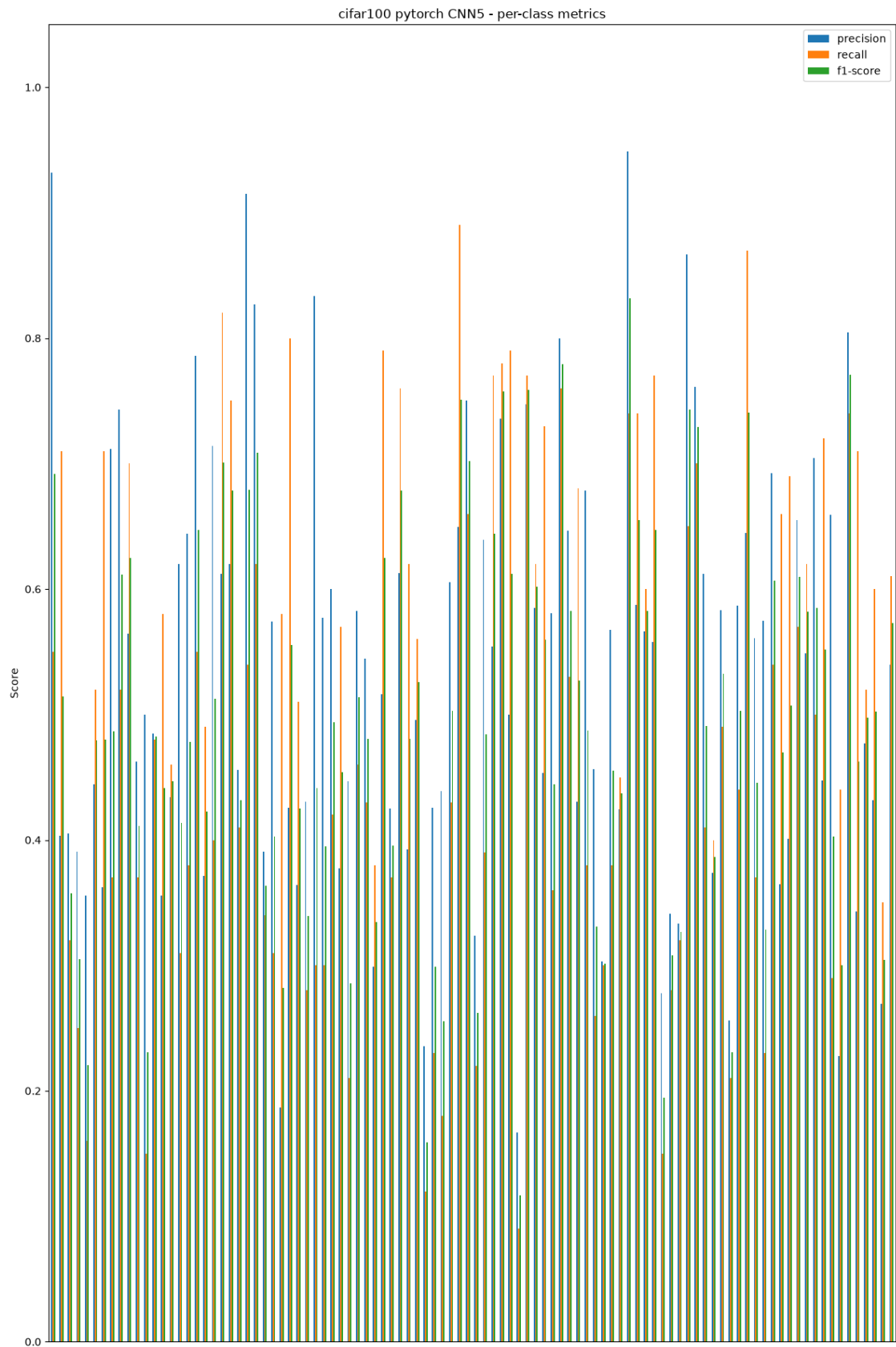
	value
accuracy	0.4981
precision_macro	0.528841
recall_macro	0.4981
f1_macro	0.49142
precision_weighted	0.528841
recall_weighted	0.4981
f1_weighted	0.49142
log_loss	1.84128
mean_confidence	0.499991
ece	0.013712
top5_accuracy	0.8003
dataset	cifar100
framework	pytorch
conv_layers	5





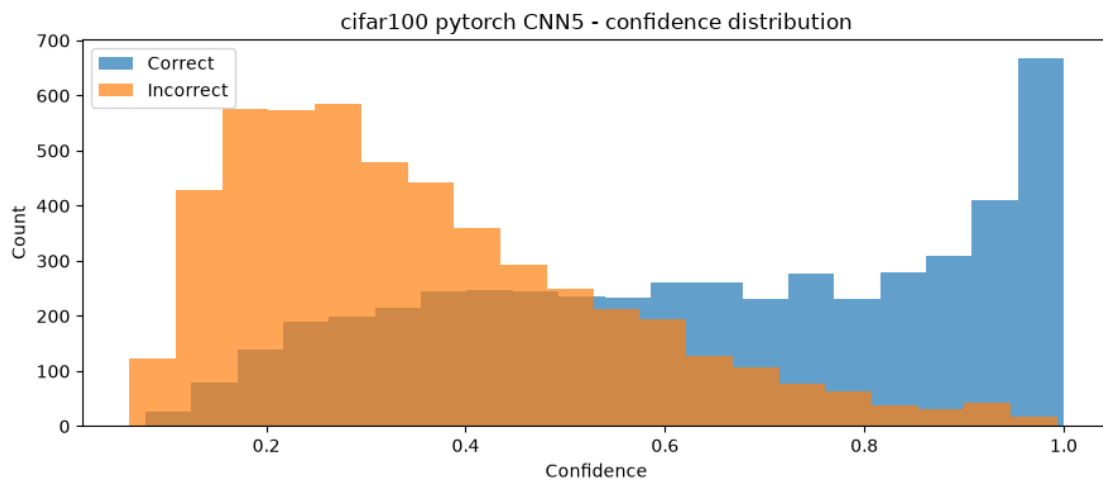


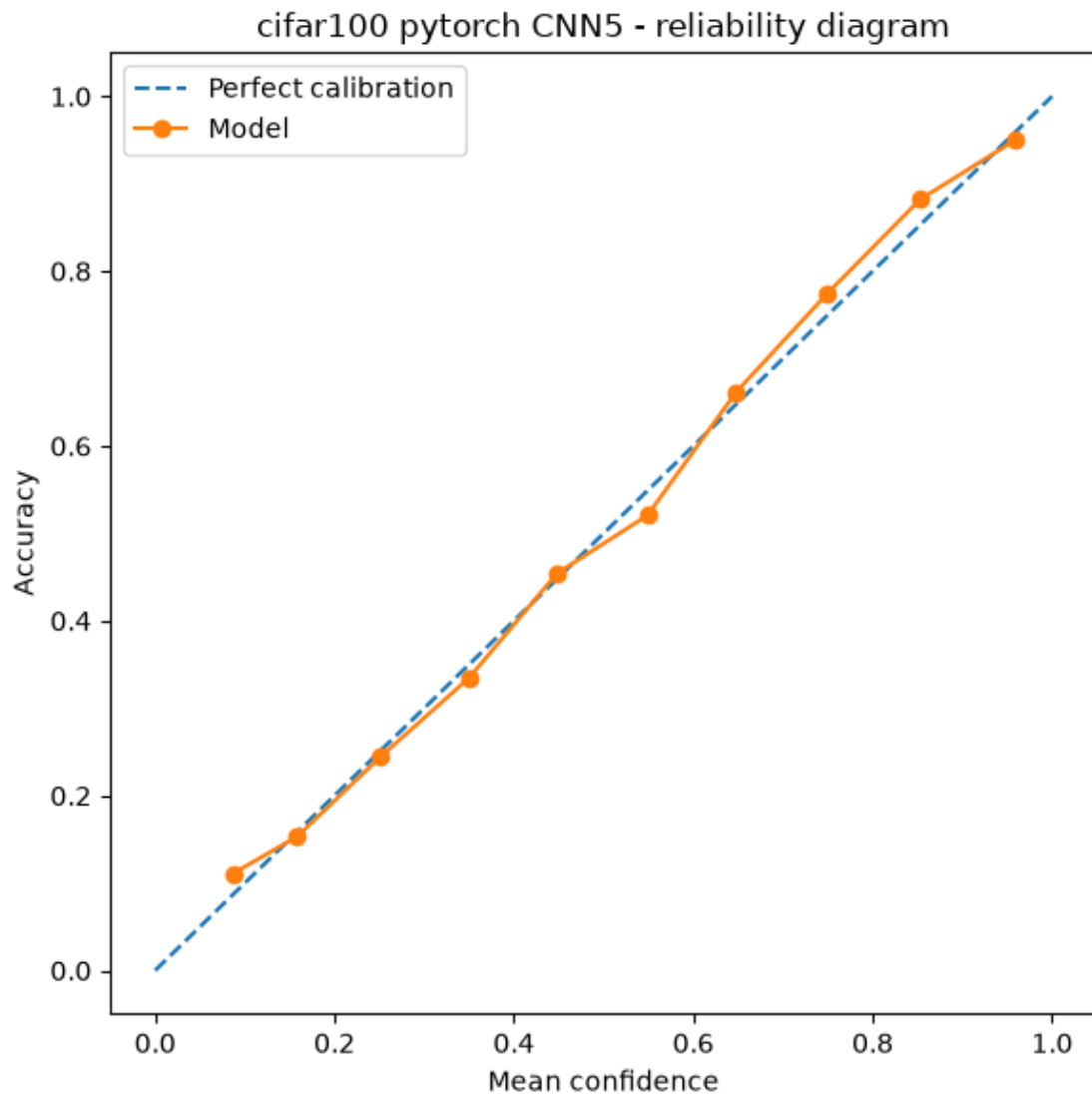
Per-class metrics:



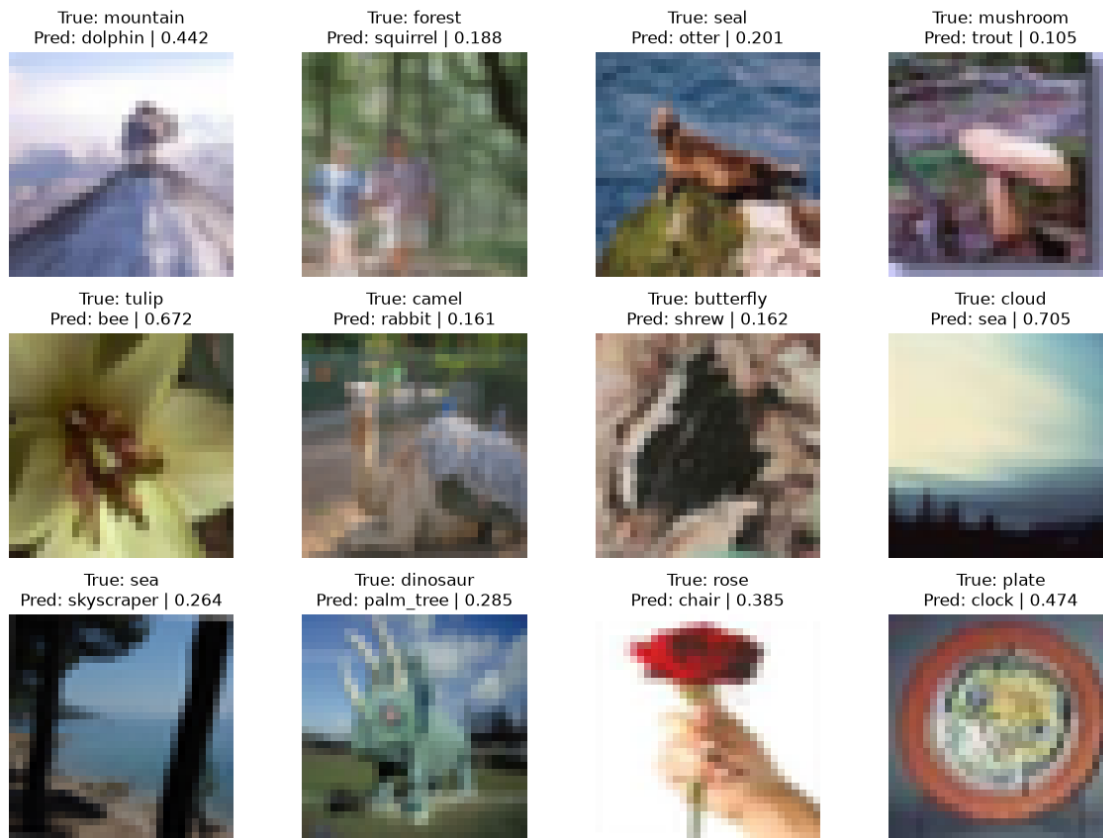
	precision	recall	f1-score	support
apple	0.932203	0.55	0.691824	100.0
aquarium_fish	0.403409	0.71	0.514493	100.0
baby	0.405063	0.32	0.357542	100.0
bear	0.390625	0.25	0.304878	100.0
beaver	0.355556	0.16	0.220690	100.0
...	...	...	...	...
whale	0.342995	0.71	0.462541	100.0
willow_tree	0.477064	0.52	0.497608	100.0
wolf	0.431655	0.60	0.502092	100.0
woman	0.269231	0.35	0.304348	100.0
worm	0.539823	0.61	0.572770	100.0

[100 rows x 4 columns]





cifar100 pytorch CNN5 - error examples



Top confusion pairs:

	count	true_class	predicted_class
0	36	dolphin	whale
1	28	girl	woman
2	25	streetcar	bus
3	25	maple_tree	oak_tree
4	23	man	woman
5	23	cloud	sea
6	23	boy	woman
7	22	pine_tree	oak_tree
8	21	lizard	crocodile
9	20	tulip	poppy
10	19	plate	clock
11	18	boy	baby
12	17	shark	whale
13	17	castle	house
14	17	bus	pickup_truck

9.5 9.3 PyTorch - Diabetes Conv1D CNN3 and CNN5

For Diabetes, PyTorch receives tensors in channel-first Conv1D format:

```
(batch, 1, feature_count)
```

The model definition follows the same conceptual pattern as TensorFlow:

Conv1D → BatchNorm1D → ReLU → Pooling → ... → Global aggregation → Linear

The criterion is weighted using the training-derived class weights. This is important because applying weights calculated from the complete dataset would let information from the evaluation partitions influence the training objective.

The test predictions are converted to NumPy probability arrays so that the same evaluation utilities can compare TensorFlow and PyTorch without framework-specific metric code.

```
[28]: diab_train_ds = NumpyTabularDataset(X_diab_tr, y_diab_tr)
      diab_val_ds = NumpyTabularDataset(X_diab_val, y_diab_val)
      diab_test_ds = NumpyTabularDataset(X_diab_test, y_diab_test)

      diab_train_dl = loader(diab_train_ds, True)
      diab_val_dl = loader(diab_val_ds)
      diab_test_dl = loader(diab_test_ds)

      for depth in (3, 5):
          model = TabularCNN(3, depth).to(DEVICE)
          path = MODELS_DIR / f"diabetes_pytorch_cnn{depth}.pth"

          params = sum(
              p.numel()
              for p in model.parameters()
              if p.requires_grad
          )

          print(f"\nPyTorch Diabetes CNN{depth} | params: {params:,}")

          history = train_torch(
              model,
              diab_train_dl,
              diab_val_dl,
              diabetes_data["class_weight"],
              path
          )

          y_true, _, probs = predict_torch(
              model,
              diab_test_dl
          )
```

```

metrics, _ = evaluate_and_visualize(
    "diabetes",
    "pytorch",
    depth,
    y_true,
    probs,
    DIABETES_NAMES,
    None,
    history
)

metrics["parameters"] = int(params)
metrics["training_seconds"] = float(
    history["training_seconds"]
)

TORCH_RESULTS[("diabetes", depth)] = metrics

save_experiment(
    "diabetes",
    "pytorch",
    depth,
    y_true,
    probs,
    metrics
)

```

```

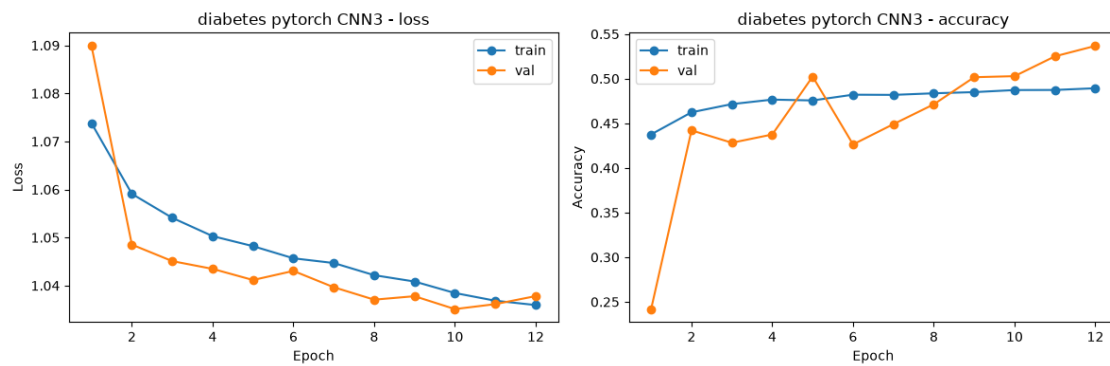
PyTorch Diabetes CNN3 | params: 31,651
Epoch 01 | train loss 1.0738 acc 0.4376 | val loss 1.0899 acc 0.2419
Epoch 02 | train loss 1.0591 acc 0.4623 | val loss 1.0485 acc 0.4421
Epoch 03 | train loss 1.0541 acc 0.4715 | val loss 1.0451 acc 0.4282
Epoch 04 | train loss 1.0503 acc 0.4766 | val loss 1.0435 acc 0.4375
Epoch 05 | train loss 1.0482 acc 0.4756 | val loss 1.0412 acc 0.5023
Epoch 06 | train loss 1.0457 acc 0.4820 | val loss 1.0431 acc 0.4264
Epoch 07 | train loss 1.0447 acc 0.4818 | val loss 1.0397 acc 0.4491
Epoch 08 | train loss 1.0422 acc 0.4836 | val loss 1.0371 acc 0.4713
Epoch 09 | train loss 1.0408 acc 0.4849 | val loss 1.0378 acc 0.5016
Epoch 10 | train loss 1.0385 acc 0.4872 | val loss 1.0351 acc 0.5028
Epoch 11 | train loss 1.0368 acc 0.4873 | val loss 1.0361 acc 0.5250
Epoch 12 | train loss 1.0359 acc 0.4893 | val loss 1.0378 acc 0.5367

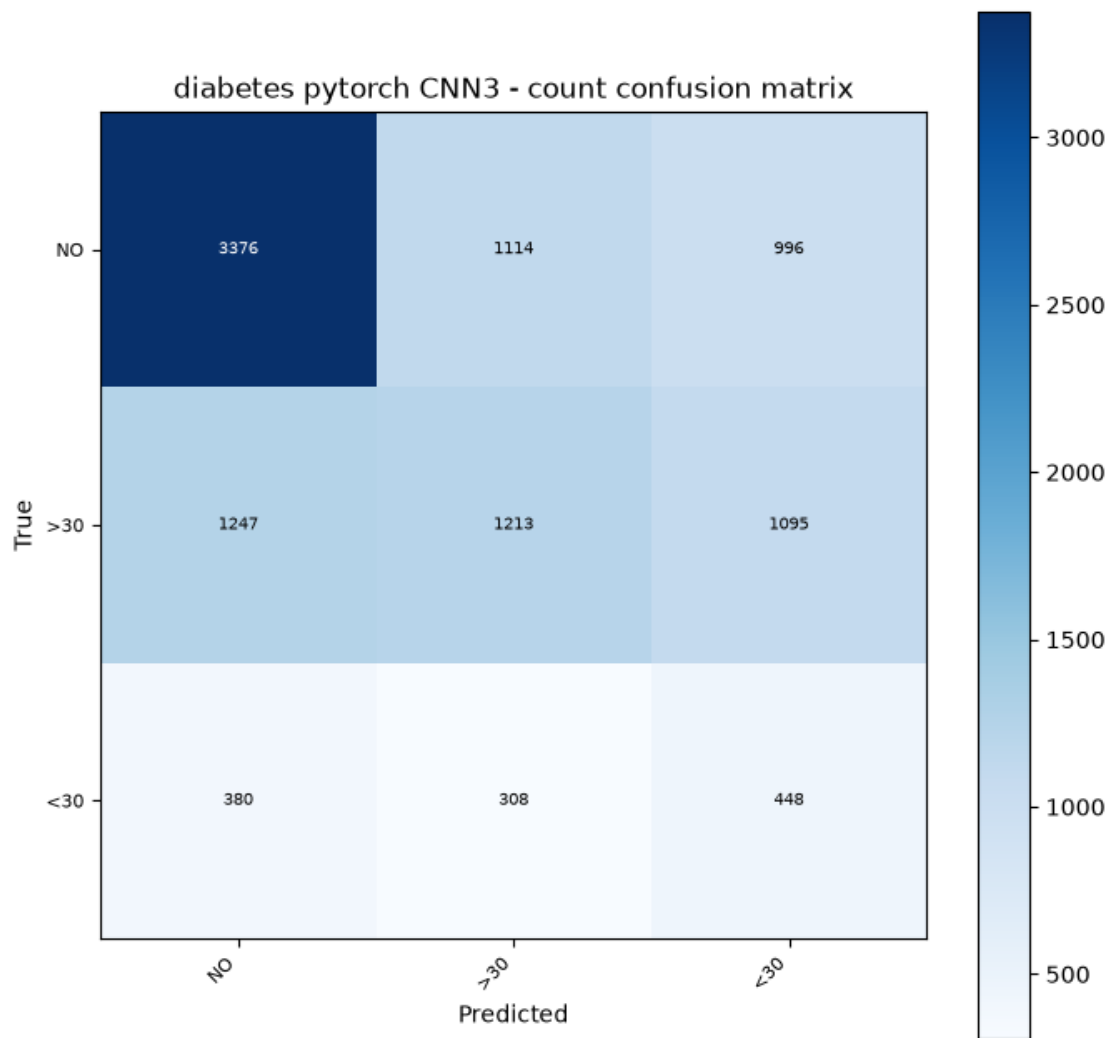
```

```
diabetes | pytorch | CNN3
```

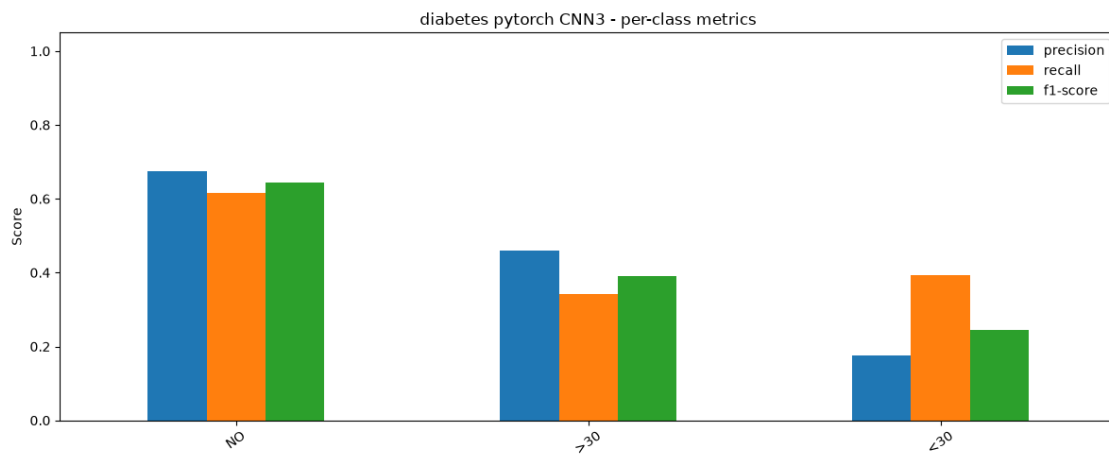
	value
accuracy	0.49494
precision_macro	0.437195
recall_macro	0.45032

f1_macro 0.426485
precision_weighted 0.544255
recall_weighted 0.49494
f1_weighted 0.511124
log_loss 1.024798
mean_confidence 0.43455
ece 0.060486
dataset diabetes
framework pytorch
conv_layers 3

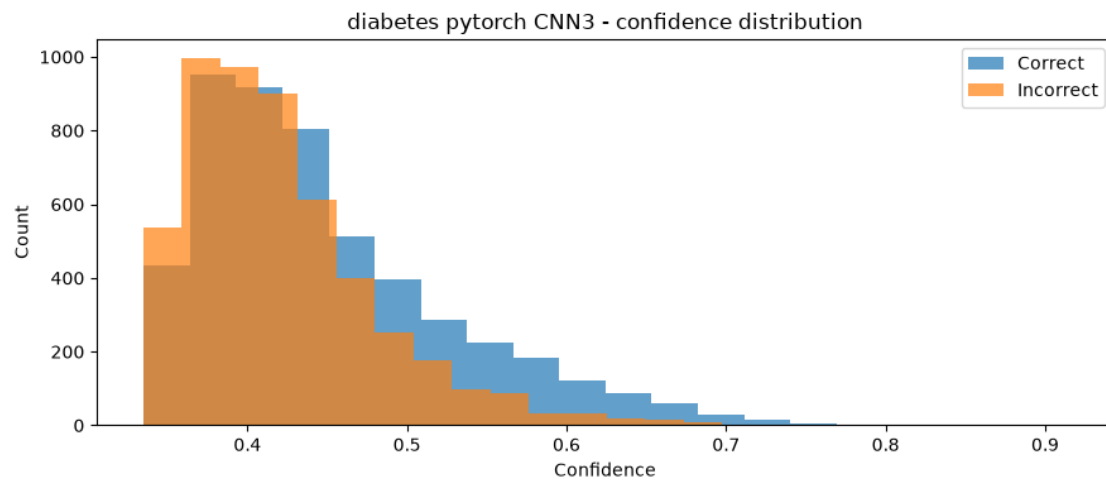


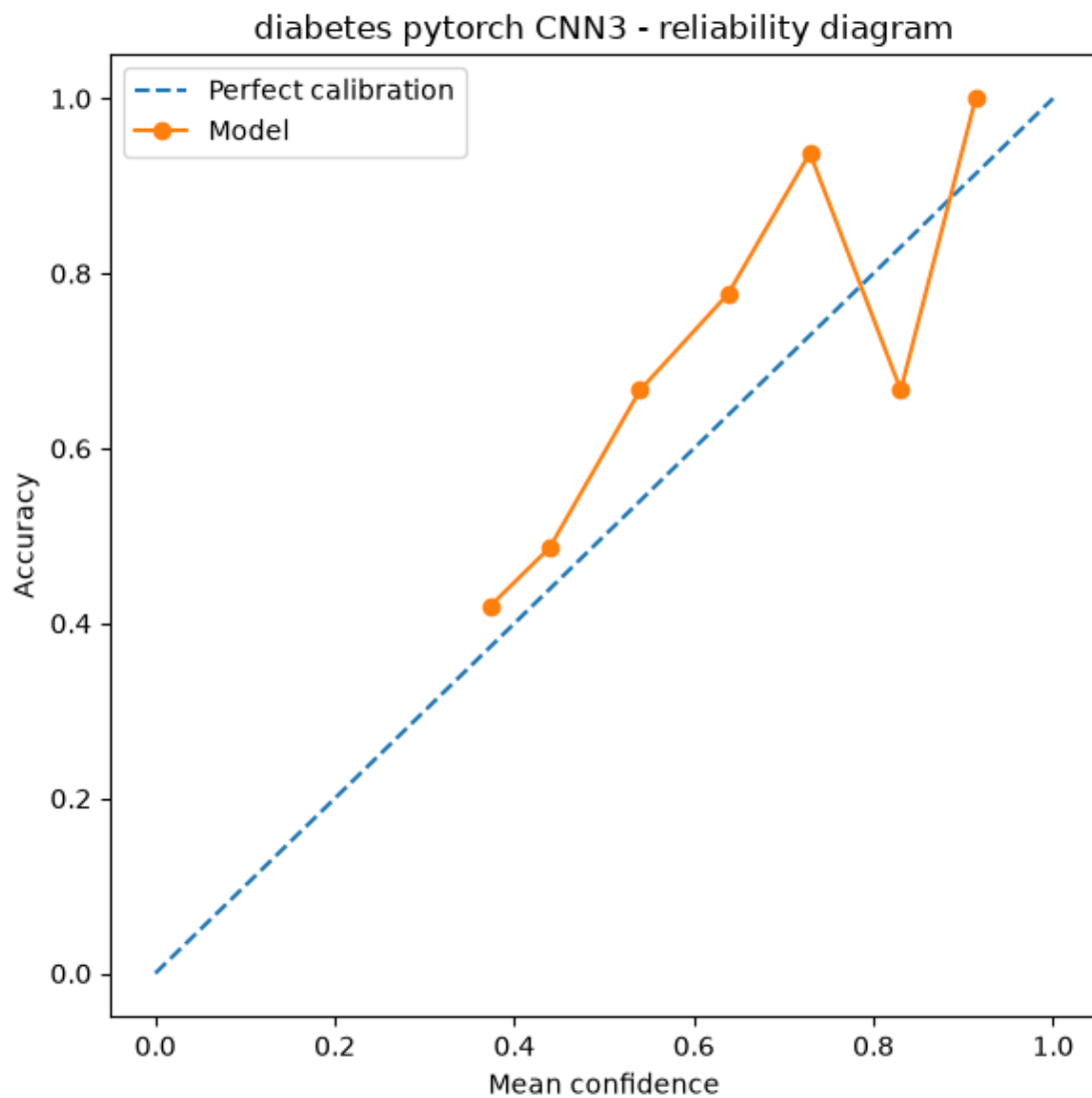


Per-class metrics:



	precision	recall	f1-score	support
NO	0.674795	0.615385	0.643722	5486.0
>30	0.460342	0.341210	0.391922	3555.0
<30	0.176447	0.394366	0.243810	1136.0





Top confusion pairs:

	count	true_class	predicted_class
0	1247	>30	NO
1	1114	NO	>30
2	1095	>30	<30
3	996	NO	<30
4	380	<30	NO
5	308	<30	>30

PyTorch Diabetes CNN5 | params: 142,915

Epoch 01 | train loss 1.0685 acc 0.4556 | val loss 1.0478 acc 0.4821

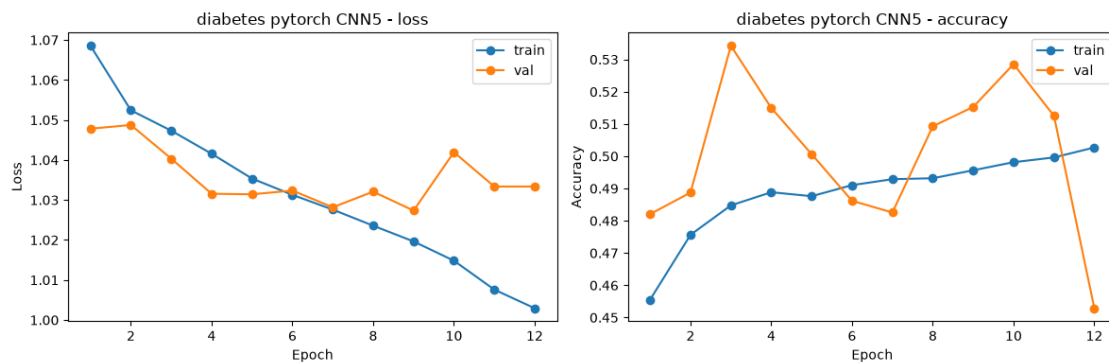
Epoch 02 | train loss 1.0524 acc 0.4756 | val loss 1.0487 acc 0.4887

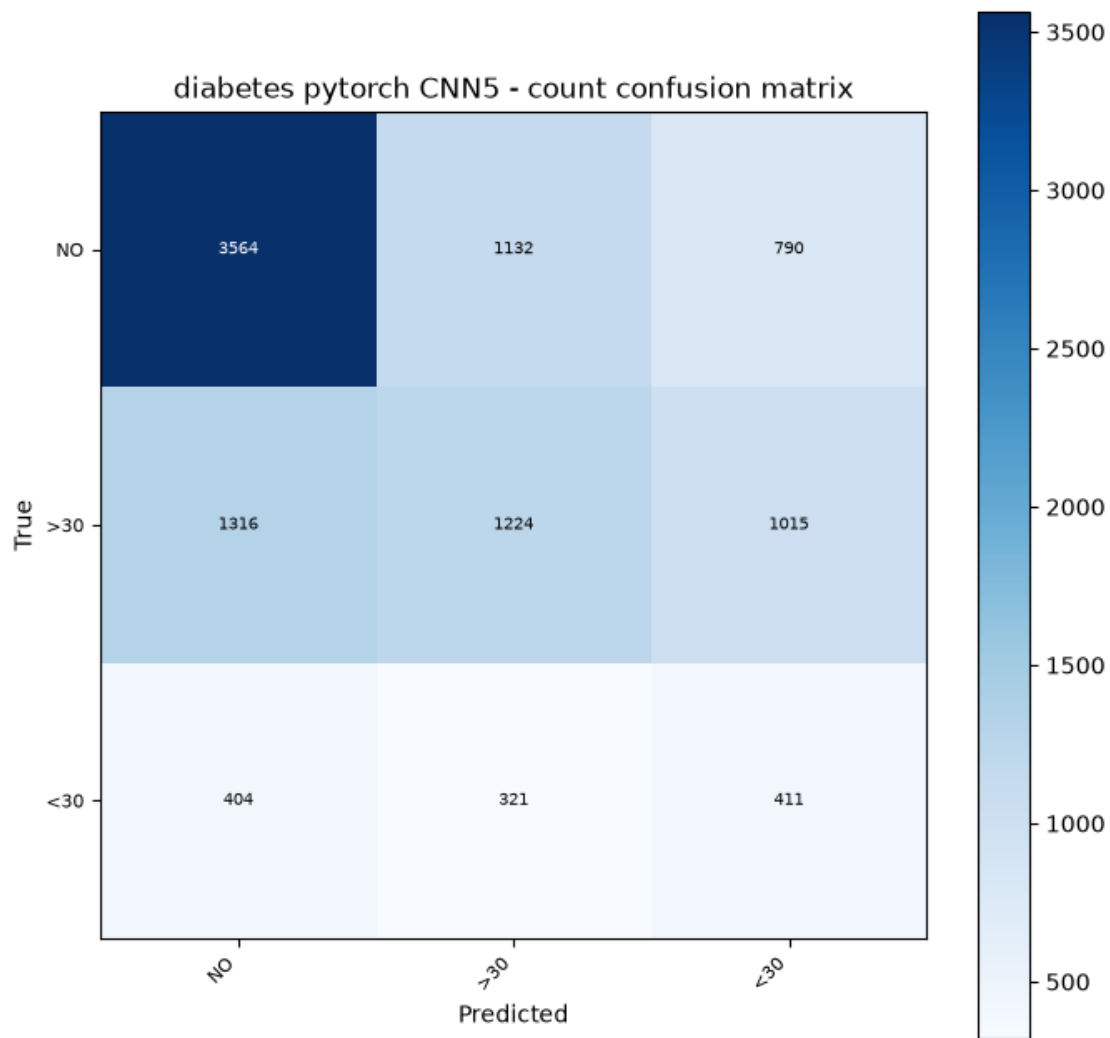
Epoch	train loss	acc	val loss	acc
Epoch 03	1.0473	0.4848	1.0403	0.5342
Epoch 04	1.0416	0.4888	1.0316	0.5150
Epoch 05	1.0354	0.4876	1.0314	0.5006
Epoch 06	1.0313	0.4910	1.0324	0.4862
Epoch 07	1.0276	0.4929	1.0282	0.4826
Epoch 08	1.0236	0.4932	1.0321	0.5093
Epoch 09	1.0196	0.4957	1.0273	0.5153
Epoch 10	1.0148	0.4982	1.0419	0.5285
Epoch 11	1.0076	0.4996	1.0334	0.5125
Epoch 12	1.0029	0.5027	1.0334	0.4529

Early stopping.

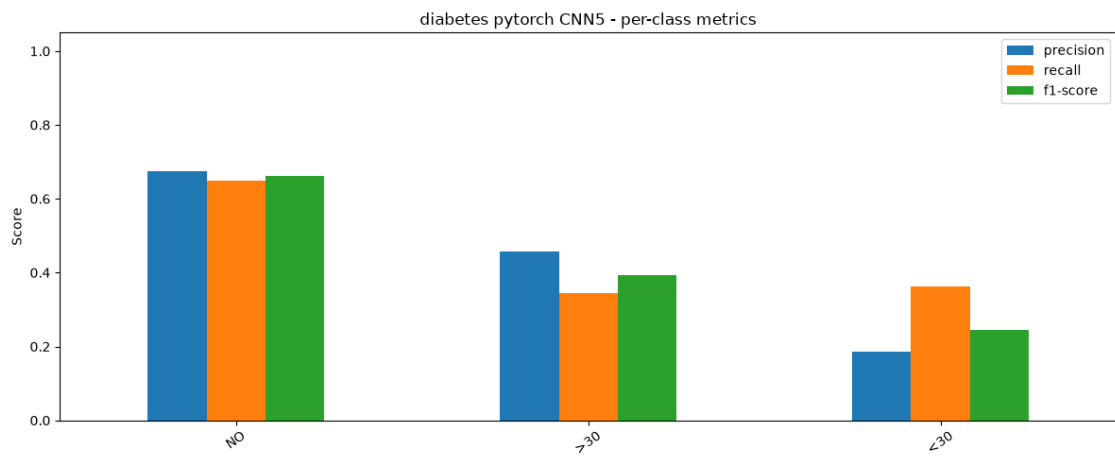
diabetes | pytorch | CNN5

	value
accuracy	0.510858
precision_macro	0.439062
recall_macro	0.451918
f1_macro	0.433292
precision_weighted	0.54401
recall_weighted	0.510858
f1_weighted	0.521359
log_loss	0.994617
mean_confidence	0.477803
ece	0.036607
dataset	diabetes
framework	pytorch
conv_layers	5

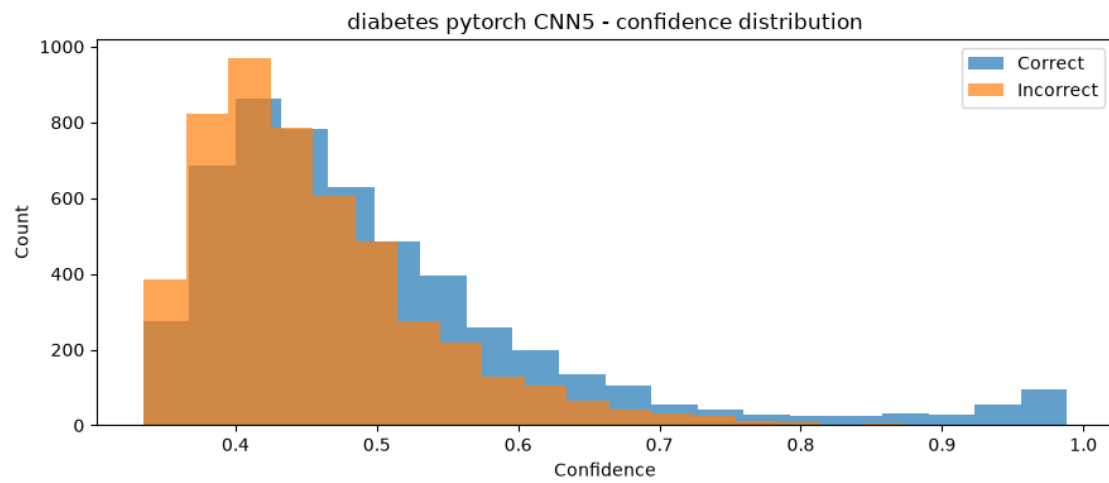


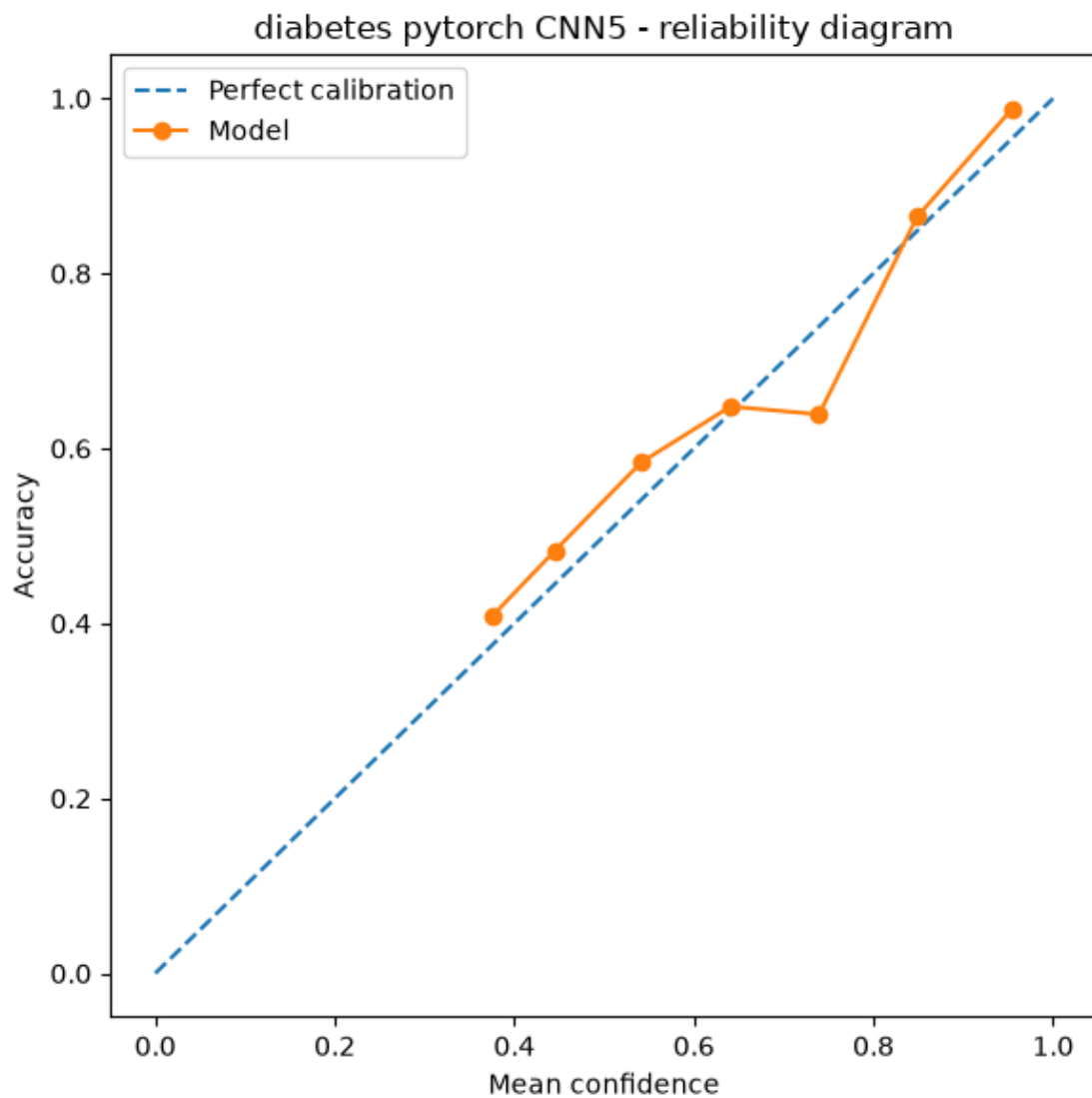


Per-class metrics:



	precision	recall	f1-score	support
NO	0.674489	0.649654	0.661838	5486.0
>30	0.457228	0.344304	0.392811	3555.0
<30	0.185469	0.361796	0.245227	1136.0





Top confusion pairs:

	count	true_class	predicted_class
0	1316	>30	NO
1	1132	NO	>30
2	1015	>30	<30
3	790	NO	<30
4	404	<30	NO
5	321	<30	>30

10 Part H - Unified comparison and technical analysis

10.1 10. Master table for the 12 experiments

The master table is the main quantitative summary of A5, but it must be read **within dataset** rather than as one global ranking.

Fashion-MNIST, CIFAR-100, and Diabetes differ in representation, class count, target imbalance, and data-generation process. Comparing their raw accuracy values as if they were one common benchmark would be misleading.

The controlled comparisons are:

Within each dataset:

CNN3 TensorFlow	CNN5 TensorFlow
CNN3 PyTorch	CNN5 PyTorch

TensorFlow CNN3	PyTorch CNN3
TensorFlow CNN5	PyTorch CNN5

The report should then connect the measurements to the architectural questions:

- Did additional depth change validation behavior?
- Did macro F1 change in the same direction as accuracy?
- Which classes benefited or deteriorated?
- Did confidence become better calibrated?
- How many additional parameters and seconds were required?

No result should be declared meaningful solely because one number is slightly larger than another.

10.1.1 10.1 Why the master table is not a leaderboard

The unified table exists to document measured trade-offs, not to rank unrelated datasets against one another. A 100-class image problem and a three-class clinical problem do not have the same statistical task.

The useful comparisons are controlled pairs: same dataset, same split, same depth or same framework. This preserves the interpretability of the experiment.

```
[29]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 50.
      records=[]
      for framework, results in [("tensorflow",TF_RESULTS),("pytorch",TORCH_RESULTS)]:
          for (dataset,depth),metrics in results.items():
              records.append(dict(metrics, framework=framework, dataset=dataset,
              ↪conv_layers=depth))

      results_df=pd.DataFrame(records)
      if not results_df.empty:
```

```

    cols=[c for c in_
↪["dataset","framework","conv_layers","accuracy","precision_macro","recall_macro","f1_macro"]
↪if c in results_df.columns]
    results_df=results_df[cols].
↪sort_values(["dataset","framework","conv_layers"])
    display(results_df)
    results_df.to_csv(METRICS_DIR/"all_experiments_summary.csv",index=False)
else:
    print("Run the training cells first.")

```

	dataset	framework	conv_layers	accuracy	precision_macro \
8	cifar100	pytorch	3	0.338400	0.370387
9	cifar100	pytorch	5	0.498100	0.528841
2	cifar100	tensorflow	3	0.343700	0.380055
3	cifar100	tensorflow	5	0.513400	0.536683
10	diabetes	pytorch	3	0.494940	0.437195
11	diabetes	pytorch	5	0.510858	0.439062
4	diabetes	tensorflow	3	0.496217	0.413964
5	diabetes	tensorflow	5	0.506633	0.419888
6	fashion	pytorch	3	0.880400	0.883201
7	fashion	pytorch	5	0.921400	0.922355
0	fashion	tensorflow	3	0.895100	0.898402
1	fashion	tensorflow	5	0.918100	0.918562

	recall_macro	f1_macro	f1_weighted	log_loss	top5_accuracy \
8	0.338400	0.317223	0.317223	2.590736	0.6578
9	0.498100	0.491420	0.491420	1.841280	0.8003
2	0.343700	0.336275	0.336275	2.566965	0.6577
3	0.513400	0.512011	0.512011	1.796770	0.8112
10	0.450320	0.426485	0.511124	1.024798	NaN
11	0.451918	0.433292	0.521359	0.994617	NaN
4	0.423122	0.397271	0.488304	1.034184	NaN
5	0.430219	0.412083	0.504001	1.021818	NaN
6	0.880400	0.877225	0.877225	0.342884	0.9971
7	0.921400	0.920566	0.920566	0.232490	0.9994
0	0.895100	0.895753	0.895753	0.294354	0.9978
1	0.918100	0.918172	0.918172	0.229283	0.9992

	mean_confidence	ece	parameters	training_seconds
8	0.269867	0.068533	106372	1055.970271
9	0.499991	0.013712	441924	1678.603299
2	0.286036	0.057664	106820	514.790900
3	0.523270	0.011540	442948	784.561548
10	0.434550	0.060486	31651	213.122202
11	0.477803	0.036607	142915	355.620783
4	0.409995	0.087090	32099	36.448085
5	0.425098	0.081683	143939	69.052236

6	0.894253	0.014683	94186	857.567616
7	0.945136	0.023736	426858	1537.701505
0	0.899210	0.004630	94634	449.085554
1	0.934951	0.017132	427882	604.081252

10.2 10.0 Fair-comparison checklist before interpreting results

Lecture 04 states the central condition for a meaningful implementation comparison:

Same Dataset + Same Split + Same Architecture + Comparable Hyperparameters

A5 implements that principle as follows.

Factor	Controlled decision
Dataset source	Fixed per dataset
Train/validation membership	Shared within each dataset
Test set	Held out until final evaluation
CNN depth	Exactly 3 or 5 convolutional layers
Filter schedule	Same conceptual schedule across frameworks
Kernel size	3
Pooling strategy	Same broad placement by depth
BatchNorm	Enabled in both frameworks
Dropout	0.30 in both frameworks
Initial learning rate	0.001
Optimizer	Adam
Maximum epochs	12
Model selection	Lowest validation loss
Loss	Sparse multiclass cross-entropy
Class weights	Same training-derived policy where required
Evaluation functions	Same framework-independent NumPy/sklearn utilities

The purpose of this checklist is not to claim perfect experimental identity. TensorFlow and PyTorch have different kernels, data-loader systems, and numerical execution paths. It is to make clear which differences are intentionally controlled and which are inherent to the implementation frameworks.

```
[30]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 51.
      if not results_df.empty:
          for dataset in results_df["dataset"].unique():
              sub=results_df[results_df["dataset"]==dataset]
              labels=[f"{r.framework}-CNN{r.conv_layers}" for r in sub.itertuples()]

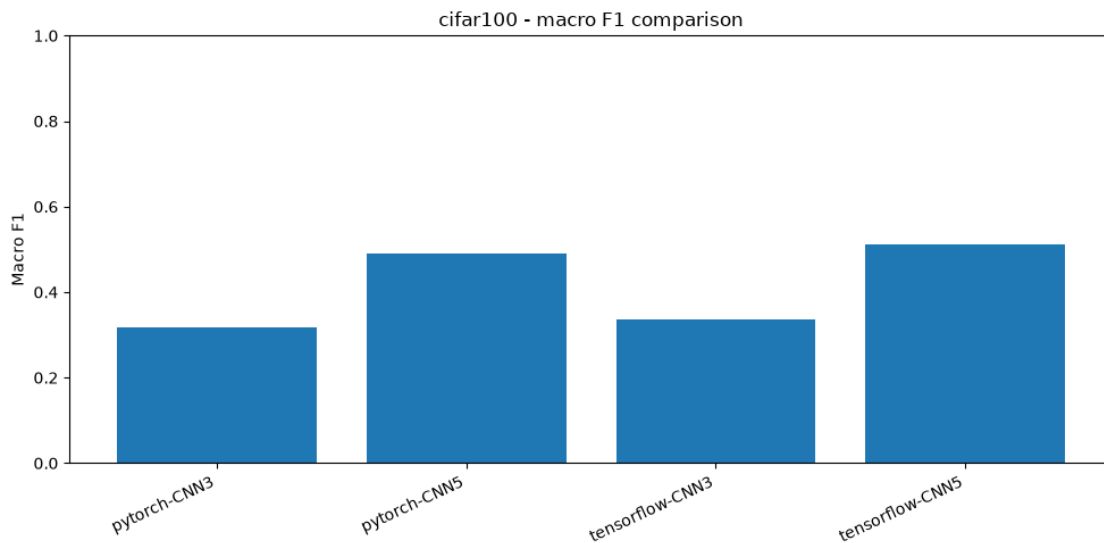
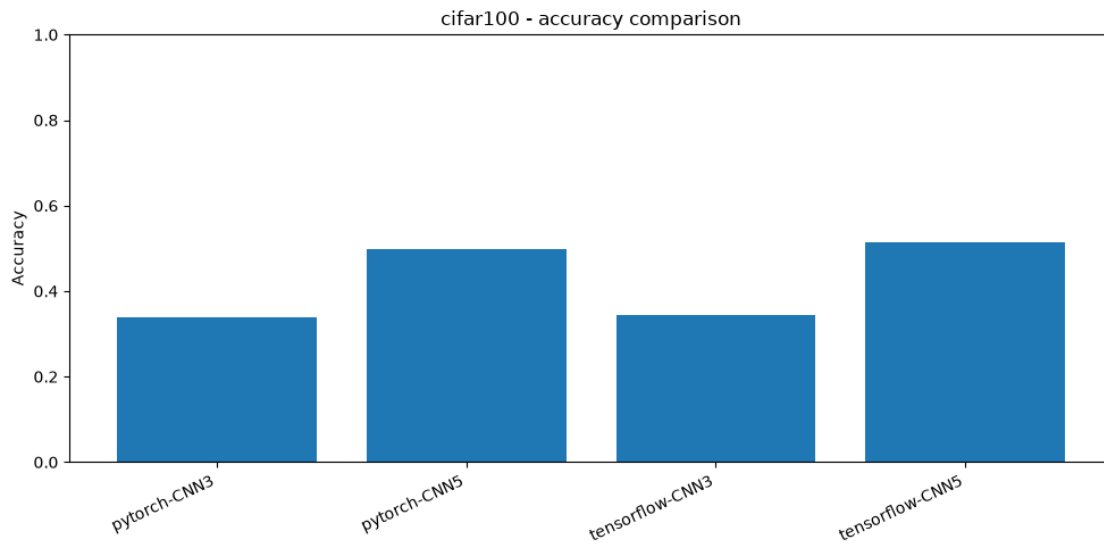
              plt.figure(figsize=(10,5)); plt.bar(labels,sub["accuracy"]); plt.
      ↪ ylim(0,1); plt.ylabel("Accuracy")
              plt.title(dataset+" - accuracy comparison"); plt.
      ↪ xticks(rotation=25,ha="right"); plt.tight_layout(); plt.show()
```

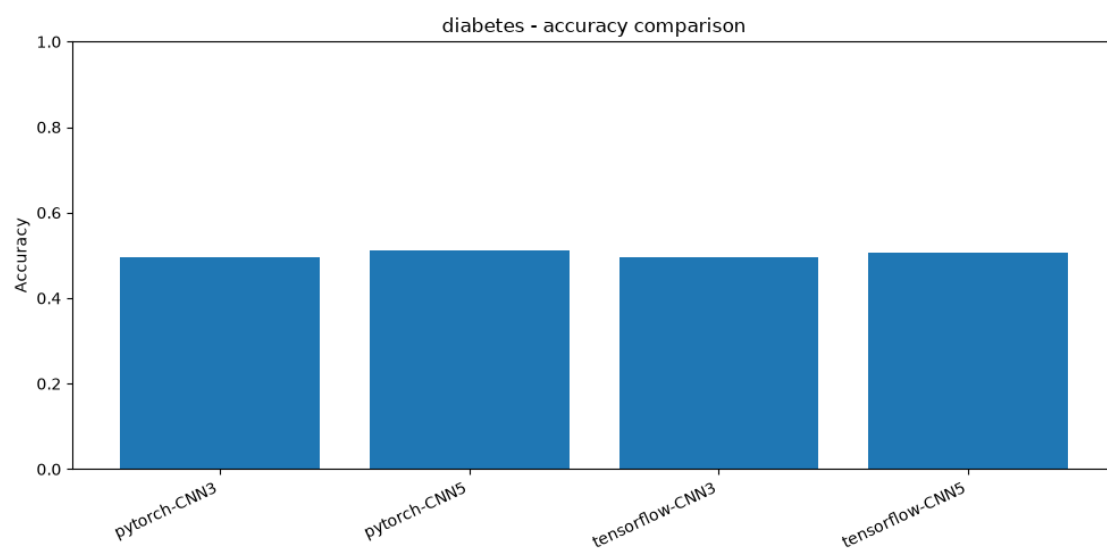
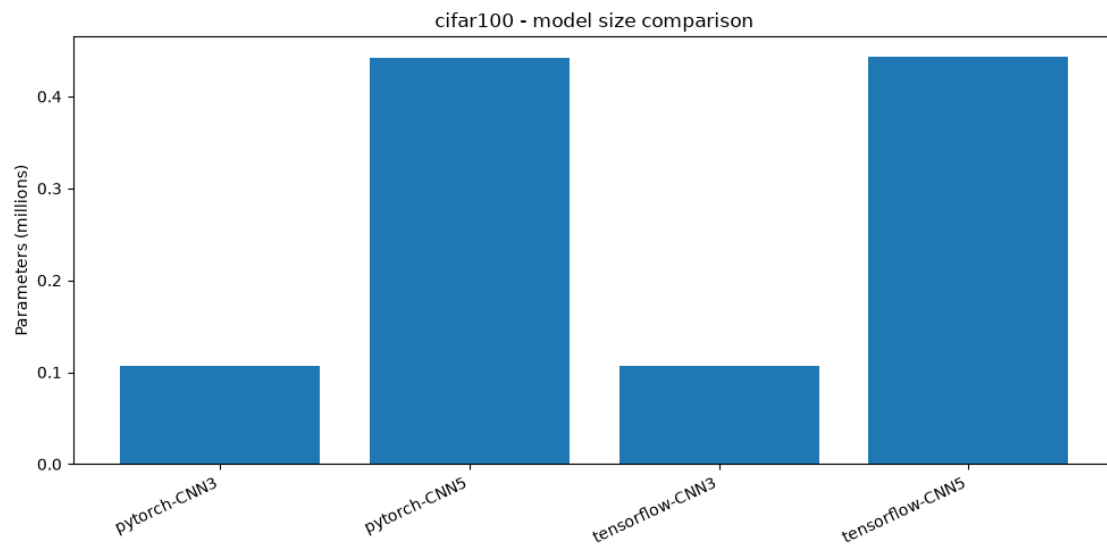
```

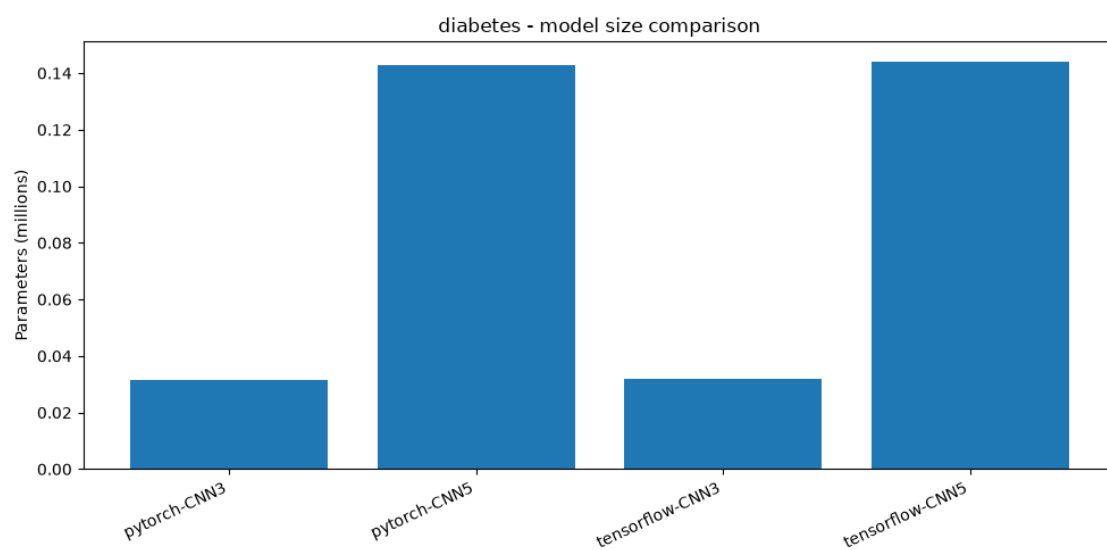
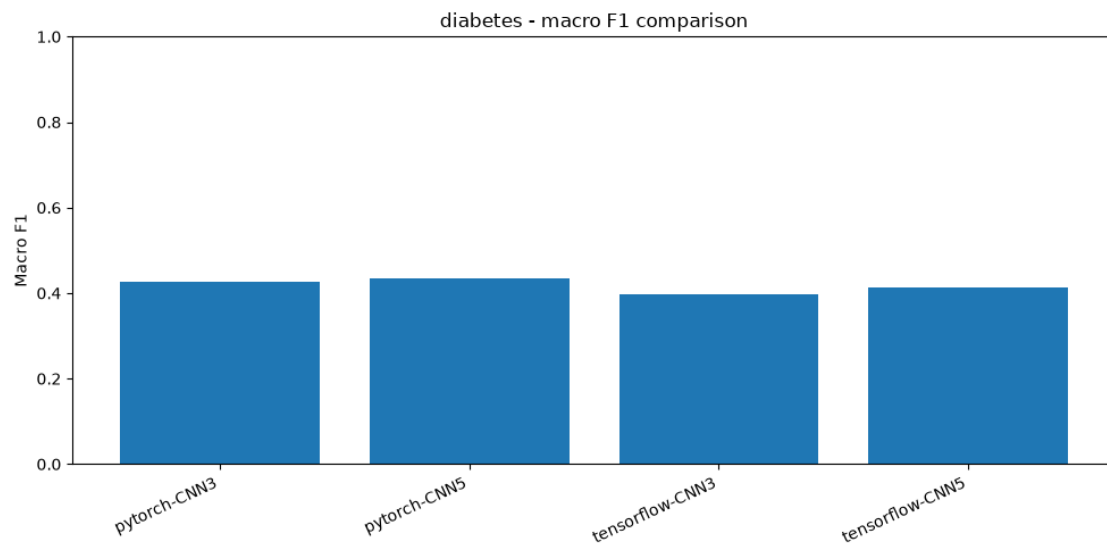
plt.figure(figsize=(10,5)); plt.bar(labels,sub["f1_macro"]); plt.
→ylim(0,1); plt.ylabel("Macro F1")
plt.title(dataset+" - macro F1 comparison"); plt.
→xticks(rotation=25,ha="right"); plt.tight_layout(); plt.show()

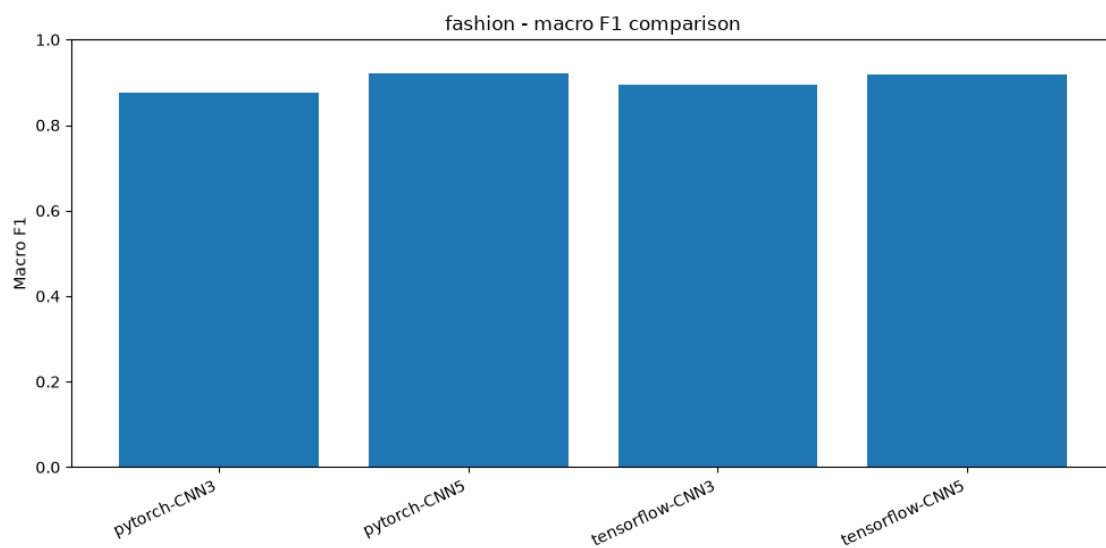
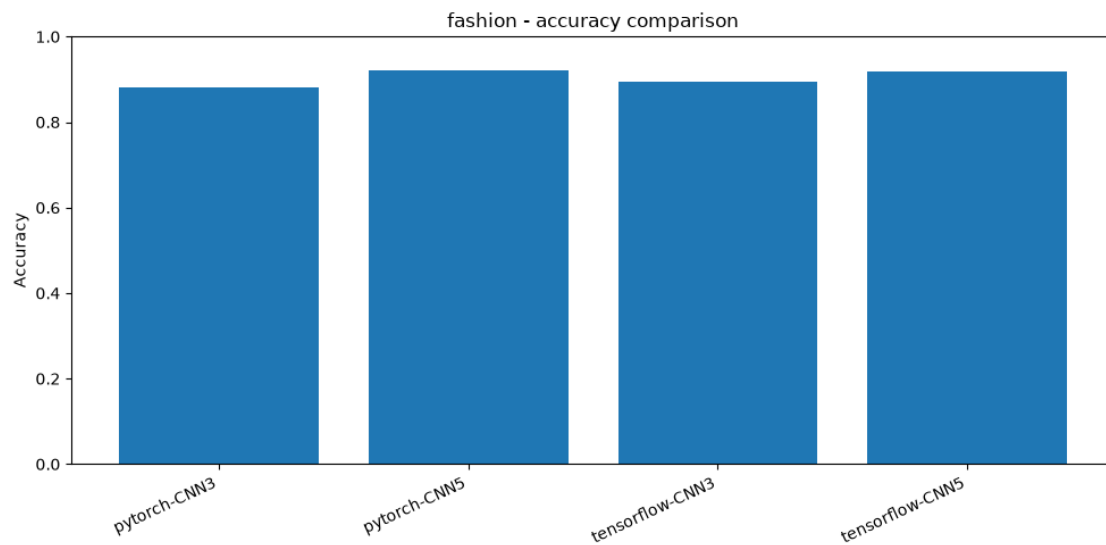
plt.figure(figsize=(10,5)); plt.bar(labels,sub["parameters"]/1e6); plt.
→ylabel("Parameters (millions)")
plt.title(dataset+" - model size comparison"); plt.
→xticks(rotation=25,ha="right"); plt.tight_layout(); plt.show()

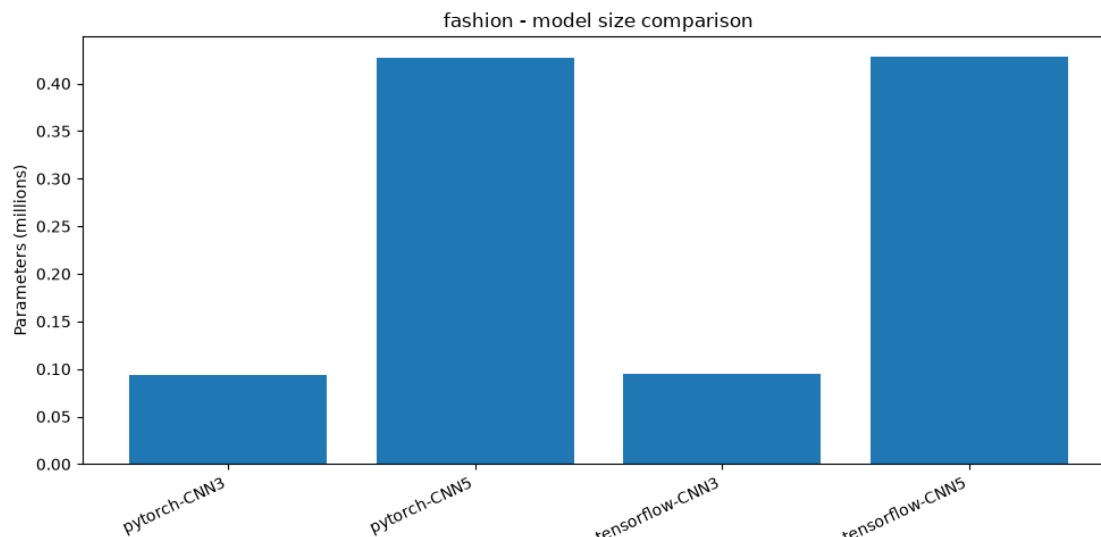
```











10.3 10.0.1 How to interpret a result correctly

Suppose one experiment reports 0.89 accuracy and another reports 0.90. The difference is not automatically meaningful. A disciplined interpretation checks the surrounding evidence.

10.3.1 First: inspect validation behavior

If training loss continues to fall while validation loss rises, the model may be fitting the training data faster than it is improving generalization. Early stopping and checkpoint restoration are intended to reduce this problem.

10.3.2 Second: inspect class-level behavior

A global accuracy improvement can occur because a large class became easier while a smaller class became worse. Macro F1 and per-class recall make that visible.

10.3.3 Third: inspect the confusion matrix

Two models can have identical accuracy while making very different mistakes. One model may spread errors across many classes; another may repeatedly confuse the same pair.

10.3.4 Fourth: inspect probability quality

A wrong prediction at 0.51 confidence and a wrong prediction at 0.99 confidence have the same top-1 outcome but very different implications for reliability. Log loss, confidence distributions, reliability diagrams, and ECE help expose this difference.

10.3.5 Fifth: inspect computational cost

Extra depth is a trade-off. A small quality improvement may come with a large increase in parameters and runtime. The report should quantify that trade-off instead of describing depth as

automatically beneficial.

10.4 10.1 Reading the results like a technical practitioner

The order of analysis below is recommended because it moves from optimization behavior to generalization, then to class-level errors, and finally to computational cost.

10.4.1 1. Training and validation curves

Look for the relationship between training and validation loss. A widening gap can indicate overfitting. A plateau in both curves can indicate that the model has exhausted the useful optimization progress under the current configuration.

10.4.2 2. Accuracy together with macro F1

Accuracy describes overall correctness. Macro F1 asks whether the model is performing consistently across classes. For an imbalanced clinical target, this distinction becomes particularly important.

10.4.3 3. Per-class recall

Recall tells us what fraction of the true examples of each class were detected. A class with low recall is a concrete error pattern that should be discussed even if the overall score looks strong.

10.4.4 4. Confusion matrix

The matrix reveals the direction of each error. In a clinical dataset, confusing NO with >30 is not the same pattern as confusing NO with <30; in an image dataset, confusing Pullover with Coat is different from confusing Sneaker with Bag.

10.4.5 5. Confidence and calibration

A reliable classifier should not routinely behave as though it is certain when it is wrong. Calibration diagnostics provide evidence about this behavior.

10.4.6 6. Parameter count and time

The final question is computational efficiency. Deeper models naturally tend to require more operations, but the exact relationship depends on where pooling occurs and on implementation details. The table records the actual measured training time rather than assuming the relationship.

11 Part I - Error analysis

11.1 11. Error analysis: from score to explanation

A final metric is a summary, not an explanation. Error analysis examines the remaining mistakes and asks whether they form a meaningful pattern.

11.1.1 Fashion-MNIST

The notebook visualizes misclassified clothing images together with the true label, predicted label, and confidence. Repeated confusions between visually similar upper-body categories can indicate that the task contains genuine visual ambiguity rather than a coding error.

11.1.2 CIFAR-100

The 100-class setting benefits from two levels of analysis:

1. the full numeric confusion matrix, which preserves all evidence;
2. the top confusion pairs, which provide a readable ranking of the most frequent class-to-class errors.

The correct interpretation is not “the model failed because the matrix is large.” A 100 x 100 matrix is large because the task itself contains 100 classes.

11.1.3 Diabetes

The clinical dataset is analyzed through target-specific precision, recall, F1, confusion counts, confidence, and calibration. The notebook avoids claiming that a model is clinically useful; this assignment evaluates classification behavior on the supplied dataset, not a clinical deployment system.

11.1.4 High-confidence errors

A particularly useful subset is the set of wrong predictions with high confidence. These examples can indicate insufficient representation, ambiguous samples, or a mismatch between the learned decision boundary and the data distribution.

Error analysis should remain descriptive. It is evidence about model behavior, not a license to invent causes that the dataset cannot support.

11.1.5 11.1 Error-analysis checklist

For every model, ask:

1. Which classes are hardest?
2. Are the hardest classes visually or statistically similar?
3. Are errors concentrated in one direction?
4. Does the deeper model change the error pattern or only the aggregate score?
5. Are wrong predictions unusually confident?
6. Does the framework change the class-level behavior even when the architecture is conceptually the same?

These questions force the report to explain model behavior instead of merely copying the final accuracy value.

```
[31]: # Purpose: Run the computation defined by this section. The detailed comments
      ↪ below explain the role of each operation.
      # Original section purpose is described in the preceding Markdown cell. 54.
      if not results_df.empty:
```

```

print("Final comparison snapshot:")
display(results_df.round(4))
for dataset in results_df["dataset"].unique():
    print(f"\nDepth comparison for {dataset}")
    view=results_df[results_df["dataset"]==dataset].
    ↪pivot_table(index="framework",columns="conv_layers",values=["accuracy","f1_macro"],aggfunc=
        display(view.round(4))

# Save the run configuration so the final report is reproducible.
run_config={
    "seed":SEED,"run_mode":RUN_MODE,"epochs":EPOCHS,"batch_size":BATCH_SIZE,
    "learning_rate":LEARNING_RATE,"device":str(DEVICE),"python":platform.
    ↪python_version(),
}
with open(METRICS_DIR/"run_config.json","w",encoding="utf-8") as f:
    json.dump(run_config,f,indent=2)
print(json.dumps(run_config,indent=2))

```

Final comparison snapshot:

	dataset	framework	conv_layers	accuracy	precision_macro	\
8	cifar100	pytorch	3	0.3384	0.3704	
9	cifar100	pytorch	5	0.4981	0.5288	
2	cifar100	tensorflow	3	0.3437	0.3801	
3	cifar100	tensorflow	5	0.5134	0.5367	
10	diabetes	pytorch	3	0.4949	0.4372	
11	diabetes	pytorch	5	0.5109	0.4391	
4	diabetes	tensorflow	3	0.4962	0.4140	
5	diabetes	tensorflow	5	0.5066	0.4199	
6	fashion	pytorch	3	0.8804	0.8832	
7	fashion	pytorch	5	0.9214	0.9224	
0	fashion	tensorflow	3	0.8951	0.8984	
1	fashion	tensorflow	5	0.9181	0.9186	

	recall_macro	f1_macro	f1_weighted	log_loss	top5_accuracy	\
8	0.3384	0.3172	0.3172	2.5907	0.6578	
9	0.4981	0.4914	0.4914	1.8413	0.8003	
2	0.3437	0.3363	0.3363	2.5670	0.6577	
3	0.5134	0.5120	0.5120	1.7968	0.8112	
10	0.4503	0.4265	0.5111	1.0248	NaN	
11	0.4519	0.4333	0.5214	0.9946	NaN	
4	0.4231	0.3973	0.4883	1.0342	NaN	
5	0.4302	0.4121	0.5040	1.0218	NaN	
6	0.8804	0.8772	0.8772	0.3429	0.9971	
7	0.9214	0.9206	0.9206	0.2325	0.9994	
0	0.8951	0.8958	0.8958	0.2944	0.9978	
1	0.9181	0.9182	0.9182	0.2293	0.9992	

	mean_confidence	ece	parameters	training_seconds
8	0.2699	0.0685	106372	1055.9703
9	0.5000	0.0137	441924	1678.6033
2	0.2860	0.0577	106820	514.7909
3	0.5233	0.0115	442948	784.5615
10	0.4346	0.0605	31651	213.1222
11	0.4778	0.0366	142915	355.6208
4	0.4100	0.0871	32099	36.4481
5	0.4251	0.0817	143939	69.0522
6	0.8943	0.0147	94186	857.5676
7	0.9451	0.0237	426858	1537.7015
0	0.8992	0.0046	94634	449.0856
1	0.9350	0.0171	427882	604.0813

Depth comparison for cifar100

	accuracy		f1_macro	
conv_layers	3	5	3	5
framework				
pytorch	0.3384	0.4981	0.3172	0.4914
tensorflow	0.3437	0.5134	0.3363	0.5120

Depth comparison for diabetes

	accuracy		f1_macro	
conv_layers	3	5	3	5
framework				
pytorch	0.4949	0.5109	0.4265	0.4333
tensorflow	0.4962	0.5066	0.3973	0.4121

Depth comparison for fashion

	accuracy		f1_macro	
conv_layers	3	5	3	5
framework				
pytorch	0.8804	0.9214	0.8772	0.9206
tensorflow	0.8951	0.9181	0.8958	0.9182

```
{
  "seed": 42,
  "run_mode": "full",
  "epochs": 12,
  "batch_size": 128,
  "learning_rate": 0.001,
  "device": "cpu",
  "python": "3.12.13"
}
```

12 12. Technical conclusion

12.1 12.1 What A5 demonstrates at the data level

The three datasets require different data reasoning before CNN training begins.

Fashion-MNIST is a compact grayscale image problem where the main preparation concerns shape, numeric range, normalization, and class integrity.

CIFAR-100 is an RGB image problem with a much larger label space. The main additional challenge is the number of classes and the resulting need for richer evaluation, especially top-5 accuracy and class-level confusion analysis.

Diabetes 130-US Hospitals is a large heterogeneous clinical table. Its main preparation challenges are missing values, mixed data types, repeated patients, target imbalance, and leakage prevention.

This contrast is important because the quality of a neural-network experiment is influenced by data preparation decisions long before the first gradient update occurs.

12.2 12.2 What A5 demonstrates at the architecture level

CNN3 and CNN5 share the same fundamental operations but differ in depth and representational capacity.

The added convolutional stages in CNN5 allow the network to apply more successive transformations before classification. That can be useful when the dataset contains more complex visual structure, but greater depth also increases computation and can change optimization behavior.

The correct conclusion therefore comes from the measured evidence:

Depth change
↓
Parameter count
↓
Training dynamics
↓
Validation behavior
↓
Test metrics
↓
Confusion pattern
↓
Confidence / calibration
↓
Runtime trade-off

12.3 12.3 What A5 demonstrates about TensorFlow and PyTorch

TensorFlow/Keras raises the abstraction level through reusable layers and high-level training APIs such as `compile()` and `fit()`.

PyTorch keeps the model structure explicit through `nn.Module` and `forward()`, while exposing the optimization cycle through `zero_grad()`, `loss.backward()`, and `optimizer.step()`.

The computational idea remains:

```
Input
↓
Transformations
↓
Prediction
↓
Loss
↓
Gradient
↓
Parameter update
```

The programming abstraction changes, not the underlying learning objective.

12.4 12.4 What should and should not be concluded from the Diabetes Conv1D experiment

The A5 requirement forces a CNN experiment on Diabetes, so Conv1D provides a consistent way to study CNN mechanics on the third dataset. However, the report should retain the limitation emphasized in Lecture 04: ordinary tabular columns do not naturally form a spatial neighborhood like pixels in an image.

Therefore the Diabetes Conv1D result is best interpreted as an architectural experiment under the assignment constraint. It should not be presented as evidence that Conv1D is universally the correct architecture for clinical tabular data.

12.5 12.5 Final reporting principle

The strongest technical report is not the one that contains the largest number of decimals. It is the one where every important number is connected to:

- a clear data pipeline;
- a documented architecture;
- controlled experimental settings;
- appropriate metrics;
- visual evidence;
- an explanation of errors and trade-offs.

The final conclusion of A5 should therefore be written after the full run and should refer directly to the observed results rather than to expectations established before training.

```
[32]: # Purpose: Save the final experiment configuration and the exact
      ↪ model-comparison table for reproducibility.
      if 'results_df' in globals() and not results_df.empty:
          results_df.to_csv(METRICS_DIR / "all_experiments_summary.csv", index=False)

      run_config = {
          "seed": SEED,
          "run_mode": RUN_MODE,
```

```

    "epochs": EPOCHS,
    "batch_size": BATCH_SIZE,
    "learning_rate": LEARNING_RATE,
    "device": str(DEVICE) if 'DEVICE' in globals() else "not_initialized",
    "python": platform.python_version(),
    "tensorflow": tf.__version__ if 'tf' in globals() else None,
    "pytorch": torch.__version__ if 'torch' in globals() else None,
}

# Store configuration as JSON so the final results can be traced to the run
↳ settings.
with open(METRICS_DIR / "run_config.json", "w", encoding="utf-8") as f:
    json.dump(run_config, f, indent=2)

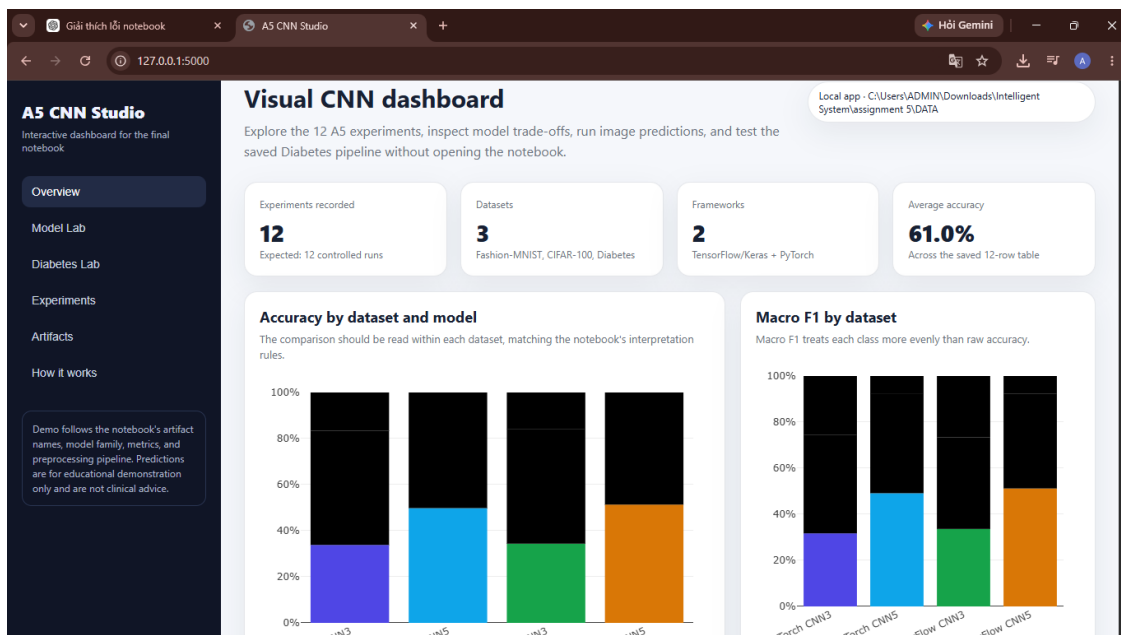
print(json.dumps(run_config, indent=2))

```

```

{
    "seed": 42,
    "run_mode": "full",
    "epochs": 12,
    "batch_size": 128,
    "learning_rate": 0.001,
    "device": "cpu",
    "python": "3.12.13",
    "tensorflow": "2.21.0",
    "pytorch": "2.14.0+cpu"
}

```



Giải thích lỗi notebook

A5 CNN Studio

Hỏi Gemini

127.0.0.1:5000

A5 CNN Studio

Interactive dashboard for the final notebook

Overview

Model Lab

Diabetes Lab

Experiments

Artifacts

How it works

Demo follows the notebook's artifact names, model family, metrics, and preprocessing pipeline. Predictions are for educational demonstration only and are not clinical advice.

Visual CNN dashboard

Explore the 12 A5 experiments, inspect model trade-offs, run image predictions, and test the saved Diabetes pipeline without opening the notebook.

Local app · C:\Users\ADMIN\Downloads\Intelligent System\assignment 5\DATA

Image model playground

Upload an image and run one of the saved Fashion-MNIST or CIFAR-100 models. The app applies the same basic 0-1 image scaling used in the notebook, then shows the top predictions and confidence.

Dataset

Fashion-MNIST

Framework

TensorFlow / Keras

CNN depth

CNN5

Image file

Chọn tệp

Không có t... được chọn

Run prediction

CNN3 architecture

Three convolutional layers. The notebook does not count BatchNorm, ReLU, pooling, dropout, or the classifier as convolutional depth.

Input ↓ Conv 32 → BatchNorm → ReLU → Pool ↓ Conv 64 → BatchNorm → ReLU → Pool ↓ Conv 128 → BatchNorm → ReLU ↓ Global Average Pooling ↓ Dropout(0.30) ↓ Dense / Linear classifier

CNN5 architecture

Adds two more convolutional transformations and one extra pooling stage.

Input ↓ Conv 32 → BatchNorm → ReLU → Pool ↓ Conv 64 → BatchNorm → ReLU → Pool ↓ Conv 128 → BatchNorm → ReLU → Pool ↓ Conv 128 → BatchNorm → ReLU → Pool ↓ Conv 160 → BatchNorm → ReLU ↓ Global Average Pooling ↓ Dropout(0.30) ↓ Dense / Linear

saved Diabetes pipeline without opening the notebook.

Image model playground

Upload an image and run one of the saved Fashion-MNIST or CIFAR-100 models. The app applies the same basic 0-1 image scaling used in the notebook, then shows the top predictions and confidence.

Dataset

Fashion-MNIST

Framework

TensorFlow / Keras

CNN depth

CNN5

Image file

Chọn tệp

76f34e6f7...-origin.jpeg

Run prediction

Prediction

✓

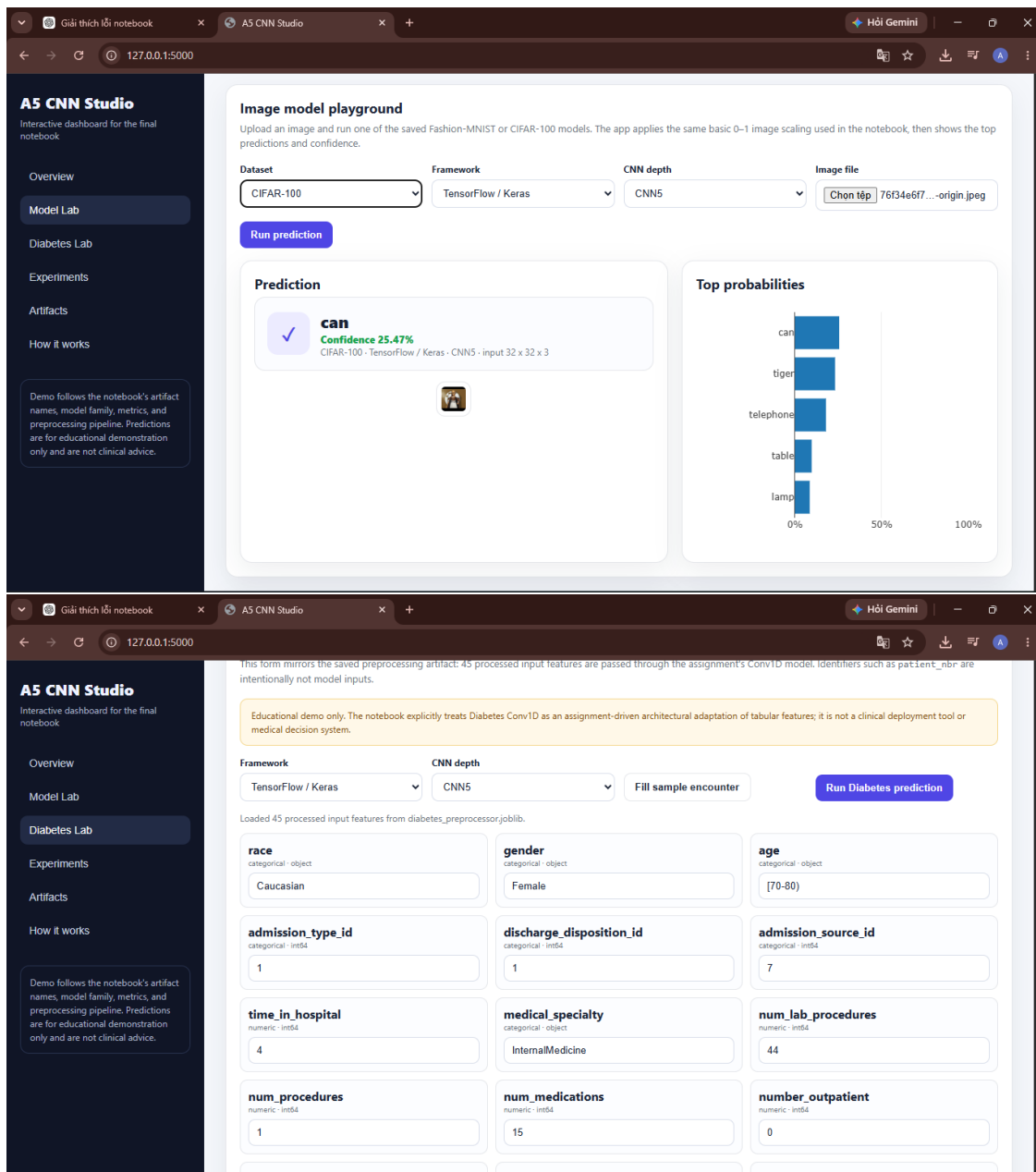
T-shirt/top

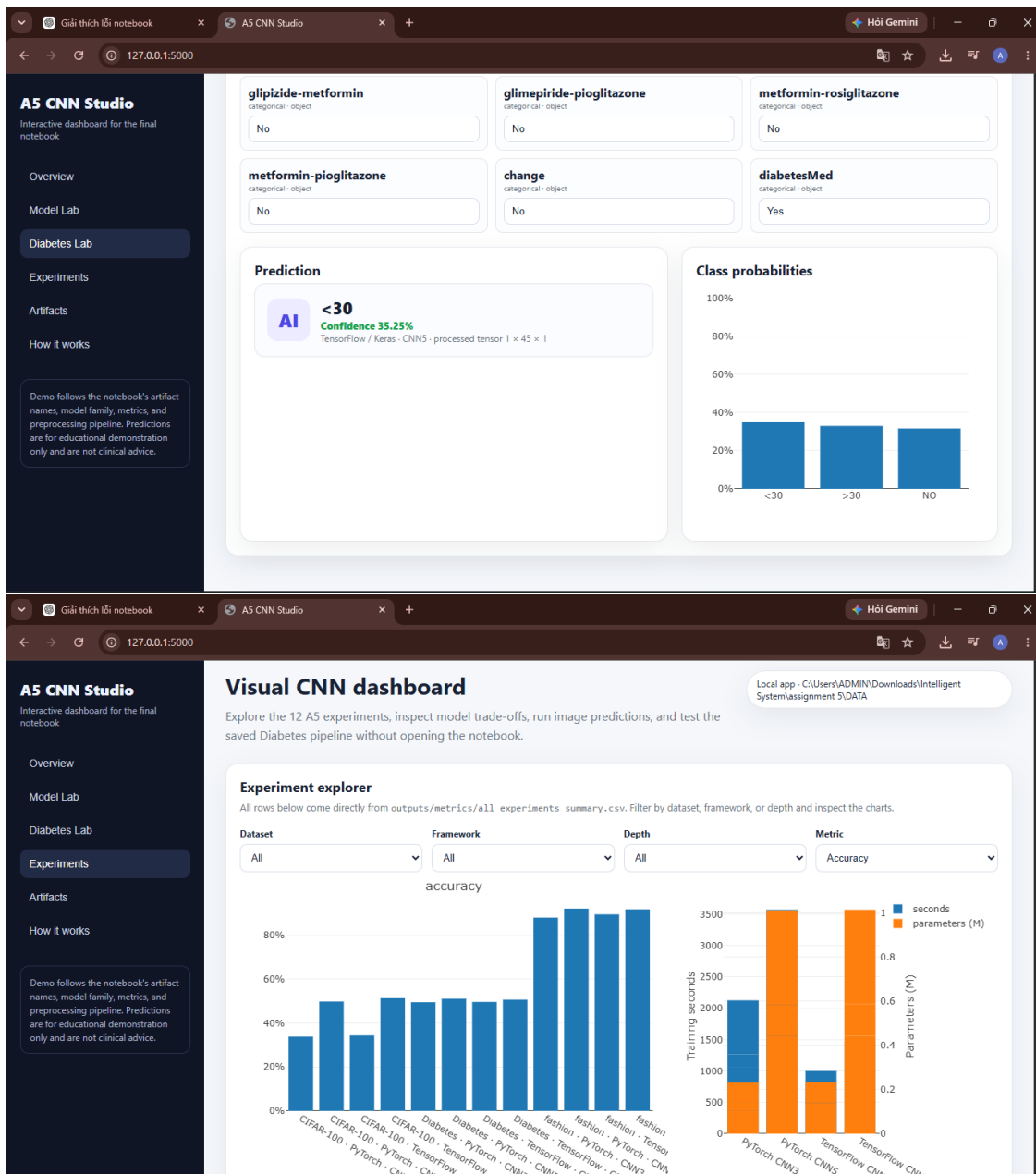
Confidence 51.83%

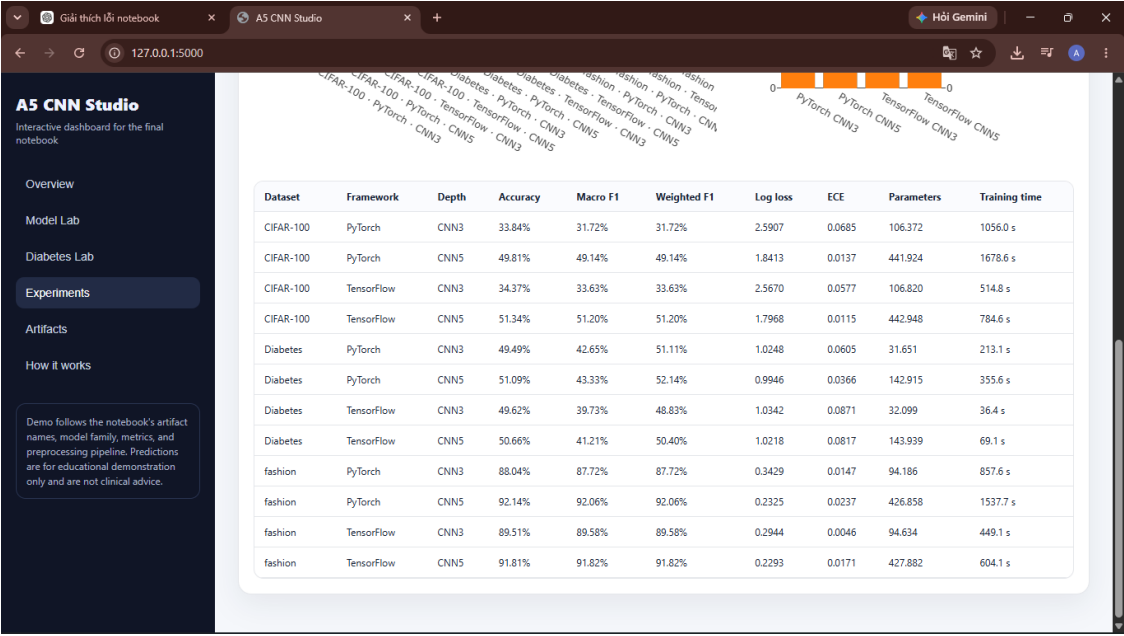
Fashion-MNIST · TensorFlow / Keras · CNN5 · input 28 x 28 x 1

Top probabilities

168







[]: